

Macroeconomic Foundation of Monetary Accounting by Diagrams of Categorical Universals

Renée Menéndez, Viktor Winschel*

August 21, 2025

Abstract

We present a category theoretical formulation of the Monetary Macroeconomic Accounting Theory (MoMaT) of Menéndez and Winschel [2025]. We take macroeconomic (national) accounting systems to be composed from microeconomic double-entry systems with real and monetary units of accounts. Category theory is the compositional grammar and module system of mathematics which we use to lift micro accounting consistency to the macro level. The main function of money in MoMaT is for the repayment of loans and not for the exchange of goods, bridging the desynchronisation of input and output payments of producers. Accordingly, temporal accounting consistency is at the macroeconomic level. We show that the accounting for macroeconomies organised by a division of labor can be consistent and stable as a prerequisite for risk and GDP sharing of societies. We exemplify the theory by five sectoral agents of Labor and Resource owners, a Company as the productive sector, a Capitalist for profits, and a Bank as the financial sector providing loans to synchronise the micro and the macro levels of an economy. The dynamics is described by eight sectoral macroeconomic bookings in each period demonstrating stable convergence of the MoMaT in numerical simulations. The categorical program implements a consistent evolution of hierarchical loan repayment contracts by an endofunctor. The universal constructions of a limit verify all constraints as the sectoral investment and learning function at the macroeconomic level. The dual colimit computes the aggregated informations at the macro level as usual in the mathematics of transitions from local to global structures. We use visual diagrams to make complex economic relationships intuitive. This paper is meant to map economic to categorical concepts to enable interdisciplinary collaboration for digital twins of monetary accounting systems.

*SRH University of Applied Sciences, email: viktor.winschel@gmail.com. Many thanks to Dusko Pavlovic, Alexander Kurz, Melanie Swan and Boyan Beronov for many past and ongoing discussions using category theory for theories and models in economics and econometrics.

Contents

1	Introduction	5
2	Macroeconomic Accounting	7
2.1	Economic Decisions	17
2.2	Recursive Program	19
3	Categories for Macroeconomics	21
3.1	Categories as Economic Structures	23
3.2	Functors as Value Transformations	24
3.3	Natural Transformations as Evolution	25
4	Evolution	27
4.1	Eight Macroeconomic Bookings	27
4.1.1	Categorically Typed Recursive Program	30
4.1.2	Categorical Program	35
4.2	Consumption Decisions	36
4.2.1	Consumption	36
4.2.2	Aggregate Demand	38
4.3	Production Decisions and Price Formation	40
4.3.1	Cobb-Douglas Production	40
4.3.2	Cost-Based Planning and Market Equilibration	42
4.3.3	Price Formation	45
4.4	Investment Decisions	47
4.4.1	Investment Decision Sigmoid	47
4.4.2	Investment Allocation	49
4.4.3	Intertemporal Contract Memory Updates	51
4.4.4	Dividend Decision	54
4.5	Universal Constructions	56
4.5.1	Pullbacks for Constraint Validation	56
4.5.2	Pushouts for Economic Computation	59
4.5.3	Categorical Types of the Eight Macro Bookings	60
4.6	Categorical Evolution	63
5	Conclusion	68
A	Appendix	70
A.1	Category Theory	70
A.1.1	Category, Functor, Natural Transformation	72
A.1.2	Pullback and Pushout	75
A.2	Programs	77
A.2.1	Recursive Program	77
A.2.2	Categorically Typed Recursive Program	83
A.2.3	Categorical Program	92
B	References	102

List of Figures

2.1	8 Macro Accounting Bookings of 5 Agents in 20 Accounts	7
2.2	Time series plots of variables in the recursive simulation	20
3.1	String Diagram: Natural Transformation $\eta_{\text{time}} : F_t \Rightarrow F_{t+1}$ showing functor composition equivalence for temporal evolution	26
3.2	Commutative Diagram: Natural Transformation Pullback Flow	26
3.3	Natural Transformation Pullback: Temporal evolution validation across periods 0, 1, 2. The pullback ensures that account transformations preserve categorical structure while evolving through time.	26
4.1	String Diagram: 8 Bookings Pushout $Real + Nominal \rightarrow Conservation$ showing universal aggregation property	27
4.2	Commutative Diagram: 8 Bookings Pushout Flow	27
4.3	8 Bookings Pushout: Flow aggregation of real flows and nominal flows into balanced economic flows. The pushout ensures conservation of accounts across all transactions.	28
4.4	Commutative Diagram: Booking Validation Pullback Structure	29
4.5	Commutative Diagram: Booking Validation Flow	29
4.6	Commutative Diagram: Booking Validation Pullback Flow	29
4.7	Booking Validation Pullback: Each of the 8 bookings is validated as a typed morphism with domain and codomain checks. The pullback ensures transaction validity before execution. . .	30
4.8	Time series plots of variables in the categorically typed recursive simulation	33
4.9	String Diagram: Categorical State Structure	33
4.10	Categorical State Composition Structure	34
4.11	Categorical State Validation Pullback: The categorical state pullback ensures that all components ($\mathcal{C}_{acc}, \mathcal{C}_{flows}, \mathcal{C}_{pars}, \mathcal{F}_{price}, \mathcal{F}_{flow}, \eta_{price}, \eta_{flow}$) satisfy universal consistency properties through functor and natural transformation validation.	34
4.12	Categorical State Composition Pushout: The categorical state pushout implements the universal composition of all categorical components into the complete economic simulation state S^{cat} , ensuring structural preservation and temporal coherence through the unique universal morphism u	35
4.13	Time series plots of variables in the categorical simulation	36
4.14	Commutative Diagram: Booking Validation Pullback Flow	37
4.15	Commutative Diagram: Consumption Function Pullback Flow	37
4.16	Consumption Function Pullback: The consumption decision pullback validates the multiplication of account balances with consumption ratios, ensuring proper economic behavior across agents.	38
4.17	Commutative Diagram: Demand Aggregation Pushout Structure	39
4.18	Commutative Diagram: Demand Aggregation Pushout Flow	39
4.19	Demand Aggregation Pushout: The demand pushout aggregates individual consumption flows (ConsumRes, ConsumLab, ConsumCap) into total market demand, providing the economic signal for production decisions.	40
4.20	Commutative Diagram: Cobb-Douglas Production Pushout Structure	41
4.21	Commutative Diagram: Production Function Pushout Flow	41
4.22	Production Function Pushout: The Cobb-Douglas production function pushout combines labor and resource inputs with scaling factor $\alpha = 0.42$ and substitution elasticity $\gamma = 0.75$, producing goods according to technology.	41
4.23	Commutative Diagram: Cost-Based Planning Structure	43
4.24	Cost-Based Planning Pullback: The cost planning pullback ensures revenue targets cover all contractual obligations (wages, debt service) plus desired profit markup.	43
4.25	Commutative Diagram: Market Equilibration Structure	44
4.26	Market Disequilibrium Pullback: The disequilibrium pullback measures market tension between consumer spending capacity and company revenue requirements.	44
4.27	Commutative Diagram: Price Formation Pullback Structure	45
4.28	Commutative Diagram: Price Formation Pullback Flow	46
4.29	Price Formation Pullback: The price formation pullback validates the equilibrium price calculation as the ratio of demand to supply plus markup, ensuring market clearing across periods. .	46

4.30	Commutative Diagram: Investment Sigmoid Pullback Structure	48
4.31	Commutative Diagram: Investment Sigmoid Pullback Flow	48
4.32	Investment Sigmoid Pullback: The investment decision pullback applies the sigmoid function to demand surplus, with parameters ($\sigma_A = 20.0, \sigma_B = 480.0, \sigma_C = 200.0$) governing the response curve.	48
4.33	Commutative Diagram: Investment Allocation Pushout Structure	49
4.34	Commutative Diagram: Investment Allocation Pushout Flow	50
4.35	Investment Allocation Pushout: The investment allocation pushout distributes total investment between resource and labor shares (0.8 and 0.2), ensuring the constraint that shares sum to 1.	50
4.36	Commutative Diagram: Intertemporal Contract Memory Structure	52
4.37	Memory Update Pullback: The memory update pullback ensures intertemporal contract obligations are correctly inserted into FILO stacks, preserving temporal ordering of contractual commitments.	52
4.38	Commutative Diagram: Dividend Decision Natural Transformation with pullback structure	55
4.39	Commutative Diagram: Dividend Decision Pullback Flow	55
4.40	Dividend Decision Pullback: The dividend decision pullback combines profit differential and bank balance with respective dividend rates, ensuring non-negative dividend payments.	55
4.41	Commutative Diagram: Investment Validation Natural Transformation with pullback structure	57
4.42	Commutative Diagram: Investment Validation Pullback Flow	58
4.43	Investment Validation Pullback: The investment validation pullback ensures that investment decisions satisfy both company capacity constraints and bank liquidity limits across all periods.	58
4.44	Commutative Diagram: Account Update Natural Transformation	61
4.45	Commutative Diagram: Account Update Pushout Flow	62
4.46	Account Update Pushout: The account update pushout implements the 8 bookings through proper double-entry accounting, ensuring conservation of the unit of account ($\sum \delta = 0$) and non-negative account balances.	63
4.47	Commutative Diagram: Three-Layer Economic Evolution Natural Transformation	65
4.48	String Diagram: Parameters $\otimes$ Flows $\otimes$ Accounts Evolution	65
4.49	Commutative Diagram: Period Evolution Natural Transformation	66
4.50	Commutative Diagram: Categorical State Transition Pushout Flow	67
4.51	Categorical State Transition Pushout	67

1 Introduction

We present a categorical foundation of the Monetary Macroeconomic Accounting Theory (MoMaT) of Menéndez and Winschel [2025]. It is a theory of the fundamental hierarchical multi-sector-agent macroeconomic process of consumption, production, investment in monetary and real units. The dynamics of such a complex system involves different types of concepts at different hierarchical levels: macroeconomic invariances, markets, prices, production, goods, demand of the sectors, therein individual agents' decisions being companies, households, or banks. We validate constraints, factorise computations of universals in pullbacks and pushouts over accounts in structural functorial constructions of a consistent operational. The hierarchical macroeconomic accounting system is to be designed to book into consistent double-entry microeconomic accounting systems. For that we categorify the equations and functions of a recursive formulation of MoMaT with functors and natural transformations operating on categories as its state. Simulations of the same data are illustrated in three different ways as recursive, categorically typed recursive and categorical programs.

Formal visual diagrams of the structures form an integrated modelling environment for economics. Complex economic relationships are more intuitive in formal geometric visualisations. They keep local parts and global properties of whole or macroeconomic systems consistent. The abstract nonsense attributed to category theory makes sense when used to separate syntax and semantics in designing domain specific programming languages (DSL) in compilers. The DSL of this paper implements macroeconomic sectors and their macroeconomic accounting structures for a faithfully cooperation enabled by the sound consistency of the macroeconomic accounting system, organised and kept consistent by lifted double-entry accounting consistency conditions to also encompass macroeconomic accounting consistency. Categorical structures enforce type discipline for the economic flows captured in eight sectoral bookings of basic events of the processes of consumption, production and investment. The events take place every period consistently connected by natural transformations as monadic endofunctors as evolution of time over a memory of past legally created and partially fulfilled contractual future obligations as a generalized economic constraint problem and its solution and consistent evolution. A monad formalises in functional programming language semantics referentially transparent functions and side effects like changing a variable's assignment in programs like $x = x + 1$. Side effects and external effects in economics and institutional dynamics where domains and codomains of the functions involved change, polycentrism of Ostrom [2005] and endogenous econometricians of the Lucas Critique can be modelled by universal constructions. Categories offer a mathematical foundation for monetary and real units as well as creation of types as currency unions and exchange rate regimes typable in initial and final categorical algebras and coalgebras, respectively.

An economically important result of the categorification is that sectoral investment learning and aggregation are both universal. The macroeconomic accounting system can represent quadruple accounting as in the standard for national accounting systems in International Monetary Fund et al. [2009]. Quadruple accounting system book parallelly in two micro double-entry ($4 = 2 \times 2$) accounting systems in each macroeconomic booking. In our approach they are decentralisable by compositionality and parallel and hierarchical combinators. Micro consistency is about booking in each micro booking the same one number into the credit and debit sides of two T-accounts of one decentralized and consistent double-entry booking system. An algebra of resources can be constructed on the linear, resource aware logic of these one numbers booked in several decentralized places of the system.¹ It also needs an operator to represent the parallel processes of a decentralized market economy. The parallel, sequential and hierarchical combinators in this paper enable us to fully type macroeconomic accounting systems for nations and institutions.

In MoMaT the macroeconomic consistency encompasses a market where the price matches consumers' demands and the planned sales of goods of producers to cover the costs and profits in universals. Economic optimisation is lifted to categorical universals and adjunctions. Category theory offers computable closure operators over composed macroeconomies for currency areas, closed over debt contracts of institutional sub-object structures like central banks, governments or generations over time as deontic modalities of legal units of polycentric management. Another important closure we also calculate as usual in economics is the closure over endowment, production and consumption of goods, to get what is usually meant by a *closed* economy.

¹Linear logic is a resource aware logic, meaning that logical resources can only be used once in a proof, where they are used up. For example, in a linear logic of the logical resources $\{A, A \rightarrow B, A \rightarrow C\}$, only B or C can be proved using A , since A is used up in one proof being not available anymore for the other. To proof both B and C we need to copy A first and with the resources $\{A, A, A \rightarrow B, A \rightarrow C\}$ we can proof B and C by using the modus ponens $\{A, A \rightarrow B \vdash B\}$ twice $\{A, A \rightarrow C \vdash C\}$. Quantum computers run on linear logic which makes them ideal to represent property right transfers, secured by physical laws whereas digital computers need redundant blockchains to ensure that costless copy-and-paste operations of digital computers are not misused to fake property right transfers.

The institutional closures of meta rules like macroeconomic budgets, or liabilities equal assets in debt systems of loans and repayments is complementing the production closure in the categorified MoMaT. This opens a path for an economic theory of sharing the wealth and values over universally factorized production and aggregation, complementing the theory of labor sharing and macroeconomic accounting for hierarchical risk sharing. Money is then a medium of labor, risk and GDP sharing. The monetary macroeconomic system of loans and repayments is consistently booked over hierarchical structural functorial evolutions. Extensions of the theory for more agents, accounts, levels and consistencies are compositional by the categorical semantics of the DSL. Universal categorical constructions are adjunct functors and generalise the economic operators of maximisation to higher-order functions, morphisms, functors with natural transformations and universals for institutional dynamics of systemic macroeconomic entities as functorial views on the system at hand.

In mathematics adjunctions are everywhere as in economics. Universals formulate systemic rationality formally as validations of economic constraints and implementations of the programs for decentral and hierarchical economic planning and coordination. The simulated data shows stable convergence of the monetary systems, it is not a Ponzi scheme, neither in need to be stabilised by governmental debt policies (especially by unsustainable ones). Categorical stability proofs and proofs of the universal construction by verifying the universal constructions used in this paper are left for future research. We are focused in this paper to map the economics concepts to the categorical ones. The *proofs* so far are the numerical simulations, to be taken as constructive witnesses of the soundness of the categorical constructions.

The DSL can be used to program a national accounting system as for the one in MoMaT. As an architectural toolbox it improves the one we have developed in industrial projects on liquidity management for holdings of decentralized subsidiaries running windparks in and outside of the EU, a platform for smart financial contracts on liquidity exchange among companies in supply chains, an IoT data based simulation of decentralized energy exchange systems of mobile decentralized energy consumption and production, a dialy production decision of a dairy holding of 7000 farmers producing 1600 products from 20 Mio liters of milk a day in 9 factories, delivering intermediate and final products to 300 logistic hubs and super markets, and a decentralized ERP system for digital farmers' outlets or shops with regional logistic hubs for dozens of shops that we equipped with standard hardware. The research started in the PhD of the first author on money theory Menéndez [1989] and the follow up in Menéndez [2010]. The second author's PhD was on central bank policies in econometric models of exchange rate regimes, see Winschel [2005].

In Appendix (A.1) we give the categorical definitions used in this paper. The order of sections in this paper corresponds roughly to the sequence of macroeconomic bookings in the recursive program. It is a usual presentation of the economic theory and rationals in the macroeconomic economy. However, we extend this sequential order by parallel combinators. In programming language design theory the tensor product is used for that, in function spaces of monoidal cartesian closed categories. We use that to represent the economics of the eight events and their macroeconomic bookings. A mathematician might feel more comfortable to read the paper in the reversed order starting from Section 4.6 with the definition of $\mathcal{C}_{Economy}$ the category of the economy. There the whole macro thing, the macroeconomy, is defined. The endo functor on this category is the evolution in time as a natural transformation. The micro foundation of macroeconomics follows the order of the sections and the macroeconomic foundation of microeconomics starts top down from the category of the economy and its endofunctor in Section 4.6. The circle from micro to macro and from macro to micro forms a *hermeneutical* circle, where the parts constitute the whole and can only be understood after understanding the whole. Similarly, in architecture it is said that only the city, composed of places, gives to places their functions.

The categorical, process oriented model of consciousness as Memory Evolutive Systems (MES) by Ehresmann and Vanbreemersch [2007] understands life to emerge from synchronised, i.e. consistent, time scales that are increasing bottom up to the top level control. Cells, tissues and organs self-organise into the identities of cognitive living beings, formed by colimits of concept formation in the brains' neural networks. Death occurs as a desynchronisation of the hierarchical levels. Similarly, we can understand economic institutions as synchronisation devices over hierarchical levels. Control is to be synchronised as well over time, levels and subobjects or parts of the economic system management. The coregulators in MES and polycentric management are both about decentralized control and our open games in Ghani et al. [2018] can capture decentralised control by compositional economic modelling within the functorial and by that compositinal national accounting system of this paper. Nations may be taken to fail and emerge from de/synchronisation of micro- and macroeconomic levels. We state the basic consistency conditions towards such types of theories and institutions.

2 Macroeconomic Accounting

[illegible]

Figure 2.1: 8 Macro Accounting Bookings of 5 Agents in 20 Accounts

Definition 2.1 (Macroeconomic Accounting). In the Monetary Macroeconomic Accounting Theory (MoMaT) of Menéndez and Winschel [2025]) money does not primarily function as a medium of exchange between goods, but rather as a medium of loan processing in solving the fundamental intertemporal problem of capitalist production. This problem is that the producers must pay their suppliers before receiving money from their customers. This payment carries the risk of not being paid for unsold products. Therefore, for producing, we need to have the institution of a monetary unit for measuring debts, loans and liabilities and a thing called money representing some amounts of the monetary units as a means to repay the loans by transferring the property right on the money thing, see Menéndez and Winschel [2025] for the legal principles underlying this view on money. We do not consider the problem to circumvent the need to exchange goods in a double coincidence of wants, as the fundamental functionality that money solves. The exchange of goods via money encompasses only a sales contract with goods and money transferred intratemporally at the same time settling debts in the good and the money. So, even a simple sales contract say a bike for money, creates debts, the seller owes a bike and the buyer some money. Risk is involved if a defunct central bank (king or finance minister) issues the money used in the exchange. In MoMaT the real factors labor and resources and the real goods can be exchanged via money or bank account transfers between the agents involved, as well as the dividend payments to the dividend owner called capitalist. These are processes with money as a means of payment and exchange, but the actually important dynamics of debt creation and repayment are not addressed. This is what we focus on in MoMaT, where the intertemporal consistency needs define what a functioning central bank is essentially about.

To prepare the technical tools to define what makes a functioning central bank and what consistency conditions are needed, where and at what levels, is the subject of this paper. In MoMaT, the argument goes that the fundamental intertemporal problem in capitalist economies characterised by a division of labor and risk can and is being solved every day. For this, we model bank loans and bank transfers for good production and labor and resources and goods exchange in a macroeconomic accounting system for five sectors also called agents.

Producers need loans from banks because labour and resource owners want money in order to buy goods for consumption immediately in exchange for their real input factors to production. Labour and resource owners do not want to bear any risk of production and sales, if so they play the role of a bank or investor. The distinction between risk takers like the company, bank or investor versus labor and resource owners is important and is modelled hierarchically in this paper. The company and the investment projects are at the micro level and the bank at the macroeconomic level. The bank provides loans to the producer and creates by that consistency conditions and invariances at the macroeconomic level of accounting over the micro level of double-entry accounting systems of the agents involved. A hierarchical risk absorption takes place, first at the company, then bank then central bank and government at the macroeconomic level, if the basic MoMaT is extended for a third meso level then of banks and central banks and governments at the then macroeconomic level. Once producers have sold their products as goods to consumers, the labor, resource and dividend owners or capitalists, they can repay the loans to the bank. We formulate the

macroeconomic consistency conditions for the agents involved. The banking sector can price risks at the aggregated level and compute from it the interest rate as a risk premium of the insurance of investments, much like mandatory insurance for bank deposits. The closures we formulate are categorical constructions over the loan, closing investments by repayments and over endowments, closing production by consumption, which is usually called *closed* economy. With a categorical closure over real and nominal units of account, we can compose and connect to formulations of circular economies and environmental economics for example of Georgescu-Roegen [1971], with physical invariances like geophysical world macroeconomic resources or Emmy Noether's rescue of Einstein's relativity theory with energy (and value) preservation in an inflating universe by endogenous units of coordinates (and accounts). We categorize the nominal and real units, the loan repayments and production consumption closure as the universal investment pullback construction for loan contract verification and the pushouts for the aggregated calculations involved while keeping the program of the macroeconomic accounting system consistent.

Company owners can be called capitalists, profit owners or dividend owners. They consume based on dividend payments rather than like workers and resource owners from their sales. There are of course important dogma historical economic, philosophical and economic policy discussions around these loaded concepts. However, in this paper we are concerned about money theory providing the tools to be able to inform these debates on complex systems by categorical types.

In Figure 2.1 we see the agents, their T-accounts in their double-entry systems and the eight macroeconomic bookings that capture the dynamics within each period. The visual representation that has proven useful for a macroeconomic system of microeconomic accountings consists of representing the agents' systems as T-balances with Asset $\blacksquare \square$ and Liability $\square \blacksquare$ T-accounts of the micro systems on the left and right sides of the T-balance. This allows visual distinction of the formally important in $\blacksquare \square$ - and outflows $\square \blacksquare$ on Asset and out $\square \blacksquare$ - and inflows $\square \blacksquare$ on Liability account types for a transfer into categorical morphisms. In the lower eight lines are the macroeconomic accounting bookings within each period of the five economic agents and their accounts. We see question marks for some bookings among the eight which will be answered by understanding the categorical types involved.

The bookings 1, 2, 3, 4 of the Labor and Resource agents record the sales of their endowments for consumption in each period. Resources and Labor are created anew each period and consumption is the purpose of running a labour sharing economy. The real inputs and finished, produced goods are also booked for completeness purposes in real accounts, because we can, and should as economists. Accountants routinely keep such real accounts as Excel spreadsheets without them being formally integrated in the accounting systems, where accounts are kept in monetary units, while the real goods and inputs are kept in hours, kg or other types and units of account. In German the data of these spreadsheets are called *Mengen-gerüst*, which is literally translated as something like *amounts-constructions*. Of course, this is called inventory or warehouse management software and they are more or less integrated with the accounts kept in monetary units in the double-entry accounting systems. In this sense we unify such programs or systems in the two functors for real and nominal accounts which in a national accounting setup can categorically become a resource theory. The real inputs and finished goods are, and actually intermediate goods could be, also booked for completeness. Creation of goods or entities in general are categorically handled in initial and final objects, which we do not do for now in this paper. These have the universal property of a morphism pointing from the initial to every object, and a morphism pointing into the final object. These two universals allow to model resources creation and deletion or consumption or usage in production and the like, like contract creation and fulfillment. A resource theory in this sense seems to be missing for economics and can be developed and properly typed in terms of initial and final objects and their universal morphisms. Symmetric monoidal categories formalise multiplicative aspects of linear logic in tensor products of combining resources. The question marks in the dividend booking 6 and especially in the loan contract bookings 5,7 will be captured by the interesting mathematical structure of contra variant functors. This points into the direction to capture the abstract legal principle of *quid pro quo* or *tit for tat* in economics by covariant and contra variant functors which may include a change of domains, codomains or units of accounts.

In the upper part of Figure 2.1 we see the monetary accounts of the agents Labor owner (Lab), Resource owner (Res), Company or producer (Com), Capitalist, dividend or company owner (Cap) and Bank (Bank). In the upper part are the nominal accounts arranged as assets and liabilities in a T-balance, which contains the T-accounts of the agents' double-entry bookkeeping systems. The accounts are arranged as asset and liability accounts in the T-balance on the left and right hand side, respectively. Note that T-accounts have the inflows on the left being an asset or right being a liability account, and outflows accordingly

on the right and left for assets and liability accounts. The in- and outflow ports of each T-account are labelled by xyz to count all the in- and outflow ports involved in the eight bookings, with x denoting the macroeconomic booking number, y counts the agents by a,b,c, for 2 as a,b and for 3 agents as a,b,c that form a macroeconomic booking. The z groups the T-accounts involved in the bookings by 1,2 for the in- and outflow ports. This weird index system counts all flows, it is implicit in the recursive implementation and categories free us elegantly from the need to keep track of consistency by such low level, confusing and error-prone indices. The xyz labels in the upper, accounts part of Figure 2.1, index the ports involved and, if projected down to the lower eight lines of the figure, connect the eight macroeconomic bookings to the account ports of in- and outflows. The xyz labels in each T-account, vertically stacked in the upper part, are the in- and outflows of one whole period with all eight bookings. As a sum the in- and outflows are added and subtracted to and from the accounts' states at the end of each period, representing a yearly financial report as a balance sheet of asset and liability accounts. These flows change the state of the macroeconomic accounting systems' accounts from one to the next period. This is the whole dynamics of the system with the ports as the places where bookings flow values in and out of the double-entry microeconomic systems, the parts of the macroeconomic accounting system.

With T-accounts, one can avoid the minus $-a$ as a unary negative operator and use it only as a binary $a - b$ subtraction operator in the equity definition of assets $-$ liabilities. To avoid unary $-$ and use binary $-$ was in 1452 the state of knowledge, when Paccioli published his history changing book on double entry bookkeeping. Negative numbers were only understood by mathematicians, let alone accountants some centuries later. The negative numbers in the outflow of assets and liabilities are arithmetically represented in equations like $a - b - c = a + (-b) + (-c) = a - (b + c)$ of a as asset and b, c as liability accounts. Without a unary $-$ operator (defined by the middle expression), we can only use the left or right expressions, where the right expression is the equity definition, using a sum over the positive numbers representing liabilities in $(b + c)$. T-accounts allow to represent liabilities and outflows (to assets and liabilities) as positive numbers and only use subtraction by a binary minus in the top level calculation of equity, in $\text{Equity} = \text{Assets} - \text{Liabilities}$. Outflows to liabilities as negative negative assets would have probably cracked the brain of an accountant anno 1452, which is now simply $-(-b) = b$, saying that less liabilities is more equity, a favourable event from an accountant's point of view. This technique has made T-accounts into a bookkeeping technology where only binary and not unary $-$ operations are needed to represent negative numbers of outflows and liabilities. The double-entry bookkeeping system is a revolutionary protocol that records parallel, decentralized events of exchanges while keeping the whole thing, centrally consistent as its essential characteristic, see Katis et al. [2008].

We clarify how this mechanics of consistency keeping through micro bookings where one and the same amount of value as a number is written on the debit and credit sides of two T-accounts, can be transferred to national accountings in quadruple entry systems, where one and the same number is written in two double-entry systems' accounts as a flow of value between the debtor and creditor involved in a loan contract. We clarify this as the content of the macroeconomic foundation of microeconomics. Categorically typed, we extend the consistency maintenance of micro- to macroeconomic bookkeeping as a pullback verified calculation by pushouts. However, to begin with, the deterministic version needs to be as equipped with consistency tools as in this paper. A stochastic version, for example by Giry Giry [1982] monads, will enable to model more suitable risk estimates at the macroeconomic level of banks. It is missing in macroeconomics and to the best of our knowledge a theory of quadruple accounting is not formulated or software systems provided.

A mathematical and computer scientific formulation with implementation details for double entry systems can be found in Rambaud et al. [2010]. We have programmed its algebra in Julia with some first categorifications using the categorical library `Catlab.jl`, see Patterson et al. [2022] for the tools at work. The algebra of double-entry bookkeeping is formulated in terms of state machines or automata on vectors of real numbers. The state vector with elements being the balances of each T-account are normalized by one account as the consistency condition to sum to zero, with equity taken as an account. The asset and liability accounts are represented as signed, i.e. negative and positive states and state transitions as vectors of the same length as the state vector, with in- and outflows as positive and negative numbers. So, bookings are state transitions as element wise additions of change vectors to consistent state vectors. This representation of the state and the state transitions uses unary $-$ in negative numbers, while in usual T-account representations numbers are never negative, which is the point of using T-accounts first of all. The double-entry algebra and code can be compositionally, i.e. categorically, integrated into our categorical macroeconomic accounting system, using standard theoretical computer scientific formulations of automata, for example in

a coalgebraic semantic of Rutten [2000].

To recap: Lab has a nominal bank account and real accounts for labor and goods. Res has a nominal bank account and real accounts for resources and goods. Com has a nominal bank account, a liability account for managing loan repayments, a liability account for dividends, and three real accounts for goods, resources and labor. Cap has a nominal bank account, an asset account for dividends, and a real account for goods. Bank has only nominal and no real accounts, one for company loans as an asset account and the bank deposits of agents as liability accounts. Only Com and Bank maintain liability accounts in the maintained invariances since we are interested in the closure of the dynamics of loans with investments and repayments as two intertemporal payments of a debt relation over time. The real and nominal account product categories are for the tit for tat relations of real and nominal flows. The dividend flows can be properly extended to become a tit for tat flow of values in contra and covariant functors of some contract on property rights on the compny. However, this part of the contract between the capitalist and the company is not included in the invariances conditions of MoMaT. Only the Bank is involved in invariances since we are modelling the loan functionality that synchronises the desynchronised payments of input factors of production and its outputs. There are 20 accounts for 5 agents to be managed in the macroeconomic accounting system of debts, investments, loans and repayments. Bookings 5, 7 involve only two agents, company and bank, in the investment and repayment bookings. The other bookings involve three agents. These bookings repeat in each period to form the overall dynamics of the macroeconomic economy which we are categorifying in this paper.

The producer's liability in the form of dividends can be closed by some form of company ownership that can be formally specified as a contract of the Capitalist type of agent. The company pays dividends to the capitalist but this contract does not appear in MoMaT as an invariance of the money and credit system. We focus to model the money for loans functionality and study the closure of debt relations in the loan contracts of investments and repayments. The contract between capitalist and company can be specified as some form of debt as loan or contribution of the capitalist to the initial endowment of the company. But these additional contracts do not change the insights from categorifying the debt relationships between company and bank in the loan contract and its repayment flows. Ownership contracts between capitalist and company would be necessary if we wanted to model and program their creation, transfer and deletion. But even then they would be additional functors and consistency conditions in natural transformations and another functionality of the macroeconomic accounting system. We opted to not model this complication for now without further insights into the nature of the monetary loan system.

The states in the formulation of MoMaT as recursion on vectors of real numbers as opposed to the categorical formulation also represent the states of information within the system about the system at each point in time. Categories contain much more mathematical information than the recursive formulation of numbers in the simply typed vectors. The categorical structures of economic accounts (category), flows (functor), parameters (category) and their relationships (morphisms in categories, bookings) such as decisions (morphisms and functors), dynamics (functors), consistency conditions (natural transformations) and optimality (universality) conditions are mathematically much more informative, for researchers, agents in the theory and users of economic programs in the economy, institutional designers or architects of macroeconomic systems.

The categorical formulation made programming in our workflow more into an engineered experience working out the types and specification and the compiler for running the programs. The compiling process is actually the non-automated way of doing economics by an economist who is translating the economic argumentation lines into, say, a simply typed recursive program. If management is about doing economics then compositional economic modelling is useful for decentralized management. This involves a centrally planned versus a decentrally organised market economy or a hierarchically decomposed polycentric management of Ostrom [2005]. The pattern is similar to the structure involved in the so called *simultaneous engineering* at BMW. It denotes the simultaneous production of cars and the production of tools to produce cars. This reflexivity made BMW very adaptive and contributed much to its success in the past.

The workflow of doing economics by extending our MoMaT, for example by a central bank and a government or public sector, is to define their accounts, bookings of basic events and the consistency conditions or invariances involved. The purpose of a DSL is to enable its users and modellers to formulate their theories using this high-levelness for economic concepts, which the compiler translates into verified, consistent low level implementations like iterations. But the categorical types do not only inform compilers what to compute, but also real agents in the economy, agents in theory, and economists - they create own functorial views on the economy in its digital twins. The categorical approach to mathematics is reminiscent of the

Kantian view that *we can not grasp the thing itself* and all experience is mediated by our senses. The Yoneda lemma as a most categorical essence, states that we do not need to grasp the thing. Categorically it is enough to consistently glue different points of view on the thing in order to understand and define it mathematically. Combined with the *social way* of defining properties of things by its behaviour in its environment, also as a prerequisite to achieve modularity and compositionality, we get by category theory a mathematics that feels like a dream of a mathematical social scientist comes true. But, the learning curve for category theory is high, even for a mathematical economist, in need of a map from the basic economic concepts into the categorical ones. This map is the point of this paper, providing a categorical semantics for macroeconomics. This paper has a paedagogical purpose to exemplify category theory to economists and economics to computer scientists. We are well aware that both parts are too simple and too lengthy for both adressed groups of researchers, but we hope to provide a starting point for both.

The categorical formulation ensures formal consistency, which in the recursive one is only implicitly encoded via the basic types of numbers and vectors and recursive functions. The low levelness of the indexing in Figure 2.1 by the names "1a1,1a2,1b1,1b2,...,2c2,...,8c2" of the in- and outflow ports of the macroeconomic accounting system is similary the result of the poorly typed recursive formulation. The functions and equations are not typed either, as invariances, decisions, bookings or memory functionality. They are functions or variable asserts by the usual syntax in programming languages using $=$ to define calculations, state transitions or function definitions. The functions as well are not, but can be annotated with their types as types of domains and codomains combined into the type of a function, like in $f : \mathbb{R} \rightarrow \mathbb{N}$, by the type operator $\rightarrow$. The bottom line is that the recursive program does not use modern compiler and type defining (by `struct`) in Julia and type checking (with multiple dispatch) technology like in the categorical programs.

The problem of missing types and units of account has profoundly shocked the second author as a young economist in the first five minutes of lectures in economics, especially being trained in unit discipline for consistency checks in physics at school. This is stunning since arithmetics in any exchange in economies involves nominal and real units, like in a price of 5.35 \$/kg. Economic theory should be explicit about units of accounts to verify their consistencies. This became more a problem then merely a stunning situation during the PhD on exchange rate regimes, where units are not only to be there but are to be created for a monetary union by political agents. The mathematical illiteracy for representing the type of the economical events and agents in an economic theory on money and exchange rate regimes was the motivation towards this paper. What is the type of the creation of the type (or unit of account) of a currency union? This obscure sounding question points to a profound field and effort of computer science, namely that type specifications are the proper way to think about DSLs, compilers, unit checking, correct programs, in short, the effort to give programs, describing a computation, a semantics and a guarantee of correctness of the program, in the sense of deriving a correct implementation given a specification.

The motivation for verified programs, for example at NASA, is that errors in programs may lead to exploding Space Shuttles and dead pilots. The motivation for the same technology in economics is that errors in inconsistent economic theories and their implementations in software may lead to currency crises and devastating results, riots or even death for whole continents. This was the content of a discussion with Fischer and Schumann [2003] on the topic of AutoBayes, a Prolog program for generating data analysis programs from the specification of statistical models at NASA. What are the types of the units, agents and hierarchical levels and the types of agents creating and deleting types? What is the political economics of consistency and transparency with the delicate issue in macroeconomic policies where those agents, entities, groups or decision units, who fail to build consistent systems as part of their job, might not want to be checked for consistency? Constitutions and institutions are meta rules for finding good rules or laws in the political process. In this sense a type theory and specification, say of the national accounting system, has to be able to represent the constitution of a central bank, which is translated into the programs of the central bank. A central bank is created together with the type of a monetary unit of account, the currency, while its operational processes create money, i.e. units of this type. However, private type creations are also part of our economic reality called a newly created crypto currency with tokens of this type being created operationally, day to day. Similarly, a cost center in internal or cost accounting within companies might be usefully conceived as a custom currency as well. However, type theory should be a highly welcomed mathematical tool available in a future economics.

Definition 2.2 (Accounts). A macroeconomic accounting system is a system with double entry accounting systems for each agent. The 5 agents in MoMaT are Labor (Lab) and Resource Owners (Res), Producer

(Com), Profit Owner (Cap) and Bank (Bank). Figure 2.1 shows the T-accounts of these agents and their inflow and outflow ports. In the lower 8 lines of Figure 2.1 are the 8 macroeconomic bookings describing the dynamics within a period. The account names are typeset in equation (1) as used in categorical formulas. In equation (2) they are typeset as used in the programs, as variable names of a real number type $\mathbb{R}$ representing the state of the account in nominal or real units like $[EU]$, $[kg]$, $[h]$. These numbers are typed by functors, however we have not programmed functionalities for creating custom units of accounts. They can be integrated by defining functors with unit checks in natural transformations. The 8 macroeconomic bookings and their executing events happen every period anew. The sequential order in the run time of the recursive program of these 8 bookings does not capture the economic logic, i.e. the sequential and parallel order of the economic events and of the bookings is not precisely enough defined in the (sequential) program. We need parallel combinators which in computer science are taken to be the tensor operation in the categorical foundations of computing theory. We will see the parallel combinators in diagrams with a categorical semantic which can be used to formalise the economics of parallel resource usages.

An important result of the categorification is to clarify the parallel and sequential execution order of the events and the bookings and to use the categorical tools made for that in monetary macroeconomics. Since we are modelling the hierarchy of the system, we need sequential, parallel and hierarchical combinators. The order of the 8 bookings can be said for now to be vaguely consistent with a macroeconomic consistent sequential, parallel and hierarchical order. We see an example of the methodic difficulties of underspecification in booking 2. In terms of DSL design, the simply typed recursive language is not expressive enough to represent the real world phenomenon of parallel, sequential and hierarchical executions, i.e. the core of economics. Alternatives can not be properly evaluated, if not represented in theory, i.e. we expect welfare losses from using underspecified economic theories. The Labor agent in booking 2 consumes even so the resource for production is not yet bought by the company from the resource agent. What happens during production, can the company use up its stocks of resources in its warehouse, i.e. in the real T-accounts, and when and how are the combined constraints of the agents taken into account? The production and consumption decisions interact with the amounts of labor and resources at the company in their warehouses and how we define them to be used and to vanish. Answers to such questions can happen to become quite complicated, as we see them even in the simplest setup of five agents of MoMaT. The only candidate, it seems to us, for a formal tool to scale to nation wide or macroeconomic systems, is category theory, and we need to start somewhere, here from the simplest economic setup of a monetary system in MoMaT.

We see that economics without parallel, sequential and hierarchical combinators, and operations for creating and deleting objects, leaves us with an underspecified theory, for programming purposes but also for consistency verifications. This is a problem especially since economics is about labour sharing in decentralised, i.e. parallel and sequentially more or less well ordered and to be improved orders, say by assembly lines. Hierarchical and temporal combinators have to specify desired properties of systems involving as complex situations as the wants of or fairness deliberations for future generations.

The categorical typing results in string and commutative diagrams in the parallel arrangement of the dependencies between state and variables of interest as intermediate calculations of decisions up to the state updates as bookings in the T-accounts of all inflows and outflows. We can think about the 2-dimensional commutative diagrams to represent two sides of an equation which as a consistency condition characterises macroeconomic invariances or desired, specified properties in general.

The following table shows the types of the T-accounts for each agent together with their in- and outflows in our MoMaT.

<i>Lab</i>	<i>Type</i>	<i>Res</i>	<i>Type</i>	<i>Com</i>	<i>Type</i>	<i>Com</i>	<i>Type</i>	<i>Cap</i>	<i>Type</i>	<i>Bank</i>	<i>Type</i>	<i>Bank</i>	<i>Type</i>
$Lab_{\blacksquare\blacksquare}^{Bank}$	$[EU]$	$Res_{\blacksquare\blacksquare}^{Bank}$	$[EU]$	$Com_{\blacksquare\blacksquare}^{Bank}$	$[EU]$	$Com_{\blacksquare\blacksquare}^{Loan}$ $Com_{\blacksquare\blacksquare}^{Div}$	$[EU]$ $[EU]$	$Cap_{\blacksquare\blacksquare}^{Bank}$ $Cap_{\blacksquare\blacksquare}^{Div}$	$[EU]$ $[EU]$	$Bank_{\blacksquare\blacksquare}^{Loan}$	$[EU]$	$Bank_{\blacksquare\blacksquare}^{Com}$ $Bank_{\blacksquare\blacksquare}^{Cap}$ $Bank_{\blacksquare\blacksquare}^{Res}$ $Bank_{\blacksquare\blacksquare}^{Lab}$	$[EU]$ $[EU]$ $[EU]$ $[EU]$
$Lab_{\blacksquare\blacksquare}^{Lab}$	$[h]$	$Res_{\blacksquare\blacksquare}^{Res}$ $Res_{\blacksquare\blacksquare}^{Good}$	$[kg]$ $[G]$	$Com_{\blacksquare\blacksquare}^{Lab}$ $Com_{\blacksquare\blacksquare}^{Res}$ $Com_{\blacksquare\blacksquare}^{Good}$	$[h]$ $[kg]$ $[G]$			$Cap_{\blacksquare\blacksquare}^{Good}$	$[G]$				
In- and Outflows for Types Asset ■ ■ Liability ■ ■													

(1)

The in- and outflows of the accounts are left ■ and right ■ for Asset ■ and right ■ and left ■ for Liability ■ accounts. The company Com produces from 2 real input factors Lab $[h]$ and Res $[kg]$ of the

Labor and Resource Owners (Lab, Res) a real good (Good [G]) with the help of the loan contract between company Com and Bank. There are 11 nominal (in [EU] monetary units) and 9 real (in [kg, h, G] units) accounts, 14 are Asset and 6 are Liability accounts. ■

Definition 2.3 (Accounts as Code). Equation (2) shows the accounts written as code with initial and repeated endowments. The initial endowments at $t_0 = 0$ of the company are 110 for Lab and 20 for Res. The repeated endowments in each period, $\forall t > t_0$, are 100 for Lab and Res in real units.

$Lab(A)$	(L)	$Res(A)$	(L)	$Com(A)$	(L)	$Cap(A)$	(L)	$Bank(A)$	(L)
$AccLabBank$		$AccResBank$		$AccComBank$	$AccComLoan$ $AccComDiv$	$AccCapBank$ $AccCapDiv$		$AccBankLoan$	$AccBankCom$ $AccBankCap$ $AccBankLab$
$AccLabLab, 100, \forall t$		$AccResRes, 100, \forall t$		$AccComLab, 110, t_0$ $AccComRes, 20, t_0$ $AccComGood$		$AccCapGood$ $AccResRes$ $AccCapGood$			
$AccLabGood$		$AccResGood$							

(2)

Definition 2.4 (Bookings as Flows). The 8 macroeconomic bookings of the 5 agents in the macroeconomic accounting system define the interactions as channels on which the payments flow in between the 20 T-accounts. Since we not only manage nominal but also real T-accounts these involve not only payment flows but also flows of real goods, resources and labour, where the real flows are measured, evaluated and kept consistent with the flows in nominal units. The 6 bookings 1,2,3,4,6,8 encompass 3 agents while bookings 5,7 only 2 agents. A quadruple accounting system always books in at least two microeconomic or double-entry accounting systems of the agents. The real T-accounts are involved in bookings 1,2,3,4,8 but not in bookings 5,7, which are nominal bookings of the loan contract. Booking 6 is between nominal T-accounts as the flow of dividends. The 100 new labor and resources every period and the 110 and 20 initial endowments of the company can be, but are not yet modelled elegantly as parts of the universal initial and final objects of some category of resources. We see the problems involved popping up in the simulated data of increasing and zero stated real T-accounts. This is to be combined into properly booked dynamics of the endowments. The stability of the system is in need to be checked over different processes and endowments to evaluate how the monetary system copes with them and their shocks. The 110 and 20 initial real endowments of the company and the 100 renewable ones at Lab and Res are just one set of parameters where stability is demonstrated to be reached but a careful sensitivity analysis like in Harenberg et al. [2019] is needed as well. Categorically, we can check for the analytic covergences in terms of the atomic analytically factorized and decomposed functions of transformations. By that nonlinear functions in economic theory formulations can be taken to modern methods of advanced numerics like cohomologies, homotopy or sheaf theoretical methods.

The price functor transforms the units between real and nominal account types. We handle functorially the types and units of both, state and flow quantities. Flows translate between units of type $[t]$ and $[t+1]$ of states and verify the consistency with the units of flows being $[[t+1] - [t]]$. Beware that the last expression is a type expression, composed from types of two account states in two subsequent periods by the $-$ as a type operator. The consistency between in- and outflows and between real and nominal units is represented by functors with natural transformations verified by the pullback constructions. We distinguish between the channels of the bookings on the one side which we create functorially and the bookings themselves on the other side, which transfer information and value via the channels as morphisms in the category of accounts.

This is an important distinction and it is to be separated from the econometric way to understand *structurality*. In *structural econometrics* we mean by *structural* that decisions are derived as maximisations as opposed to the way macroeconomics has been conducted since Keynes, by postulating causalities between macroeconomic aggregates like inflation or interest rates without stating how they come about from individual decisions. This is the program of the microeconomic foundation of macroeconomics. However, these kind of structural theories are still missing the macroeconomic foundation namely stating whether micro decisions are consistent with the macroeconomic aggregates which is what we do in this paper in its very basic form by stating how a macroeconomic aggregate is to be represented mathematically, which we answer by colimits. The decisions in MoMaT are not structural in the decision theoretic sense but we type them by universals which can represent maximizations or even more general selection functions, see Ghani et al. [2018]. *Structural* in the sense of *institutional* are categorically definable as functorial dynamics which can represent the change of domains and codomains of transformations. We can take home that institutional economics is functorial and that universals are suited to represent an *institutional structurality* but also the *structurality of explicit decision making* derived from maximising utilities or preferences of agents subject to constraints.

An interesting feature is that bookings may occur logically simultaneously, i.e. in parallel. Even more interesting is that they do not need to occur simultaneously in real time. This even more points to the need for parallel and sequential combinators to check consistency conditions, including plain vanilla logical modalities of necessity and possibility of truth or should and could as legal deontic modalities. These technologies are available as coalgebraic constructions. The simplest algebra is probably formed by addition on natural numbers of type $\mu : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$, defined by $\mu(n, m) = n + m$. The simplest coalgebra is probably the observation of the first element of an infinite list of natural numbers of type $\Delta : N^\omega \rightarrow N \otimes N^\omega$ with $\Delta(n) = n_0 \otimes (n_1, n_2, n_3, \dots)$. It splits an infinite list of natural numbers $n = (n_0, n_1, n_2, n_3, \dots)$ also called stream into a first (or finite first part) of element n_0 and the infinite rest of the stream $(n_1, n_2, n_3, \dots)$. We see that coalgebras $X \rightarrow \mathcal{F}(X)$ are dual to algebras $\mathcal{F}(X) \rightarrow X$, arrows reversed, see Rutten [2000]. Coalgebras are the mathematics of infinite streams and trees and formalise observable data of unobservable structures which in economics are for example underlying the well known Kalman filter, see Abramsky and Winschel [2015] for a game theoretical economic application of infinite trees.

In the Lucas Critique the observing econometrician agent and the observed agents, within the theory of the econometrician, and some logically needed statement of the true state of the system might be differentiated by the amount of information available to the agents about the system. These differences are conceptualised by the notion of *model communism*: do the observed and the observer share the same information and model (i.e. theory)? In quantum physics the observer changes the system while in the economics of the Lucas Critique the observed system also changes the observer, the laws of the dynamics are not given by the universe (or if preferred by God) as in classical mechanical or quantum physics. In physics these kind of discussions are headed under the notion of multiple Lapacian daemons.

As a starting point of these deliberations we can use the categorification of Tarski's theory, model and homomorphisms by Lavwere's conceptualisation thereof by category, functor and natural transformations. Categories representing system states and theories thereof and functors being views on the system with natural transformations to encode the relation of the observer and the observed. As programs and formulas, the algorithms used by the observer and the observed might be the same, implementing for example some likelihood functions for classical or Bayesian estimators. However, even this set up might not be general enough since the relations of the observed and the observer do change dynamically as the core of their interaction, the theory does change the system and the system the theories. Hence, the suitable mathematical generalisation of natural transformations to dynamically and endogenously changing relations and consistency conditions between both has to be found. The notion of the true state of the economy which includes all agents and their perfect or imperfect information about the system is moreover a case for emergent properties in complex systems, like formalised in the MES of Ehresmann and Vanbreemersch [2007]. The discussions in the Dagstuhl seminar (Abramsky et al. [2015]) on the coalgebraic semantics of reflexive (Lucas Critique resolving) economics can be now framed within an extended MoMaT. The operations of the observer and the observed might be algorithmically the same, namely the available AI and econometric tech for both, the observer and the observed, but economically they are different, being possibly based on different data and priors processed. We might think about the provision of the observer's or econometrician's information as an institution to coordinate the informations of the observed agents in the economy. How all these concepts unite and how to categorify them, is an interesting topic for future research.

These theoretical problems do start with the question of what kind of processes are involved in macroeconomics and the simplest thereof are what we model in MoMaT as macroeconomic bookings. In order to categorify them, we first take a look on the dynamics described by the 8 macroeconomic bookings involved in the following table.

1. Lab sells Lab to Com	$1a : \text{Lab}^{\text{Lab}} \xrightarrow{h:EU} \text{Lab}^{\text{Bank}} \quad 1b : \text{Com}^{\text{Bank}} \xrightarrow{EU:h} \text{Com}^{\text{Lab}} \quad 1c : \text{Bank}^{\text{Com}} \xrightarrow{EU} \text{Bank}^{\text{Lab}}$
2. Lab buys Good from Com	$2a : \text{Lab}^{\text{Good}} \xleftarrow{G:EU} \text{Lab}^{\text{Bank}} \quad 2b : \text{Com}^{\text{Bank}} \xleftarrow{EU:G} \text{Com}^{\text{Good}} \quad 2c : \text{Bank}^{\text{Com}} \xleftarrow{EU} \text{Bank}^{\text{Lab}}$
3. Res sells Res to Com	$3a : \text{Res}^{\text{Res}} \xrightarrow{kg:EU} \text{Res}^{\text{Bank}} \quad 3b : \text{Com}^{\text{Bank}} \xrightarrow{EU:kg} \text{Com}^{\text{Res}} \quad 3c : \text{Bank}^{\text{Com}} \xrightarrow{EU} \text{Bank}^{\text{Res}}$
4. Res buys Good from Com	$4a : \text{Res}^{\text{Good}} \xleftarrow{G:EU} \text{Res}^{\text{Bank}} \quad 4b : \text{Com}^{\text{Bank}} \xleftarrow{EU:G} \text{Com}^{\text{Good}} \quad 4c : \text{Bank}^{\text{Com}} \xleftarrow{EU} \text{Bank}^{\text{Res}}$
5. Com gets Loan from Bank	$5a : \text{Com}^{\text{Bank}} \xrightarrow{\text{EU:Loan}} \text{Com}^{\text{Loan}} \quad 5b : \text{Bank}^{\text{Com}} \xrightarrow{\text{EU}} \text{Bank}^{\text{Loan}}$
6. Com pays Div to Cap	$6b : \text{Cap}^{\text{Div}} \xrightarrow{\text{Div:EU}} \text{Cap}^{\text{Bank}} \quad 6a : \text{Com}^{\text{Bank}} \xrightarrow{\text{EU:Div}} \text{Com}^{\text{Div}} \quad 6c : \text{Bank}^{\text{Com}} \xrightarrow{EU} \text{Bank}^{\text{Cap}}$
7. Com repays Loan to Bank	$7a : \text{Com}^{\text{Bank}} \xrightarrow{\text{EU:Loan}} \text{Com}^{\text{Loan}} \quad 7b : \text{Bank}^{\text{Com}} \xrightarrow{\text{EU}} \text{Bank}^{\text{Loan}}$
8. Cap buys Good from Com	$8b : \text{Cap}^{\text{Good}} \xleftarrow{G:EU} \text{Cap}^{\text{Bank}} \quad 8a : \text{Com}^{\text{Bank}} \xleftarrow{EU:G} \text{Com}^{\text{Good}} \quad 8c : \text{Bank}^{\text{Com}} \xleftarrow{EU} \text{Bank}^{\text{Cap}}$

(3)

The arrow heads indicate inflows while the arrow origins denote outflows in between the T-accounts. However, this interpretation as flowing from out- to inflows in T-accounts does not always hold. Some arrows in bookings 5,6,7 are missing and their would-be-captions are shown in red. In micro booking 5a both T-accounts involved increase as well as both in micro booking 5b. This ultimately, speaking in high level concepts, reflects the fact that the bank is securitising loans by its own assets, a bankrupt company not repaying the loan implies a loss of the bank. We do see that in the macroeconomic invariance of equation (9) formally stated as a zero condition to be fulfilled by a consistent system, i.e. an non-bankrupt bank. This happens as well in creating loans for banks by the central banks as a prerequisite of a functional and operating bank, which clarifies that banks to NOT create money since then, they could never go bankrupt and the investment processes of banks (selecting suitable companies) would not improve due to the natural selection of banks implemented in markets for loans. The universality of the pullback, as to be shown later, representing the *learning* of the banking sector, would collapse and the economy would not be adaptive but inherently unstable, just as centrally planned economies are. Their macroeconomy is understood to be unable to properly process available microeconomic informations, like some well-known inabilities of a company or entrepreneur at the microeconomic level, not transported to the macroeconomic level, where control decisions are located.

The flows and their directions involved in bookings 5,7 operate on a different level compared to the other bookings. There it involves two temporally opposing micro bookings - the investment inflow to the company and repayment outflow from the company back to the bank. In micro booking 7a both T-accounts decrease as in 7b when the loan is repaid and the risk of the bank vanishes. Booking 6a shows dividend debt settlement from company to the capitalist via bank transfers. Here, unlike in booking 5,7 of investment and repayment, there is no flow back. It is just the profit which is actually typed by the capitalist agent or role, which we could equally leave in an T-account of the company. But then, we would need to model the company to be a consumer as well, since the point of profits is to consume. It is a modelling choice without any more insights into the nature of monetary systems. The unclear structure of this core functionality of monetary systems are to be resolved once we type the entities involved. To anticipate, we will find that the functors must really be on another level, i.e. being typed as so called covariant or contra variant functors, pointing in opposite directions, for a consistent whole of the type of flows of values in real and monetary units, in between the involved agents and their T-accounts. Especially confusing is that there are real flows which interactions with nominal flows which will admittedly not be fully clarifiable without initial and final objects and some resource theory which we will not approach in this paper. However, we do need anyway to first type the product categories of real and nominal account categories with co- and contra variant functors operating on them, done in Section 4.5.3 for properly typing the 8 macroeconomic bookings. ■

Definition 2.5 (Invariances). The invariances of the macroeconomic accounting system encompass the 4 bank accounts of the agents Labor, Resources, Company, Capitalist, the loan contract of the Company with the Bank and one macroeconomic invariance as solvency requirement for the Bank. The bank invariances $I_{Lab}^B, I_{Res}^B, I_{Cap}^B, I_{Com}^B$ say that the bank accounts of the agents Lab, Res, Cap, Com are managed by the Bank which must have the same state or balance in the double-entry micro accounting systems of the bank and the agents. The invariance I_{Com}^L says that the loan contract of the company with the bank must be the same in the Bank's and in Company's Loan accounts. The macroeconomic invariance I_{Mac} says that the bank must always be balanced, because it is insolvent or bankrupt otherwise, which is the core goal of managing liquidity in banking. In the real world the banks all together balance each day at the

interbanking market organised and regulated by the central bank. This market maintains a macroeconomic accounting systems just as markets and banks do in general. In MoMaT the macroeconomic invariance I_{Mac} represents this market's or the Bank's functionality. An extension by including a central bank would bring about cash and money-in-circulation T-accounts by money creation at the central bank, with appropriate functors and natural transformations representing consistency conditions on the appropriate T-accounts. A nominal exchange rate regime with central banks has more of this kind of accounts, functors and consistency conditions. The recursive program calculates the invariances as calculations that must always produce a zero, being by that as a predicate or test that the T-accounts are equal invariantly for all periods, bookings and flows. The categorical formulation uses pullbacks to ensure consistency between accounts and pushouts to synchronize flows between accounts as computations.

$$I_{Lab}^B = Lab_{\square\square}^{Bank} - Bank_{\square\square}^{Lab} \quad \text{Labor Bank Invariance} \quad (4)$$

$$I_{Res}^B = Res_{\square\square}^{Bank} - Bank_{\square\square}^{Res} \quad \text{Resource Bank Invariance} \quad (5)$$

$$I_{Cap}^B = Cap_{\square\square}^{Bank} - Bank_{\square\square}^{Cap} \quad \text{Capitalist Bank Invariance} \quad (6)$$

$$I_{Com}^B = Com_{\square\square}^{Bank} - Bank_{\square\square}^{Com} \quad \text{Company Bank Invariance} \quad (7)$$

$$I_{Com}^L = Bank_{\square\square}^{Loan} - Com_{\square\square}^{Loan} \quad \text{Company Loan Invariance} \quad (8)$$

$$I_{Mac} = I_{Lab}^B + I_{Res}^B + I_{Cap}^B + I_{Com}^B + I_{Com}^L \quad \text{Macroeconomic Accounting Invariance} \quad (9)$$

These differences test for identical T-account states in two double-entry bookkeeping systems. The need for these consistency conditions arises from the booking of one number in two microeconomic accounting systems being (decentral) subparts of the macroeconomic accounting system. A resource theory based on linear, resource aware logic can be formulated to check the invariances as a proof (like in a logical program in Prolog). The formulation as differences obscures that the invariances are equality constraints. The liabilities we are interested in, exist between bank and company, as loans that resolve the desynchronization of payments for input factors and goods. This is the core of MoMaT where money exist to solve this desynchronization. The bank invariances arise as the tit for tat content of the contract of the agents having a bank deposit. This is the core of the usual approach in money theory to resolve the double coincidence of wants of barter exchange without money. It is a rather simple functionality of the banking systems as opposed to the desynchronization problem and both need to be modelled at the same time, which we see in the macroeconomic invariance I_{Mac} of the bank. The bank is balanced, if the sum of the bank balances of the agents' deposits and the loan account are balanced into the macroeconomic invariance I_{Mac} of the bank, the only inhabitant of the *macroeconomic* level in MoMaT. This closes the relation that for each debtor there must be a creditor, the agents in any loan contract. All these invariances guarantee a sound and faithful monetary macroeconomic accounting with loans and repayments over time and bank deposits for payments of real goods, labor and resources exchanges. We have the contra and covariant loan and bank deposit contracts represented in equation (8) as $I_{Com}^L = Bank_{\square\square}^{Loan} - Com_{\square\square}^{Loan}$ where the $-$ obscures the contra variance which could be highlighted by $I_{Com}^L = Com_{\square\square}^{Loan} - Bank_{\square\square}^{Loan}$. ■

Definition 2.6 (Parameters). We use the following 22 parameters $\in \mathbb{R}^{22}$ to specify the economic behaviours, decisions and contracts as the money theory of MoMaT (actually $\tau \in \mathbb{N}$).

Parameter	Value	Description	Parameter	Value	Description
Investment			Prices		
τ	= 10	investment length	p_r	= 25.0	price of resources
λ	= 0.2	investment to labor vs resources	p_l	= 12.0	price of labor
σ_a	= 20.0	learner sigmoid constant	p_0	= 30.0	initial price of good
σ_b	= 480.0	learner sigmoid nominator	Production		
σ_c	= 200.0	learner sigmoid DemandSurplus factor	γ	= 0.75	elasticity of substitution
Consumption			α	= 0.42	factor
ρ_r	= 0.8	resource's part to consume	Decay		
ρ_l	= 0.95	labor's part to consume	β_l	= 0.95	labor's good
ρ_c	= 0.6	capitalist's part to consume	β_r	= 0.7	resource's good
Profit			β_c	= 0.6	rapitalist's good
μ	= 0.5	markUp	New Resources		
ω	= 0.5	windfall profit	ν_l	= 100.0	new labor
Dividends			ν_r	= 100.0	new resources
δ_c	= 0.15	dividend for companies	Memory		
δ_b	= 0.4	dividend for banks	$\mathcal{H}_{wages}$	= $\mathbb{R}_{\geq 0}^{\tau=10}$	wages to pay by contracts
			$\mathcal{H}_{repays}$	= $\mathbb{R}_{\geq 0}^{\tau=10}$	repays by loan contracts

(10)

2.1 Economic Decisions

Definition 2.7 (Decisions). We manage a memory that stores and remembers future payment obligations from past contracts on labor and loans. The parameter $\tau = 10$ defines the length of the memory. It contains τ real numbers for 10 wage and repayment payments as obligations from labor and loans contracts, given by equation (11) as the result of past investment decisions. Labor and loan contracts have very different risks or termination conditions and we opt for the simplest case. We take the 2×10 payments as those to be paid from both contracts for 10 periods. The memory remembers these payments that are to be done in each of the following 10 periods.² The memory starts initially with no obligations, represented by 10 zeros in the vectors which reduce to 9 in the first period until in the 10th period the memory is full.

$$\begin{aligned} \text{WagesPayment} &= \sum_{i=1}^{\tau=10} \mathcal{H}_{\text{wage}}[i] \\ \text{RepayPayment} &= \sum_{i=1}^{\tau=10} \mathcal{H}_{\text{repay}}[i] \end{aligned} \quad (11)$$

The economic decision on **consumption** is given by equation (12). Consumption of labour, resource agents and capitalist are assumed to be simple fractions of the state of their bank accounts. These are then the amounts each consumer decides to spend on goods measured in monetary units and together they sum to the total **Demand** D of the economy.

$$\begin{aligned} C_{\text{Lab}} &= \rho_l \text{Lab}^{\text{Bank}} \\ C_{\text{Res}} &= \rho_r \text{Res}^{\text{Bank}} \\ C_{\text{Cap}} &= \rho_c \text{Cap}^{\text{Bank}} \\ \text{Demand} = D &= C_{\text{Lab}} + C_{\text{Res}} + C_{\text{Cap}} \end{aligned} \quad (12)$$

There are three parameters more that govern consumption, $\beta_l, \beta_r, \beta_c$. They specify the decay of goods as a percentage of outflows as a kind of warehouse technology. The economic meaning is that hording and not consuming costs. The parameters could be explicitly derived as a universal by utility maximisation, for example, rationalising the behavioural decisions as an economic tradeoff between consuming today versus hording and consuming tomorrow.

The producer wants to pay the wages, repay the loans and make a profit, calculated as a markup on the costs which we call the **DemandPlan** D^* of the company. It is important to note that this is a decision about future events. The company only plans to realise the profits discribed by a markup μ over the costs.

$$\begin{aligned} D^* &= \text{WagesPayment} + \text{RepayPayment} + \text{Profit}^* \\ &= (\text{WagesPayment} + \text{RepayPayment}) * (1 + \mu) \end{aligned} \quad (13)$$

We could explicitly take the future into account in a maximisation of goals or utilities as a behavioural model. We could use structural econom(etr)ic theories like in (Winschel and Krätzig [2010]) or categorical and compositional game theory by (Ghani et al. [2018]) with a teleological causality from the (contra variant) future while observing the (covariant) past. The decision theories can be all united in the categorical universal constructions as the top level of our constructions. Universals unite and generalise optimality conditions and may be so called reduced form theories, like in equation (13), structural theories of intertemporal maximisations, strategic open games, or in the most general way as an adjunction of functors.

The market matches **Demand** D of consumers and **DemandPlan** D^* of the producing company in **DemandSurplus**. This happens by the price adjustment behaviour specified in equation (16) also influencing the investment decisions in equation (17).

$$\text{DemandSurplus} = D - D^* \quad (14)$$

²The memory is used to calculate the sum of the last agreed $\tau = 10$ payments. We implement this as a FILO (First In Last Out) stack. The last entries are forgotten at the end of the period and the numbers calculated in the current period from the investment decision and how this affects future wage payments and loan repayments are written to memory. The implementation in Julia is a simple concatenation with vectors and elements therein. `1:end-1` takes the elements from the first to the second-to-last element of vector `hist`, adds `newelem` before all these other elements via the operator `;` and from that forms the new vector with the vector generator `[]` representing the new returned memory. The implementation of the memory function `p2H` is `p2H(hist::Vector{Float64}, newelem::Float64) = [newelem; hist[1:end-1]]`, in equation (21). This is the `memorize` function of the memory. The `recall` reads the state `wagehist` and `repayhist` and adds them by sum as an implementation of equation (11). In the recursive program in Appendix A.2.1 in lines 66-68 the global `push_to_history` and the local `p2H` in line 74, implement the memory function.

The **GoodProduction** follows a standard Cobb-Douglas production function with input factors of labor and resources in equation (15). The real accounts Com^{Lab} and Com^{Res} represent the state of the warehouse of the company. We denote the unit of the good and its accounts to be [G]. Type G is not formally defined in our categorical programs but goods are typed and integrated into the real and nominal verification by functors and natural transformations. The custom units are so to say empty such that the types of transformed units are different but look the same.

$$GoodProduction = 1 + \alpha (Com^{Lab})^\gamma (Com^{Res})^{1-\gamma} \quad (15)$$

The good production and demand surplus together drive the **GoodPrice** in equation (16), as a fraction of DemandPlan D^* of GoodProduction starting with 30.0 and a additional price tag if the demand is larger than the planned demand of the company. This expression denotes a mix of aspects and types with the initial state of the price being a parameter InitialGoodPrice used in the decision function which is a program of an if_then_else condition. This defines a locally linear function with kinks by inequalities.

$$\begin{aligned} GoodPrice &= 30.0 \text{ in } t_0 \\ &+ \frac{DemandPlan}{GoodProduction} \\ &+ DemandSurplus * \omega \text{ If } DemandSurplus > 0.0 \end{aligned} \quad (16)$$

The **investment** decision is the most important one in the monetary economy. It happens after all other decisions are made, the markets have equilibrated and the company's success is determined to inform the investment decision. It is a sigmoid activation of the $DemandSurplus$ in equation (17). It represents the learning of the banking sector to invest in companies and to share the investment among resources (18) and labour (19). The repayment scheme of 10 equal repayments over 10 years is given in equation (20). The investments are saved in the memory as a FILO stack in equation (21). In general, if the repayment scheme were not specified to be 10 equal fractions of the investment, but any general function of time or other variables, the memory would be an element of a functional space or category.

$$Investment = \sigma_a + \frac{\sigma_b}{1 + e^{-\frac{DemandSurplus}{\sigma_c}}} \quad (17)$$

$$InvestmentRes = Investment * (1 - \lambda) \quad (18)$$

$$InvestmentLab = Investment * \lambda \quad (19)$$

$$Repayment = Investment / \tau \quad (20)$$

The investment decision **updates the memories** of wage payments and repayments, which also contain the other payment obligations of the past, contracted among company, labor agent and bank. Since every number remembered in the memory is there for $\tau = 10$ periods, each number appears 10 times in the sums of repayments over the memory in equation (11). The FILO stack is a simple way to implement the remembering of the 10 constant payment schema of the labour and loan contracts.

$$\begin{aligned} \mathcal{H}_{wage} &= p2H(\mathcal{H}_{wage}, InvestmentLab) \\ \mathcal{H}_{repay} &= p2H(\mathcal{H}_{repay}, Repayment) \end{aligned} \quad (21)$$

The **dividend** decision is programmed in the bookings as the state updates of the T-accounts. The dividend depends on the actual realisation of the turnover and realised profit of the company calculated in $Diff$. The calculation is again a non-linear, kinked function arising from the inequality checks for positive profits and bank account state of the company, as a mix of concerns and aspects.

$$\begin{aligned} Diff &= InAccComBank - outAccComBank \\ DividendDecision &= Diff * \delta_C \text{ If } Diff > 0.0 \\ &+ AccComBank * \delta_B \text{ If } AccComBank > 0.0 \\ &+ 0.0 \text{ If } AccComBank < 0.0 \end{aligned} \quad (22)$$

Diff like some of the other formulations of the theory mixes aspects of behavioural assumptions, checks of conditionals in programs and mathematical expressions. This is all to be properly sorted out by the types of the parts and the whole. Also the missing of explicit sequential, parallel and hierarchical combinators

of composable parts of the systems makes economic interpretation complicated and programming difficult. The same for popping up endowments and used up production inputs without initial and final objects in a category of resources. The contractual content of the memory vectors is of concern for legal tech by modal logic where coalgebras for infinite time series and stochastics are needed anyway. This then allows to include the dividend, loan and bank contracts as timed expressions as musts and cans which can be mapped into programs and formulas for unambiguous and transparent legal obligations and options implemented by logical programs or theorem provers. String diagrams and commutative diagrams will be helpful to disentangle these concerns and aspects of programs and mathematical types by making the sequential, parallel and hierarchical structures explicit. Modal logic for theory formulation in logic programs (like Prolog) based on coalgebraic semantics is for further research. ■

2.2 Recursive Program

We have implemented three programs in the Julia language, a simply typed recursive, a categorically typed recursive and a categorical program. The listing are in Appendix A.2. The recursive program in Appendix A.2.1 takes the account states as the important state of vectors of real numbers beside memory, parameters and initial states and endowments. The time series are calculated as an iteration or recursion using initial values, parameters and memory to update the states of the T-accounts. The categorically typed recursive program in Appendix A.2.2 implements categorical structures as record types, called `struct` in Julia, with fields for the actual categorical data. The categorical types as `struct` are wrapped around the numbers calculated in an iteration or recursion. The categorically typed program implements a minimal categorical state of parameters, flows and accounts as categories and adds by generators the theory of five agents, parameters, initial states, accounts, bookings and consistency conditions. These generators are the helper functions as they are called in the categorically typed program. This implementation, even so it does not implement the actual evolution of time as natural transformations (even so the functions are defined), makes the advantage of categorical modelling more explicit than the third, categorical program. We see the abstract syntax of the compiler at work as the categorical `struct` being defined and used as a custom type by the multiple dispatch system of Julia. We see the endogenous economist or econometrician at work in the helper functions, which allow an agent in the theory or us, as observers, to construct the theories or econometric models represented in categories, functors and natural transformations. The agent in the theory could write this paper as an explanation of the results of the simulated data. Finally, the categorical program is in Appendix A.2.3. Time is an evolution represented by applying the endo functor to the category of the economy $\mathcal{C}_{Economy}$, the whole macroeconomic object.

In Figure 2.2 we see the time series plots of the simulation of the recursive program. The recursive formulation is computationally effective because it delivers the correct results. We see this in the invariance variables, which are (red) zero for all calculations in the periods. The period grows as a time index, also the resources and labor and the goods produced. This reminds us that the deletion and creation of resources in set theory is functorial in the domains and codomains of functions. Since category theory is in need of dependent types (which is implemented in the categorical data types), we can use them for structural theories of networks. The economics of double entry bookkeeping is about networks and keeping decentralised exchange in a consistent accounting and booking system. We extend these consistency needs from decentralised needs in microeconomic accounting to hierarchical needs in macroeconomic accounting.

Definition 2.8 (Recursive State). The state of the recursive formulation is given by

$$S^{rec} = (\theta :: \mathbb{R}^{22}, \mathcal{H}_{wage} :: \mathbb{R}^{10}, \mathcal{H}_{repay} :: \mathbb{R}^{10}, \text{Acc} :: \mathbb{R}^{20}) \quad (23)$$

where S^{rec} is a composite record type of vectors of parameters $\theta :: \mathbb{R}^{22}$, of wage history $\mathcal{H}_{wage}$, of repayment history $\mathcal{H}_{repay}$, and of 20 states of accounts Acc , each a real (positive or zero, non-negative) number.

In the recursive program bookings are state transitions of the changing state as balances of the accounts. Parameters and initial values are constant parts of the state. Beware, the categorical state will be the whole mathematical description of the theory and economy, not just the balances of the T-accounts. Accordingly, all parts of the economy and theories can be changed, functorially, we can elegantly model institutional changes as part of the decisions of the agents involved.

The company has an initial endowment of resources and labor in its real accounts $Com_{\blacksquare\blacksquare}^{Lab} = 110[h]$, $Com_{\blacksquare\blacksquare}^{Res} = 20[kg]$. The memory of wages and payments $\mathcal{H}_{wage}$ and $\mathcal{H}_{repay}$ are used and updated at the

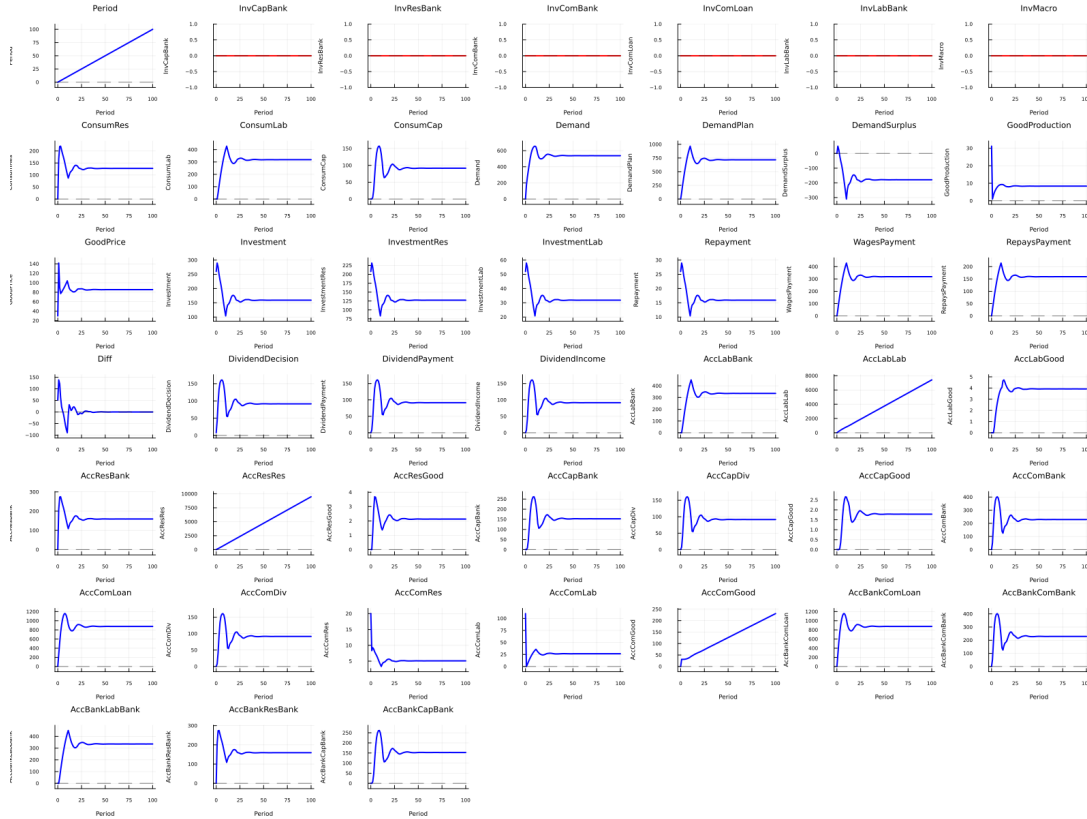


Figure 2.2: Time series plots of variables in the recursive simulation

beginning and end of each period. The memory is part of the state of the system and the state of the system is part of the state of the recursive program. The resources and labor endowments of labor and resource agents pop up each period in the real accounts as described by two parameters $\nu_l = 100[h]$, $\nu_r = 100[kg]$.

The new real resources endowments every period are a main dynamic driving force. They are transferred by their owners into the units of the bank account for buying goods. The company buys labor and resources to produce the goods according to a production function. The company pays dividends and repays loans while the profit, labor and resource owners consume. This in turn gives revenues to the company and the bank accompanies all these events by granting loans, making bank transfers and receiving loan repayments. This is what happens economically and what is translated into the recursive program. The state is used to iterate on using and updating the memory, calculating banking, loan and macroeconomic invariances, the behavioral decisions and the actual bookings in the accounts as state transitions. The recursive formulation uses what we call simply typed numbers and untyped equations for the invariances, memory usage and update, decisions and bookings.

Categorifying the recursive formulation means to type parameters, state and lift structural morphisms over operational bookings. The equations in the Julia code uncover the mathematical need for a type discipline which we take to the mathematical extreme abstraction of the categorical data types of categories, functors, natural transformations and calculations as pullbacks and pushouts. In the categorical formulation we also use some helper functions to define, update and apply the categorical data structures and constructions for the concrete agents, accounts and bookings. In the categorical formulation the mathematical formulas are programs.

In the recursive formulation, the state is a vector of real numbers (actually, τ is of type natural number). The recursive state S^{rec} is defined in equation (23). The categorical state S^{cat} encompasses categories, functors, natural transformations in equation (25). The recursive state S^{rec} can be derived from the categorical state $S^{rec} = \mathcal{F}(S^{cat})$ by a functor. The helper functions for managing the categorical state are mathematically formulated and allow an endogenisation of the economist as modeler or the econometrician as an estimator of parameters of empirical theories on data of the model. In Figure (2.2) we see the time series plots of the recursive formulation of the five agent theory of MoMaT.

3 Categories for Macroeconomics

We have seen a simply typed recursive formulation and program of the macroeconomic theory of MoMaT. The formulation has difficulties in the mathematical and software specification. We reformulate the key economic issues from the point of view of the categorification to do.

The categorical implementation forms a domain-specific language (DSL) for monetary macroeconomic accounting. This is best seen in the categorically typed recursive program. The data structures are defined, wrapped around the calculated numbers and the creation of the categorical structures are explicit in the helper functions. However, the natural transformation functions are not used, i.e. the iteration is not a categorical application of natural transformations but simply an iteration. The implementations represent the theory as a category with objects as accounts, morphisms as monetary flows between the accounts, pullbacks and pushouts for consistency verification and calculations, functors for different views on the system, an endofunctor as top level functor on the category of the economy as dynamics or evolution.

While the recursive implementation is a rather simple program of the dynamics, the categorical version provides a more abstract representation that enforces the mathematical structure of the theory. All three implementations produce the same simulated data. The categorical formulation offers several advantages. Mathematical rigor through categorical structure. Clear separation of abstract theory and concrete computation. Multiple interpretations via different functors. Natural representation of temporal evolution and of type checks. Built-in consistency checks through categorical constraints. A compiler from a high-level language (DSL) to Julia code. The DSL allows to define a concrete syntax for different uses in science, engineering, or media. Proofability of code which is important for security checks and financial regulation and compliance. A separation of aspects in operational morphisms with the category as the state and functorial morphisms as the structural dynamics, possibly including a change in the context or institution. The theory is the code, making the working of economists explicit as typing and formulating the constraints and consistency conditions. Category theory connects economics to other sciences via the categorical interface.

The Fundamental Economic Problem: The MoMaT addresses the core challenge of capitalist production: temporal desynchronisation between production and consumption. Producers must pay their suppliers (workers and resource owners) before receiving payments from customers, creating a fundamental financing gap. This problem cannot be solved through simple exchange of paper money for solving intratemporally the double coincidence of wants. If it is extended to intermediate goods and companies we get the same intertemporal problem of MoMaT. To solve it, a sophisticated hierarchical credit system that enables intertemporal synchronisation is needed. We use money in MoMaT also as a neutral medium of exchange but the crucial role of money is in credit creation and debt repayment as the essential monetary structure in MoMaT. Therein we argue that credit creation and repayment through the banks' investments and repayments and not profit growth and money circulation are the essential features of the macroeconomic evolution that enables complex production in decentralised market economies organised by the division of labour. We will see that this financial technology can be stable, consistently typed, composable, implementable and verifiable.

The Five-Agent Economic Model: The closed economy with five distinct agent and sector types describes specific economic roles with suitable account structures. The Labor agent are workers who provide labor services and consume goods. They seek immediate payments for their services and avoid production risk. The Resource owners supply raw materials and resources who, like the workers, demand immediate payments and avoid production risk. The Company produces and has a fundamental financing problem. It must pay suppliers before receiving customer payments, requiring loans from the banking system. The Capitalist receives dividends and consumes only based on these profits rather than by owning productive resources. He represents ownership claims on productive assets. The Bank is the financial intermediary that provides loans to bridge the temporal gap between production costs and sales revenues and also processes payments. The Bank performs the system critical function of learning to aggregate risk and temporal coordination, the opposite of producing bad banks, booked or more like buried in black wholes of interbanking double-entry systems, central banks, public sector's and national accounting systems. The Bank is the only agent that can create loans, the other agents are only able to use money and the loans. The money is created implicitly in the MoMaT so far. In a next paper the central bank is to be typed as the entity which initiates the only booking of macroeconomic accounting that involves only one single double-entry micro system. By this booking the central bank has the authority to *create* money in its cash asset T-account with the opposite booking in the circulating money liabilities T-account, as discussed in Menéndez and Winschel [2025]. It will be a universal booking of the central banks that is uniquely authorised to create

money and therefore will be part of the learning pullback of the banking sector. For consistency some more accounts are needed for the contracts of banks with central banks but the investment functionality will stay at the banking level, searching and selecting investments. The creation of money is central bank's authority, to declare papers with registered IDs to be the legally accepted means of debt repayments in loan contracts, of debts in exchange contracts of goods for money or in the payments of the liabilities of the company to the capitalist.

The Eight Macroeconomic Bookings: The dynamics of the system are determined by eight fundamental events and macroeconomic bookings that represent all monetary and real flows in the system. All bookings are processed with bank intermediation for the payments of the dividends, production factors and goods. The details of the legal principles of abstraction and separation to legally type the events are quite intricate in their relation to the issues of interest and pitfalls of the theory of money, as discussed in Menéndez and Winschel [2025]. The legal principles may even involve intergenerational aspects like protecting the natural environment of future generations in §20a of the German constitution. Institutional goals or agents are usually modelled in economics with a planning horizon without known end, i.e. by infinite horizon models, for which coalgebraic semantics have been developed in computer science. The legal principles help us to disentangle the issues involved in contracts, like *quid pro quo*, or that opposing contracts of the original overall contract is settled by two mutual contracts on exchanging the promised goods or money among the exchanging agents. The details are not of interest here, but that macroeconomic bookings in different double-entry systems result from the fundamental activities as the eight bookings: 1. Labor sells Labor to Company: Workers provide labor services in exchange for wages. 2. Labor buys Goods from Company: Workers consume goods using their wages. 3. Resource owners sell Resources to Company: Resource suppliers provide inputs for production. 4. Resource owners buy Goods from Company: Resource suppliers consume goods from selling resources. 5. Company gets Investment as a Loan from Bank: The crucial credit creation that enables production and risk sharing. 6. Company pays Dividends to Capitalist: Profit distribution to shareholders. 7. Company Repays Loan to Bank: Credit contract deletion by repayments of the loans. 8. Capitalist buys Goods from Company: Dividend recipient consumes goods. Bookings 5 and 7 (loan creation and repayment) are the purely monetary transactions that solve the fundamental temporal problem. These involve only nominal accounts and represent the core of the credit system. Dividend payments is the other pure monetary transaction that does not involve repayments.

The Puzzle of Marx: The categorical framework sheds light on Marx's puzzle of profit realisation: How can profits be realized when workers don't have enough money to buy the products they produce? The light emerges through debt spirals rather than money circulation and profit dynamics. The question is actually why does it look like money circulates while the crucial dynamics is in the creation and repayment of debts. 1. Credit Creation: Banks create purchasing power by extending credit to companies. 2. Production Financing: Companies use credit to pay workers and resource owners before sales. 3. Consumption Enabling: Workers and resource owners can consume using their wages and revenues from selling the resources. 4. Profit Realisation: Sales revenue enables loan repayment and profit distribution. 5. Temporal Coordination: The credit system bridges the gap between production costs and sales revenue. This process creates spirals of debt and repayment that enable complex production without requiring workers to have sufficient money. The banking system performs the crucial function of risk aggregation and temporal synchronisation.

Categorical Implementation: The categorical ansatz provides several advantages over the recursive formulation. Type Safety: Categorical structures ensure proper typing like nominal vs. real accounts, asset vs. liability positions, and flow vs. stock variables. Compositional Structure: Categories, functors, and natural transformations make parallel, sequential and hierarchical relationships in economic decisions, institutions and accounting explicit. Universal Constructions: Pullbacks verify loan contracts and investment decisions, while pushouts aggregate consistency conditions across multiple double-entry systems. Diagrammatic Reasoning: String diagrams and commutative diagrams provide visual representations of complex economic relationships, making the parallel, sequential and hierarchical structures of economic processes explicit. The categorical state encompasses layers of embedded categories in the top level category of the economy and its endofunctor of time evolution. There are accounts, flows, parameters, two functors of price and flow transformations, and natural transformations for the functors. They ensure consistency across temporal, hierarchical, parallel sequential dynamics, nominal, real or any scientific units.

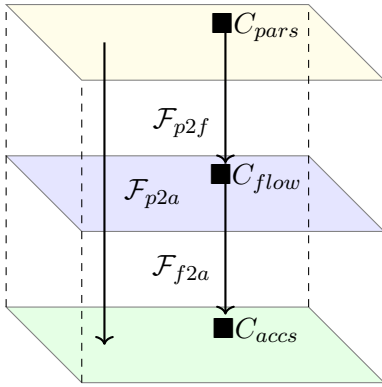
Hierarchical Risk Management: The model demonstrates hierarchical risk management through the banking system: 1. Micro Level: Individual agents maintain double-entry accounting consistency. 2. Macro Level: Banks aggregate individual risks and provide system-wide liquidity. 3. Temporal Risk: Credit creation enables production despite the timing mismatches of costs and revenues. 4. Systemic Stability:

Proper categorical typing ensures overall system consistency. This hierarchy can be extended to include central banks as "lenders of first liquidity" or government spending financed by taxation.

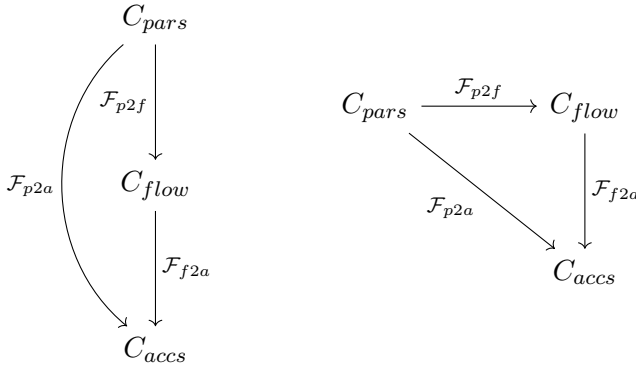
Implementation: The economic theory has been implemented as a recursive simulation which has to be typed for a categorical simulation. 22 Parameters: Rules for agent behavior and system dynamics. 8 Bookings: State transitions between periods to become the evolution by a functorial composition over time. 5 Invariance Conditions: Macro accounting consistency conditions constitute the contracts of the financial sector. Investment Function: Bank learning and verification process to become a universal construction to represent the generalized optimisation behaviour of the sector. Categorical Structures: Mathematical foundation for programming and proofs of systemic properties. We want to use category theory for typing a simulation, providing both theoretical insights and practical computational methods for monetary economics. MoMaT demonstrates a hierarchical risk management system through the banking system at the macroeconomic level. The mathematical formulation and the specification of the program are the same. The program in the high level operations of the DSL in terms of categories of banking is a type system, specifying what is to be compiled as the macroeconomic accounting system.

3.1 Categories as Economic Structures

Visualising the Macro of Economics: This section provides the basic categorical infrastructure needed to categorify the macroeconomics of the eight bookings. We model a macroeconomic or global structure and accordingly can use the geometric and visual tools of category theory first developed in algebraic geometry. We think about the theory as a stack of 3D layers of the category of parameters C_{pars} , flows C_{flow} and accounts C_{accs} .



Since we are not dealing with a 41×100 matrix of real numbers of 41 variables representing the state of a macroeconomic economy over 100 periods, but with a mathematically complete description, we need appropriate tools for dimension reduction and composition. Functors can also be used for 2D slices of the 3D layers of categories of geometric macroeconomic structures and to rotate layers around corners in triangles, for example.



Natural transformations between functors ensure consistency. The visualisation shows that parameters influence flows which in turn affect account balances, with natural transformations ensuring consistency between different paths between the system and its parts. The types, functors and universal constructions decompose and compose the system and its parts, zoom in and out as different views in the system, see Willems [2007]. The visualisation provides a comprehensive view of the categorical state, i.e. categories, functors, natural transformations and memory state while executing the program. The categorical ansatz is the IME's (Integrated Modelling Environment) of mathematics with a cockpit's view on a multi-layered

visualisation of the multi-sector-agent system. The purpose of an IME in economics is to observe and analyze foundational hierarchical structures and institutions that compose into governances of macroeconomic behaviour, to design institutions, i.e. meta-rules. Category theory provides tools into the mathematical foundations of this kind of structural economic dynamics. With the visual tools of category theory organigrams of organisations can become the program of the digital twin. The accounts can be visualized as categorical networks of graphs where vertices represent economic objects and edges represent morphisms, functors or natural transformations and universal constructions as generalisation of economic maximisation operators of economic theory. The simulation updates the categories, functors and natural transformations, rather than reals in vectors. The category is the state and the formulas are the program. We see during simulations how accounts balance, update and how categorical structures change, visualise, witness and calculate constructively the informations needed.

We will take accounts to be the objects in the fundamental categorical structure where the category is the state. There are different ways to define and implement the mathematical structures of MoMaT. In the categorically typed recursive program we take the state to be represented by three categories of parameters, flows and accounts and the memory of the contracts with functors and natural transformations for keeping always all units and accounts consistent. The category in Section 4.6 is finally used for the categorical program which is to be discussed in the next sections. For now, we start with a category of accounts.

Definition 3.1 (Category as State). The economic category $\mathcal{C}_{Economy}$ consists of:

- Objects $A \in \text{Ob}(\mathcal{C}_{Economy})$ - economic accounts with current value, as amounts of monetary, real or other units of account
- Morphisms $f, id_A \in \text{Mor}(\mathcal{C}_{Economy})$ - economic transactions with metadata
- Composition: $f : A \rightarrow B, g : B \rightarrow C \Rightarrow g \circ f : A \rightarrow C$
- Identity morphisms: $id_A : A \rightarrow A$ for each object A

Some of the objects in **periods 0, 1, 2** that represent the account balances:

$$\begin{aligned}\text{Ob}(\mathcal{C}_0) &= \{Lab^{Bank} : 0.0, Res^{Bank} : 0.0, Com^{Bank} : 0.0, \dots\} \\ \text{Ob}(\mathcal{C}_1) &= \{Lab^{Bank} : 0.0, Res^{Bank} : 208.0, Com^{Bank} : 52.0, \dots\} \\ \text{Ob}(\mathcal{C}_2) &= \{Lab^{Bank} : 52.0, Res^{Bank} : 273.19, Com^{Bank} : 190.50, \dots\}\end{aligned}$$

$$\begin{aligned}Lab^{Bank} &: \mathbb{R}_{\geq 0} \rightarrow \mathcal{C}_{\text{accounts}} \\ Res^{Bank} &: \mathbb{R}_{\geq 0} \rightarrow \mathcal{C}_{\text{accounts}} \\ Com^{Bank} &: \mathbb{R}_{\geq 0} \rightarrow \mathcal{C}_{\text{accounts}} \\ Cap^{Bank} &: \mathbb{R}_{\geq 0} \rightarrow \mathcal{C}_{\text{accounts}} \\ Bank^{Loan} &: \mathbb{R}_{\geq 0} \rightarrow \mathcal{C}_{\text{accounts}}\end{aligned}$$

■

3.2 Functors as Value Transformations

Definition 3.2 (Functor as Value Transformation). Functors as maps between categories are for transforming values between categories as representations or variables of interests. In MoMaT we have the price functor as a functor between the real and nominal categories of accounts. The second functor is between the states and the flow category, also transforming units and points of view. This ensures the stock flow consistency. Both are accompanied by two natural transformations to keep the functors consistent among each other. We take a look at the price functors. The price functor transforms values expressed in nominal units to values expressed in real units.

Object mapping :	$F(A) \in \text{Ob}(\mathcal{D})$	$\forall A \in \text{Ob}(\mathcal{C})$
Morphism mapping :	$F(f : A \rightarrow B) =$	$F(f) : F(A) \rightarrow F(B)$
Composition preservation :	$F(g \circ f) =$	$F(g) \circ F(f)$
Identity preservation :	$F(\text{id}_A) =$	$\text{id}_{F(A)} \forall A \in \text{Ob}(\mathcal{C})$

$$\begin{aligned}
\mathcal{F}_{\text{price}} &: \mathcal{C}_{\text{nominal}} \rightarrow \mathcal{C}_{\text{real}} \\
\mathcal{F}_{\text{price}}(\text{GoodPrice}) &: \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0} \\
\mathcal{F}_{\text{price}}(\text{LaborPrice}) &: \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0} \\
\mathcal{F}_{\text{price}}(\text{ResourcePrice}) &: \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0}
\end{aligned}$$

$$\begin{aligned}
\mathcal{F}_{\text{price}}(\text{GoodPrice}_0) &= 30.0 \rightarrow 30.0 \\
\mathcal{F}_{\text{price}}(\text{GoodPrice}_1) &= 141.7 \rightarrow 141.7 \\
\mathcal{F}_{\text{price}}(\text{GoodPrice}_2) &= 89.94 \rightarrow 89.94
\end{aligned}$$

Although in economics types are not used in theory as functors, like here where the reals are mapped to reals, the mathematical structure is nevertheless there, the functors. We can define any currency or real or custom unit of account. We have already noted the different units like $[EU]$ for monetary units or $[kg]$ for resources, $[h]$ for labor, $[G]$ for goods. The definition of custom types is not programmed but could be integrated into the functorial set up. However, important types are implicitly defined in the recursive code and are defined as the second functor for the flows between the accounts. The account types have units of account and their time differences in the flows as in equation (3) accordingly as well. Their category $\mathcal{C}_{\text{flow}}$ has objects of type $\text{flow} :: [[t+1] - [t]]$. The minus $-$ in between types is a type operator or functor. ■

3.3 Natural Transformations as Evolution

Natural transformations model temporal evolution while preserving categorical structure.

Definition 3.3 (Natural Transformation as Evolution). A natural transformation $\eta : F \Rightarrow G$ assigns to each object A a morphism $\eta_A : F(A) \rightarrow G(A)$ such that for every morphism $f : A \rightarrow B$, $\eta_B \circ F(f) = G(f) \circ \eta_A$. The typed natural transformation has the following components.

$$\begin{aligned}
\eta_{\text{time}} &: F_t \Rightarrow F_{t+1} \\
\eta_{\text{LabBank}} &: F_t(\mathbb{R}_{\geq 0}) \rightarrow F_{t+1}(\mathbb{R}_{\geq 0}) \\
\eta_{\text{ResBank}} &: F_t(\mathbb{R}_{\geq 0}) \rightarrow F_{t+1}(\mathbb{R}_{\geq 0}) \\
\eta_{\text{ComBank}} &: F_t(\mathbb{R}_{\geq 0}) \rightarrow F_{t+1}(\mathbb{R}_{\geq 0})
\end{aligned}$$

The temporal evolution natural transformation η_{time} preserves account relationships.

$$\begin{aligned}
\eta_{\text{time}}(\text{Lab}^{\text{Bank}}) &: 0.0 \rightarrow 0.0 \rightarrow 52.0 \\
\eta_{\text{time}}(\text{Res}^{\text{Bank}}) &: 0.0 \rightarrow 208.0 \rightarrow 273.19 \\
\eta_{\text{time}}(\text{Com}^{\text{Bank}}) &: 0.0 \rightarrow 52.0 \rightarrow 190.50
\end{aligned}$$

Economics: Period Evolution Through Natural Transformations: The evolution in definition 3.3 is visualised in time in the string diagram 3.1. In the commutative diagram 3.2 we see the natural transformations at work in time with the pullback denoted by the corners $\lrcorner$. The consistency condition in Figure 3.3 is more informative than the zeros of equations (4)-(9). ■

The natural transformation pullback captures the fundamental economic principle of temporal consistency across accounting periods. The typed morphism $F_{\text{evol}} : \mathcal{C}_t \rightarrow \mathcal{C}_{t+1}$ ensures that account balances evolve coherently through natural transformations that preserve the categorical structure of the economy.

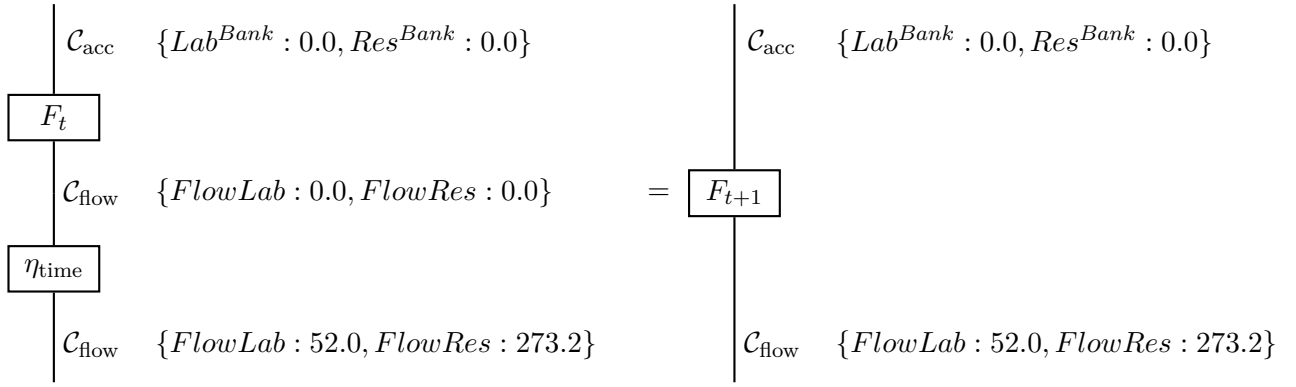


Figure 3.1: String Diagram: Natural Transformation $\eta_{\text{time}} : F_t \Rightarrow F_{t+1}$ showing functor composition equivalence for temporal evolution

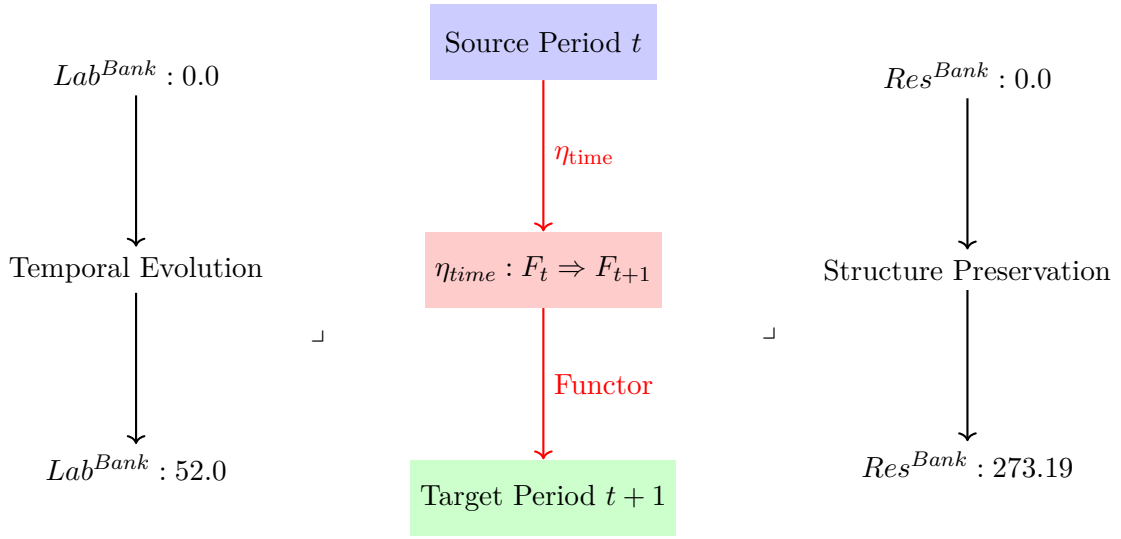


Figure 3.2: Commutative Diagram: Natural Transformation Pullback Flow

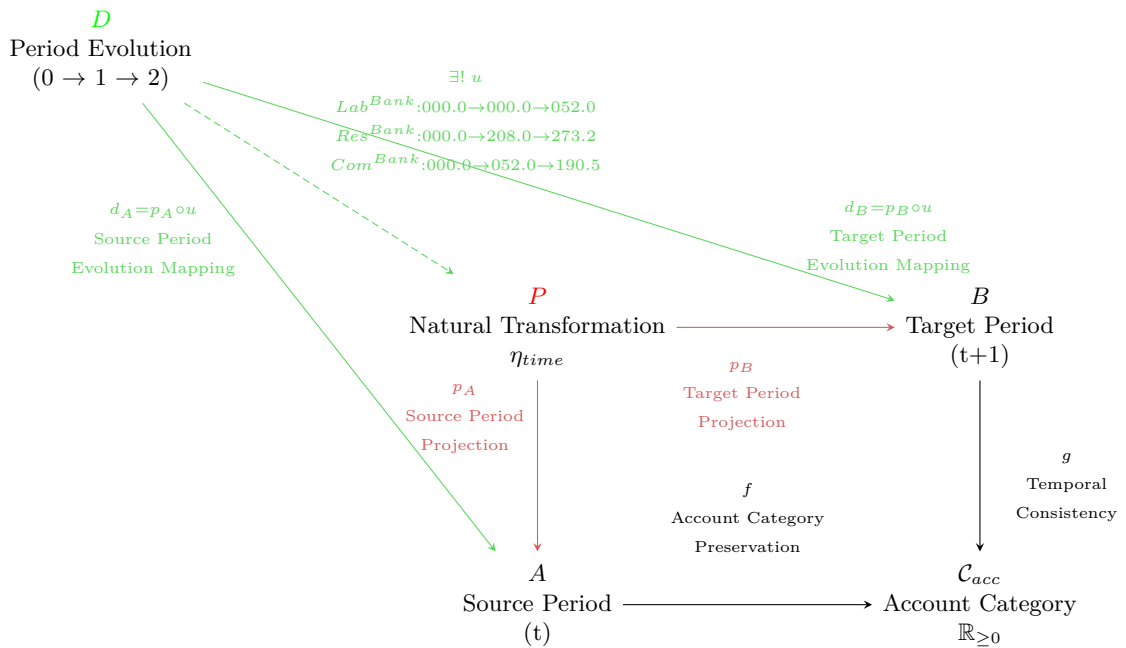


Figure 3.3: Natural Transformation Pullback: Temporal evolution validation across periods 0, 1, 2. The pullback ensures that account transformations preserve categorical structure while evolving through time.

Economically, this means that the transition from period 0 through period 1 to period 2 maintains accounting identities allowing for consistent booking and investment. The pullback constraint validates that each period's state is reachable from the previous period through valid economic transactions. For instance, Lab^{Bank} evolving from $0.0 \rightarrow 0.0 \rightarrow 52.0$ reflects labor owner's accumulation of bank deposits through wage payments, while Res^{Bank} growth from $0.0 \rightarrow 208.0 \rightarrow 273.19$ shows resource owner's increasing wealth through sales to the company.

The commutative diagram's 3.2 three-layer structure shows the categorical nature of time: the purple layer represents the source period's economic state, the red layer embodies the transformation mechanism of the natural transformation, and the green layer shows the target period's evolved state. The pullback corners $\lrcorner$ note that this evolution respects categorical limits. Economically this means no units of accounts are created or destroyed arbitrarily, and all flows among the double-entry systems are consistent across temporal boundaries which is what macroeconomic consistency is about.

4 Evolution

4.1 Eight Macroeconomic Bookings

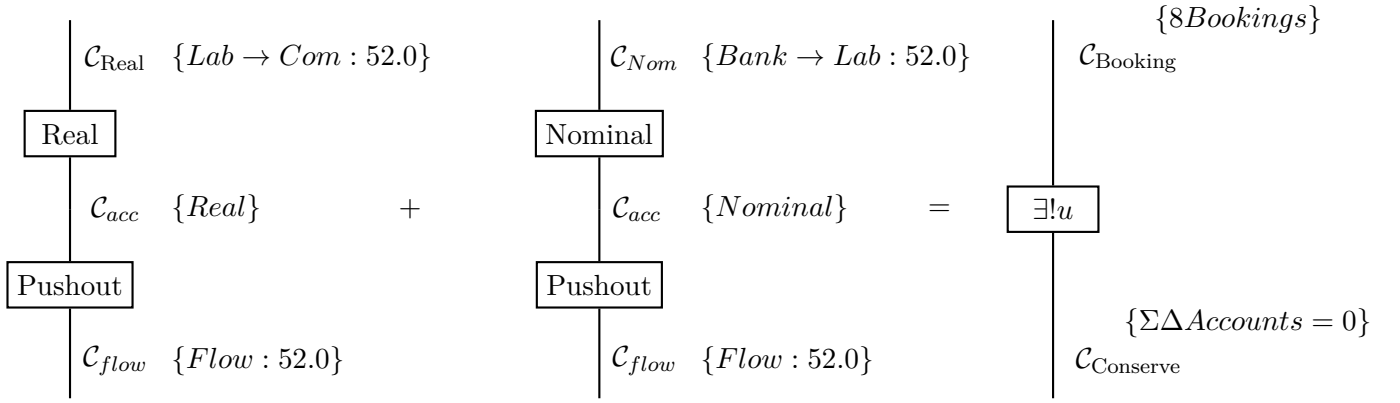


Figure 4.1: String Diagram: 8 Bookings Pushout $Real + Nominal \rightarrow Conservation$ showing universal aggregation property

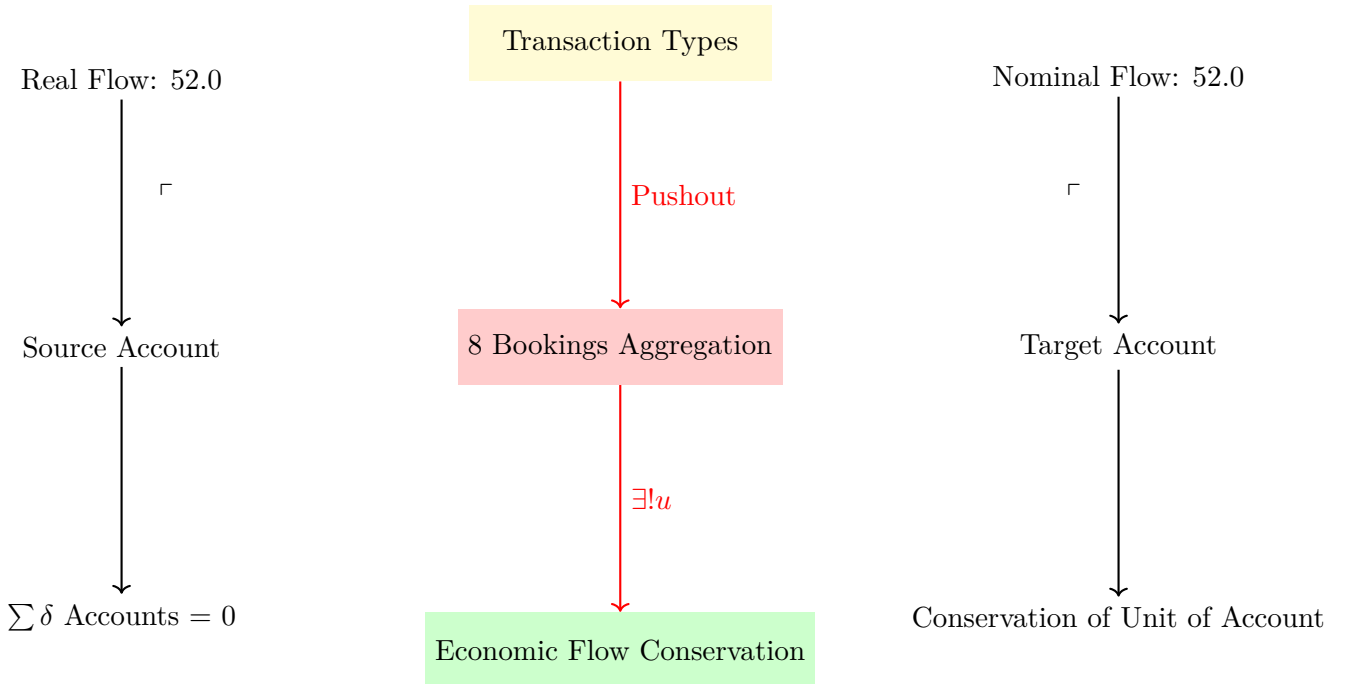


Figure 4.2: Commutative Diagram: 8 Bookings Pushout Flow

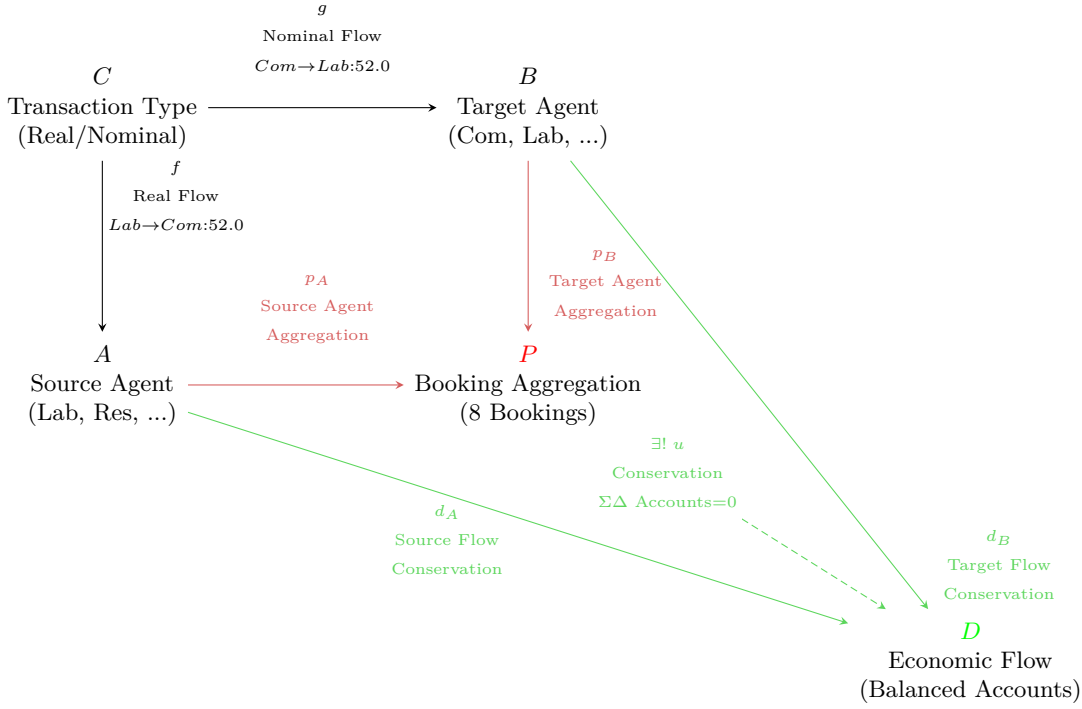


Figure 4.3: 8 Bookings Pushout: Flow aggregation of real flows and nominal flows into balanced economic flows. The pushout ensures conservation of accounts across all transactions.

Economics of Transaction Aggregation and Flow Conservation: The 8 bookings pushout embodies the fundamental principle of double-entry accounting lifted to categorical level. Each booking represents a simultaneous exchange where real flows and nominal flows move in different directions, creating the pushout structure that aggregates individual transactions into the economy’s global flow pattern. The fundamental economic flows in equation (3) become morphisms in categorically typed constructions.

Economically, this captures how elementary flows compose into complex economic interactions. The pushout universal property ensures that the aggregation preserves the conservation laws: when labor sells services to the company (Booking 1), real labor flows from Labor to Company while nominal euros flow from Company to Labor, mediated by the Bank. The commutative diagram 4.2 reveals that the Real Flow (52.0) and Nominal Flow (52.0) are equal and opposite, maintaining the zero-sum property of monetary consistency.

The categorical structure shows that Source Agents and Target Agents are connected through Transaction Types, which then pushout to Economic Flow Conservation. This means that all individual bilateral exchanges aggregate coherently into a global pattern that respects both microeconomic transaction constraints and macroeconomic conservation laws. The pushout corners in the commutative diagram mark the points where individual transaction morphisms compose into the universal aggregate morphism, ensuring that the sum of all real flows equals the sum of all nominal flows with opposite signs in Figure 4.3. Figure 4.4 shows the natural transformation sequence from abstract validation to concrete booking instances (Booking 1, Booking 3, Booking 5) with the pullback’s universal property ensuring transaction validity.

Economics of Transaction Validation and Type Safety: The booking validation pullback ensures economic transactions respect fundamental constraint structures before execution. The typed morphism $\mathcal{F}_{\text{Validation}} : \text{SourceAccounts} \times \text{TargetAccounts} \rightarrow \{0, 1\}$ acts as a categorical gatekeeper, verifying that proposed transactions satisfy accounting rules and economic logic.

Economically, this represents the critical validation layer that prevents impossible transactions: accounts cannot go negative (unless explicitly allowing credit), funds cannot be created ex nihilo, and transaction types must match account capabilities. The pullback structure validates that Booking 1 ($0.0 \rightarrow 52.0$) and Booking 3 ($208.0 \rightarrow 231.59$) represent feasible state transitions from their source accounts to target accounts.

The three-layer architecture of the diagram 4.6 shows validation as a categorical limit: the yellow layer represents proposed transactions, the blue layer embodies the validation mechanism (typed morphism validation), and the green layer confirms the transaction as either valid (1) or invalid (0). The pullback corners ensure that validation respects both source account constraints and target account capacities. This

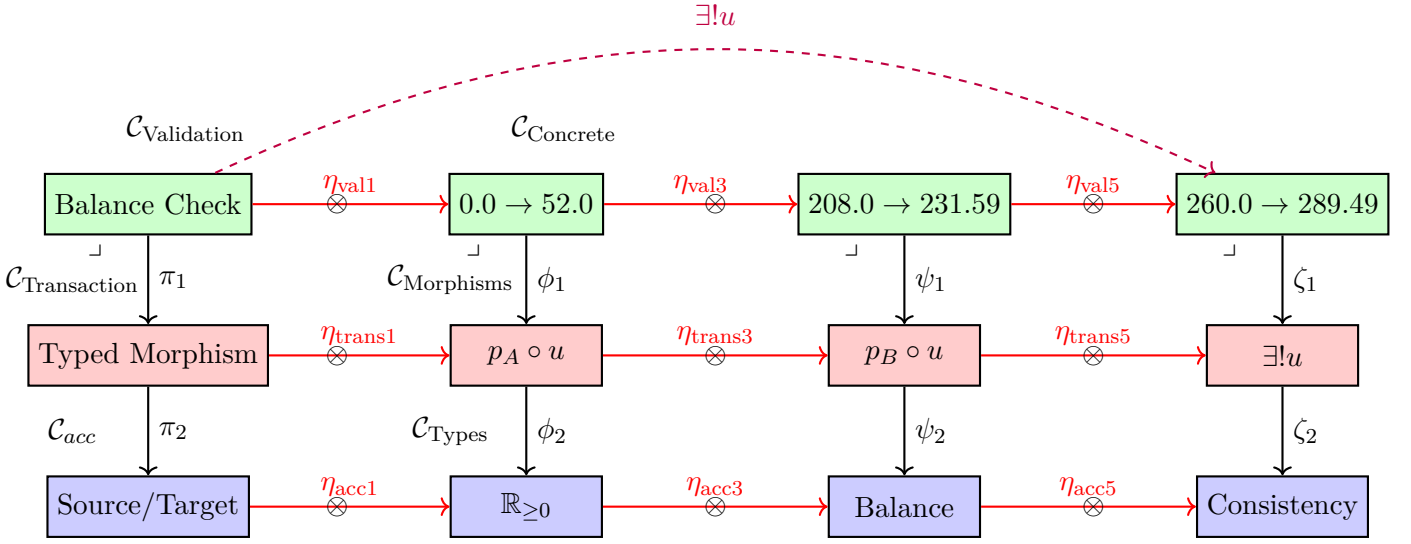


Figure 4.4: Commutative Diagram: Booking Validation Pullback Structure

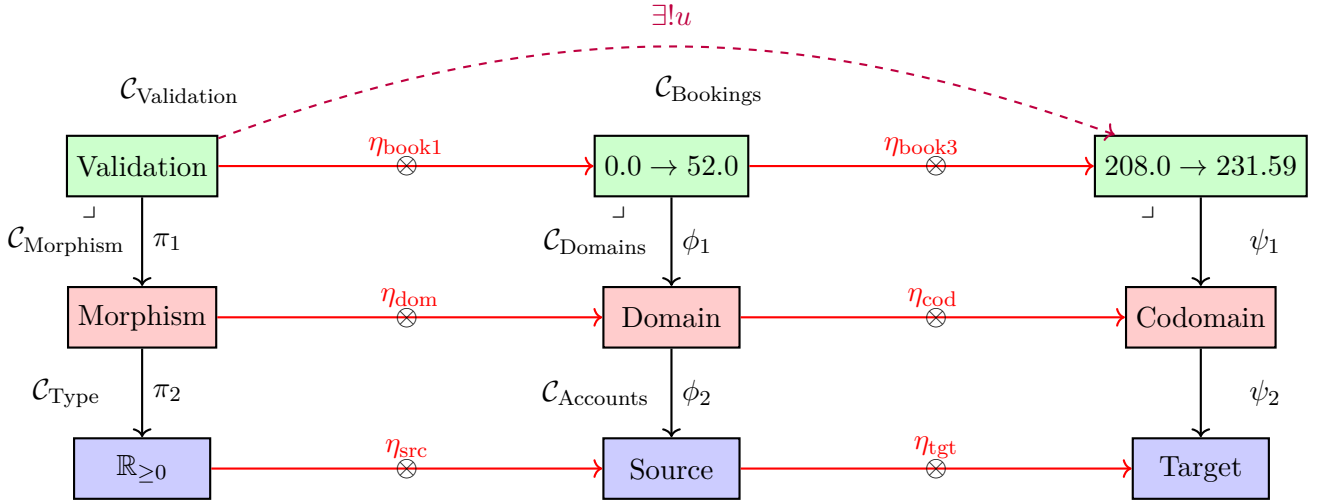


Figure 4.5: Commutative Diagram: Booking Validation Flow

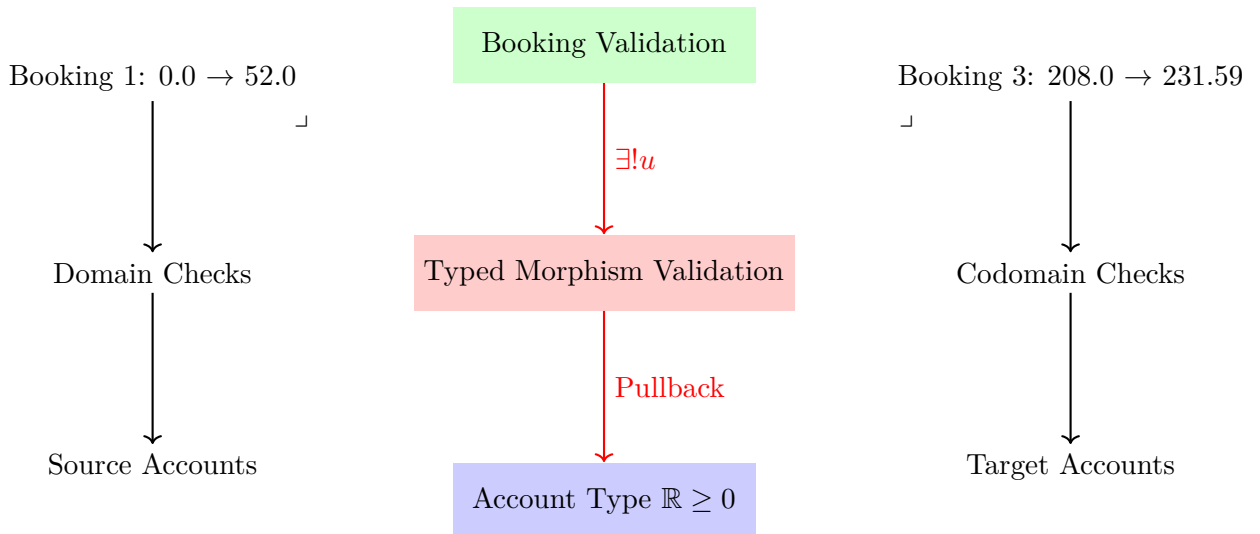


Figure 4.6: Commutative Diagram: Booking Validation Pullback Flow

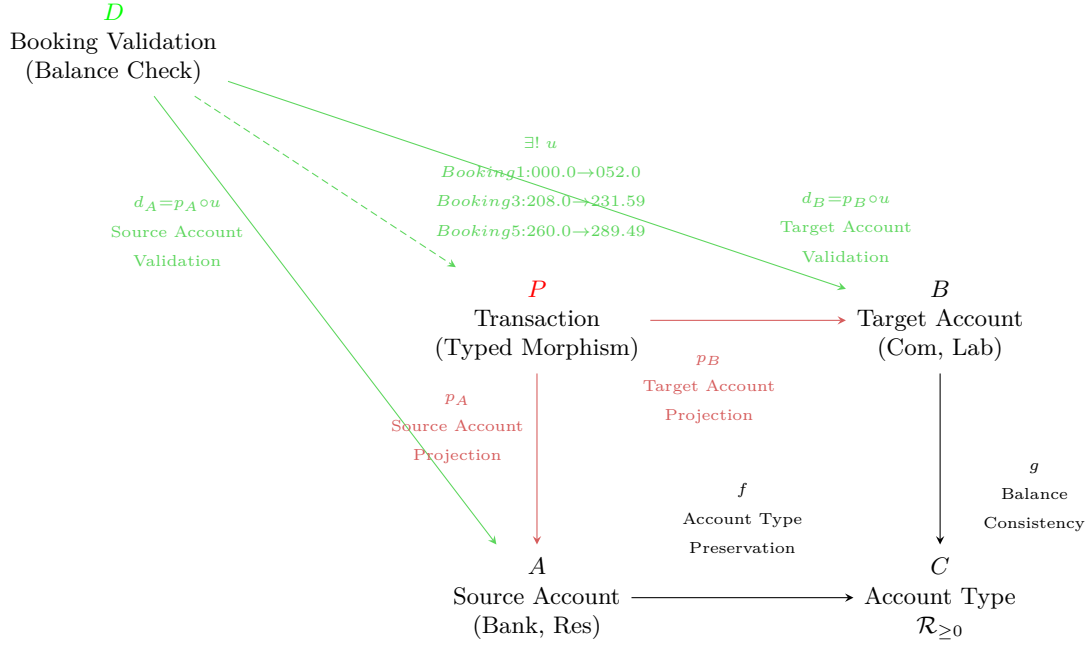


Figure 4.7: Booking Validation Pullback: Each of the 8 bookings is validated as a typed morphism with domain and codomain checks. The pullback ensures transaction validity before execution.

categorical approach to transaction validation provides mathematical foundations for economic transaction processing systems, ensuring that only economically feasible operations propagate through the system. Figure (4.5) shows the natural transformation between abstract validation structure and concrete booking instances, with pullback universal property ensuring transaction validity.

4.1.1 Categorically Typed Recursive Program

Formal Operations on Economic Categories: We define now the creation of the accounting system itself, mathematically and as a program. By that we can obey to the Lucas Critique to endogenize the observing econometrician and theory constructor. This equips the agent in the theory with the same abilities as the observing econometrician to learn from data. The econometrician can provide, as we do in this paper as observing economists, implementations and theories of a monetary macroeconomic accounting system. This can be done categorically by taking the operations to be the endogenous creation and change of the categorical state of the monetary accounting system. As an econometric generalisation of the pullback of the investment learning of the banking sector can be some portfolio selection, other sectoral financial instruments, market making and clearing structures of the bank sector or the central bank. The endogenous creation of the categorical state of the monetary accounting system is done by the helper functions.

Mathematical formulas for the construction of an economic category is a first step to endogenize the econometric observer, the economist and econometrician, maintaining an economic theory about data sharing or not in the memory of the economy as in a Bayesian game of incomplete information. Helper functions are the ability of the observer and observed likewise to construct perspectives on or transformations of the system functorially. This can be used with universal initial and final objects functionality for creating, deleting and copying structures. Functors and categorical morphisms allow us to distinguish between structures and operations, observers' and observeds' views in hierarchical orders. We can use them for any functorial change of the domains or codomains that describe the state and institutional structures of the economy. We can extend MoMaT by categorifying structural macroeconomic theories like in Winschel and Krätzig [2010] by open games of Ghani et al. [2018] or micro econometrics as in Heiss and Winschel [2008] by adjunctions. Forward looking agents, structural in the sense of rational expectations, can be formulated in categories by open games Ghani et al. [2018]. The two flows in the adjunctions could then be simulations as in this paper from parameters to data and the backpropagation could be the other direction of adjunctions where the data is used to update the parameters, possibly also taking advantage of open games to also represent backpropagation as shown in Fong et al. [2019]. A structural model or theory is in economics an explicit decision model as the so called micro foundation of macroeconomics.

In Ehresmann and Vanbreemersch [2007] on Memory Evolutive Systems a categorical model is formu-

lated for emergent effects, which can be used in economics in institutional design. Elinore Ostrom [2005] polycentrism, once categorified, can be included as open games for decentral, parallel endogenously optimal hierarchical control structures. Macro and sectoral decisions in universals of money issuing central banks and government spending from debts and taxes can be integrated. Interbanking market making in theory and models as programs in distributed computing environments are specifyable as universal Shapley values of liquidity pooling. The categorical apparatus may also unify management science and economics by categorifying ERP systems like the object oriented perspective in Scheer [2024] based on earlier work in the nineties on ARIS as a blue print for customizing SAPs R/3. However, SAP's accounting was untyped and two dimensional for internal and external accounting only, whereas we have none of these constraints.

With the helper functions of the categorically typed functions and their mathematical types we create, update, and relate economic objects and morphisms in the categories of accounts and flows. To run an accounting system for MoMaT, we need to create the economic categorical state with 5 agents and accounts. This extension can formally define agents as economy creators of the five agent economy $MoMaT^5$ and extending for a central bank $MoMaT^{cb}$ or government $MoMaT^{gov}$. Hence, learning institutional designs and modelling structural breaks in a natural evolution is a generalised economic functionality of the compositionality of category theory.

The functionality of the operations are basic manipulations of the economic categorical state. Vertex Operations: Creating, retrieving, and updating economic accounts. Edge Operations: Creating and managing economic flows between accounts. Functor Operations: Establishing and applying transformations between categories. Natural Transformation Operations: Managing consistent changes across transformations.

Definition 4.1 (Helper Functions). The creation of the economic state is done by the helper functions. First we create three categories $\mathcal{C}_{accounts}$, flows $\mathcal{C}_{flows}$ and parameters $\mathcal{C}_{parameters}$. Then we create the accounts of the agents labor, resource, company, capitalist and bank. We add the parameter vertices to the category of parameters. We create the functor from nominal to real mapping Lab^{Bank} to Lab^{Lab} , Res^{Bank} to Res^{Res} , Com^{Bank} to Com^{Good} . We create the functor from $Accounts$ to $Flows$. We initialize the memory vectors for the wage and repayments.

The return of nothing () is because the purpose of update functions (names end in ! by style convention in Julia) is to change the first argument of the function as a side effect and not as usual to return by purpose some calculation without changing the arguments. The function `add_vertex!` is used to add a vertex to the category, it returns the ID of the new vertex. The return of `add_edge!` beside the side effect to change the category also returns the added edge. The `update_vertex!` function returns nothing () beside the sideeffect to update the amount of an account. The `add_mapping!` function returns nothing () and is as well called for the side effect to add a mapping to the functor $\mathcal{F}$. The two functions on the natural transformations are defined but not used in the categorically typed program. They are implemented as the

categorical program.

$\text{Ob}(\mathcal{C})$	$:: \text{Ob}(\mathcal{C}_{par}), \text{Ob}(\mathcal{C}_{flow}), \text{Ob}(\mathcal{C}_{acc})$	Parameters, Flows, Accounts
$\text{Hom}(\mathcal{C})$	$:: \text{Morphisms of}$	$\mathcal{C}_{par}, \mathcal{C}_{flow}, \mathcal{C}_{acc}$
$\text{Op}(\mathcal{C})$		
get_vertex	$:: \mathcal{C} \times \text{String} \rightarrow \text{Ob}(\mathcal{C})$	Retrieves vertex by name
add_vertex!	$:: \mathcal{C}_{acc} \times \text{String} \times \text{String} \times \mathbb{R} \rightarrow \mathbb{N}$	Add account in $\mathcal{C}'$ named::String typed::String with amount:: $\mathbb{R}$ return ID:: $\mathbb{N}$ of new vertex
add_edge!	$:: \mathcal{C} \times \mathbb{N} \times \mathbb{N} \times \mathbb{R} \rightarrow \text{edge} :: \text{Edge}$	Adds edge with source:: $\mathbb{N}$ target:: $\mathbb{N}$ weight:: $\mathbb{R}$ return new edge::Edge
update_vertex!	$:: \mathcal{C}_{acc} \times \text{String} \times \mathbb{R} \rightarrow ()$	Updates an account::String amount:: $\mathbb{R}$ weight:: $\mathbb{R}$ return <i>nothing</i> ()
$\text{Op}(\mathcal{F})$		
add_mapping!	$:: \mathcal{F} \times \mathbb{N} \times \mathbb{N} \rightarrow ()$	Add map of morphisms to functor $\mathcal{F}'$ source morphism:: $\mathbb{N}$ in $\mathcal{C}$ target morphism:: $\mathbb{N}$ in $\mathcal{F}(\mathcal{C})$ return <i>nothing</i> ()
apply_functor	$:: \mathcal{F} \times \mathbb{N} \rightarrow \mathbb{N}$	Apply functor $\mathcal{F}$ to vertex:: $\mathbb{N}$ returns vertex ID:: $\mathbb{N}$ of vertex_map of $\mathcal{F}$
$\text{Op}(\eta)$		
add_component!	$:: \eta \times \mathbb{N} \times \text{Edge} \rightarrow \eta'$	Adds to η edge::Edge with ID:: $\mathbb{N}$
$\text{apply_transformation}$	$:: \eta \times \mathbb{N} \rightarrow \eta'$	Applies η to vertex:: $\mathcal{N}$

These functions satisfy the following mathematical properties:

- Identity preservation: $\text{get_vertex}(\text{add_vertex!}(c, n, t, a)) = n$, account name
- Consistency preservation: All operations maintain the category structure
- Type safety: Operations respect the economic types of accounts and flows

Each function corresponds to a categorical operation:

- Vertex operations correspond to object operations in the category
- Edge operations correspond to morphism operations in the category
- Update operations correspond to endomorphism operations in the category

Definition 4.2 (Categorically Typed State). The categorical state S^{cat} is a categorical structure composed of three categories, two functors, two natural transformations, and historical vectors.

$$S^{cat} = (\mathcal{C}_{acc}, \mathcal{C}_{flows}, \mathcal{C}_{pars}, \mathcal{F}_{price}, \mathcal{F}_{flow}, \eta_{price}, \eta_{flow}, \mathcal{H}_{wage}, \mathcal{H}_{repay}) \quad (25)$$

The categorical state is implemented in the categorically typed recursive state program. The economic simulation is programmed by annotating the calculated numbers with the categorical structures by the helper functions as categorical type (`struct`) generators. In the categorical program the natural transformations are implemented as the simulation of the evolution and the state category becomes the one of the endofunctor in section 4.6. The categorical state S^{cat} is then just one view on the category of the economy $\mathcal{C}_{Economy}$ as the categorical state as a view on the categorical description of the evolution. The three categories

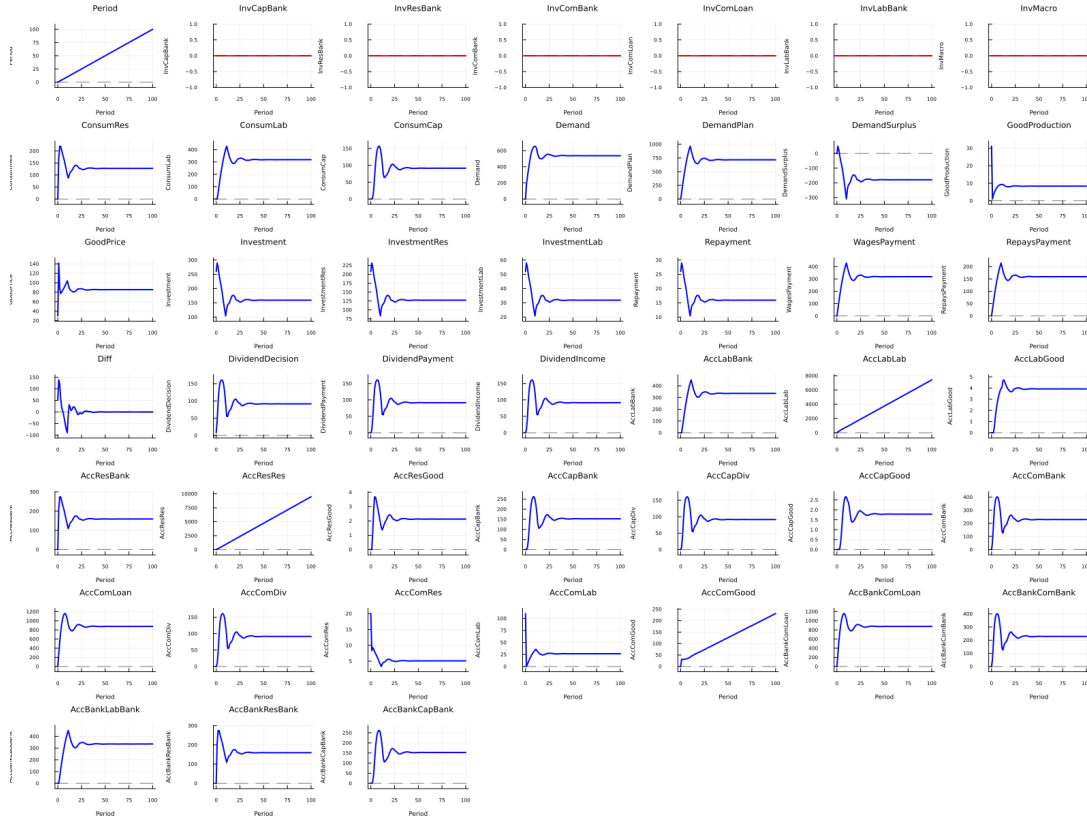


Figure 4.8: Time series plots of variables in the categorically typed recursive simulation

$\mathcal{C}_{acc}$, $\mathcal{C}_{flows}$, $\mathcal{C}_{pars}$ organize accounts, flows, and parameters. The functors $\mathcal{F}_{price} : \mathcal{C}_{acc} \rightarrow \mathcal{C}_{acc}$ and $\mathcal{F}_{flow} : \mathcal{C}_{acc} \rightarrow \mathcal{C}_{flows}$ connect nominal and real values as well as accounts with flows. The natural transformations $\eta_{price} : \mathcal{F}_{price} \Rightarrow \mathcal{F}_{price}$ and $\eta_{flow} : \mathcal{F}_{flow} \Rightarrow \mathcal{F}_{flow}$ manage temporal changes.

These categorical constructions implement the entire economic simulation as categorical computations. Pullbacks validate transaction constraints through universal properties, while pushouts compute economic transitions through flow aggregation. The categorical machinery becomes from a typing system to the actual computational engine. Just like a compiler that derives an implementation for a specification in categorical types.

In the categorical program we implement the natural transformations as actual code and not annotations of recursive calculation of the numbers with wrapped around helper functions that generate the categories as Julia struct. The function `add_mapping!` and `apply_transformation` are implemented by the categorical program. In Figure 4.8 we see the time series plots of the categorically typed simulation to be

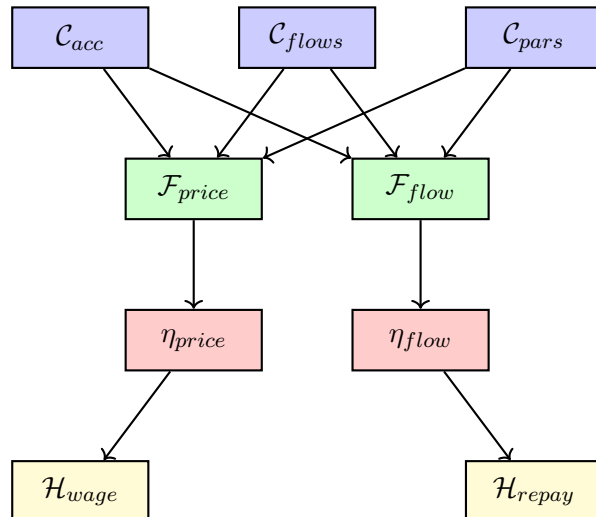


Figure 4.9: String Diagram: Categorical State Structure

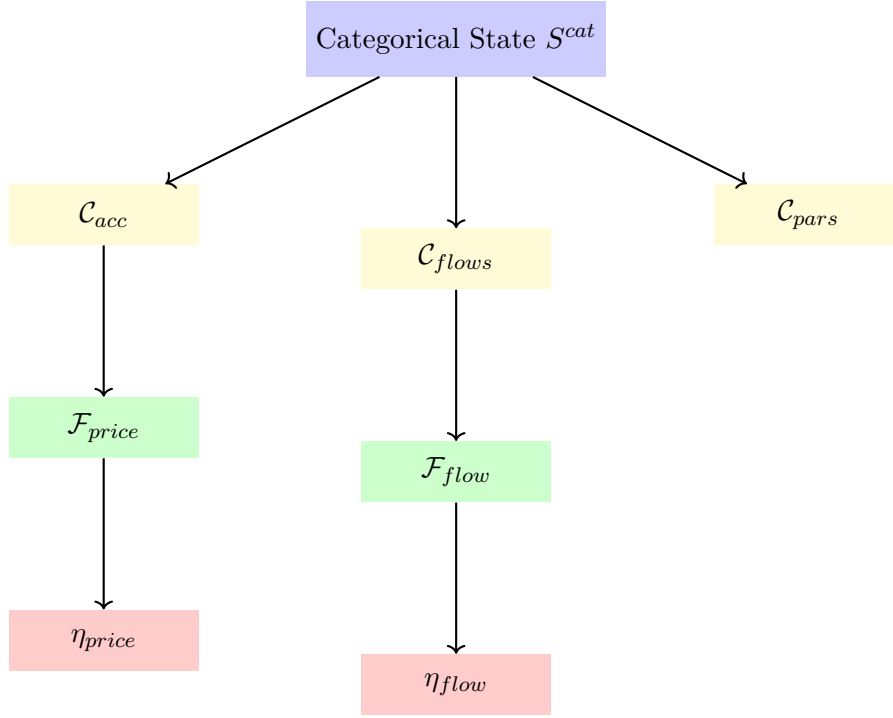


Figure 4.10: Categorical State Composition Structure

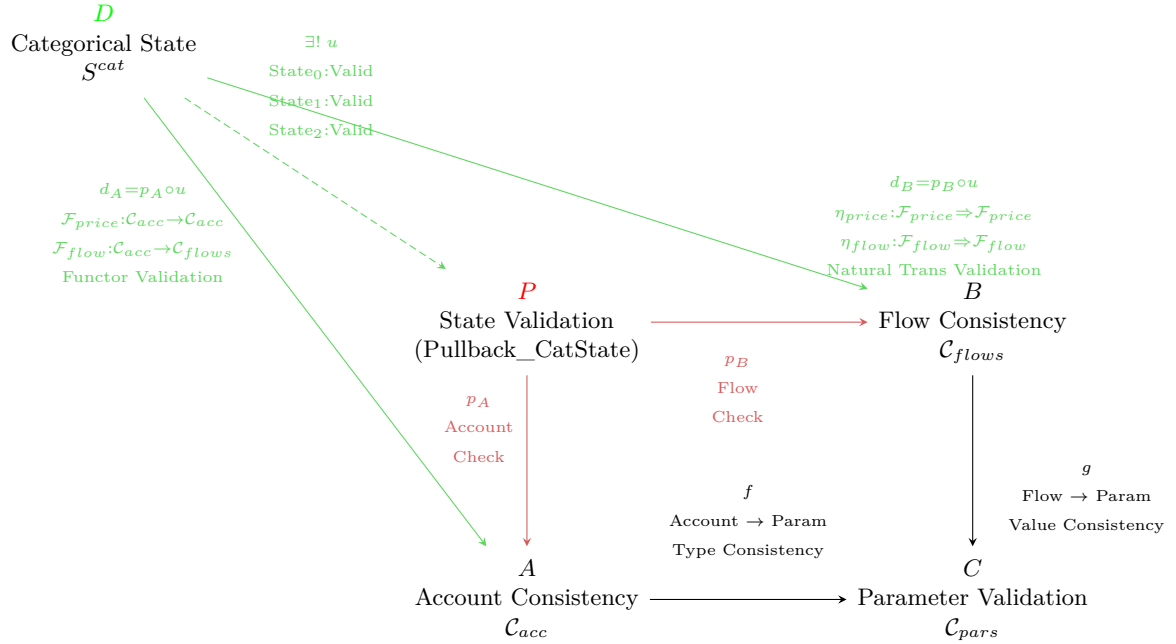


Figure 4.11: Categorical State Validation Pullback: The categorical state pullback ensures that all components ($\mathcal{C}_{acc}, \mathcal{C}_{flows}, \mathcal{C}_{pars}, \mathcal{F}_{price}, \mathcal{F}_{flow}, \eta_{price}, \eta_{flow}$) satisfy universal consistency properties through functor and natural transformation validation.

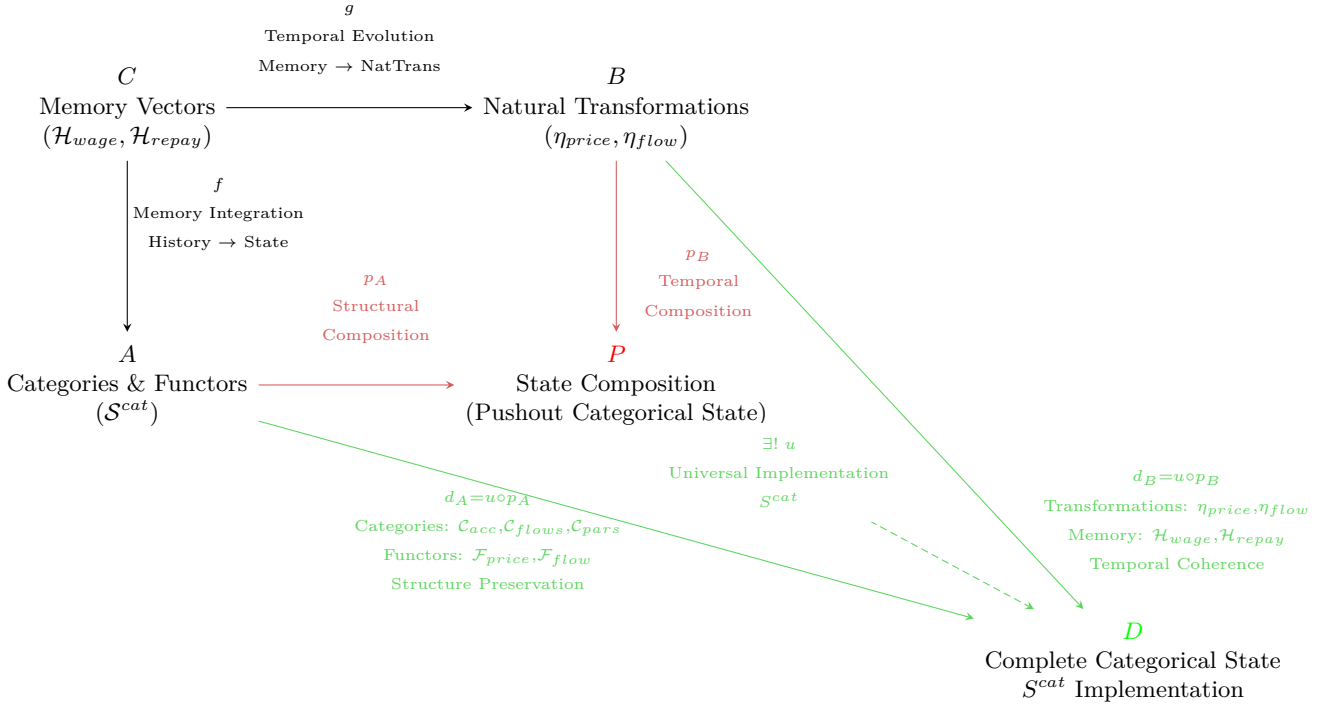


Figure 4.12: Categorical State Composition Pushout: The categorical state pushout implements the universal composition of all categorical components into the complete economic simulation state S^{cat} , ensuring structural preservation and temporal coherence through the unique universal morphism u .

the same as in Figure (2.2) of the recursive program.

4.1.2 Categorical Program

The categorical simulation extends beyond just typing the recursive program as annotations by categorical types. We use categorical structures as the actual computational machinery. While the categorically typed recursive program uses categorical types but still computes through iterations, the categorical simulation performs all computations through natural transformations between functors.

This opens the door to coalgebraic treatments or formally distinguish between theory and model through homomorphisms as in Tarski's logic and we can use Lawvere's formulation as categories for theories, functors for models and natural transformations for homomorphisms in the econometrics of the Lucas Critique. For an epistemological stance of the concepts *theory* and *model* in economics see Klein [1998]. A Lawvere inspired categorical approach might resolve issues of the so called *model communism* in rational expectations as locally and globally consistent theories and models of local or global agents. Do the agents in the theory share the knowledge about the structures of the economy with the observing econometricians? The observer versus observed duality of equipping agents in economic theories with estimation capabilities for estimating parameters from data might involve a gradual type of agents. The more the agent observes of the full mathematical state of the system the more he is observer, economist or econometrician. Econometric models are to be formulated compositionally showing how to use the informations of the subsystems in the information aggregation for the econometric informations about the whole macroeconomic system. The agent's view is a functor and natural transformations tell the story how the observer and the observed interact. However, a natural transformation might be not enough to represent the dynamically changing situation. The situation is more complex structured as the quantum effect of social systems where the observed system also changes the observer, he learns, and again we need endogenous econometricians. The observers point of view changes are concept formation or colimit constructions, that manifest themselves in the memory of the agents, like now in the memory of the whole macroeconomy. Theory becomes an infinitely composable structure of structural econometric theories and of the real world implementation of finite models of data structures, computing machineries and information processors. Theory is for defining models and improving them by learning. However, it is too early to speculate how the categorical machinery and tool set can be used in old standing issues of the epistemology of economics. In this paper, we want to

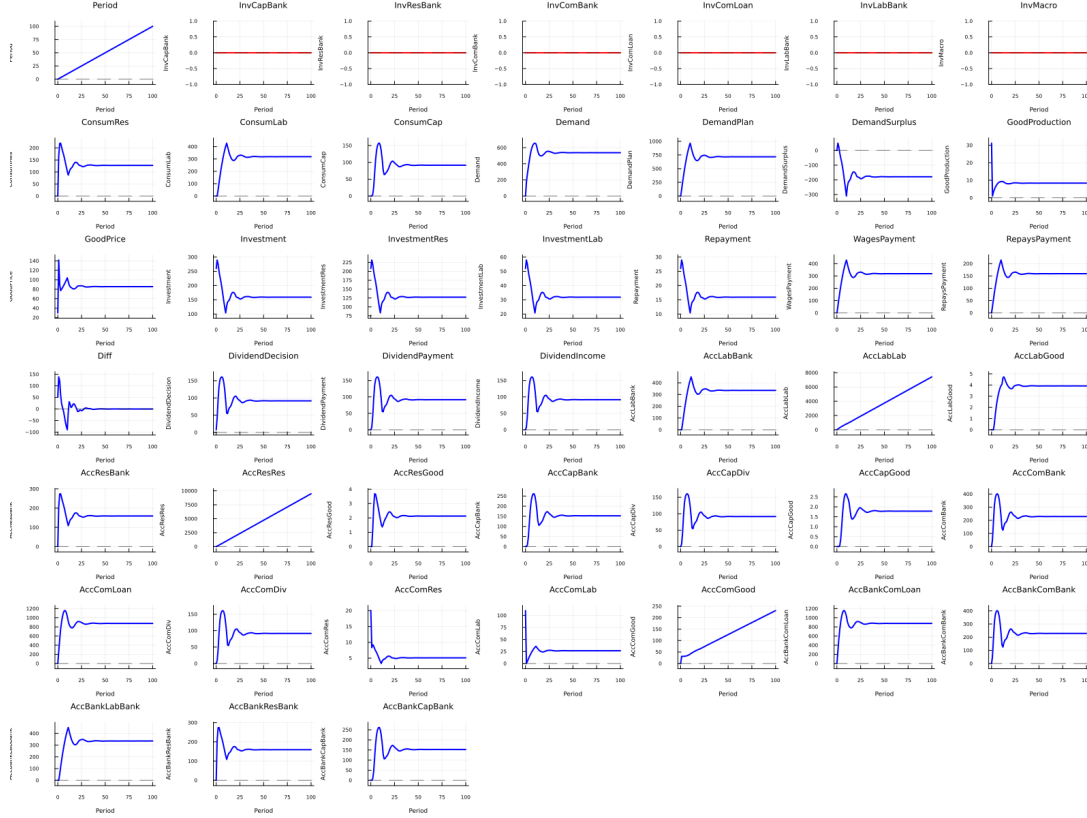


Figure 4.13: Time series plots of variables in the categorical simulation

categorify the seemingly simple recursive macroeconomic sectoral consistent model of MoMaT with proper types and implementation. The rest is for future work and discussion and developments.

In the categorical set up each booking is represented as a natural transformation $\eta : \mathcal{F} \Rightarrow \mathcal{G}$ between functors mapping the account categories to updated states. The components of these natural transformations encode the actual monetary flows and account updates. This allows the entire simulation to run by categorical composition of natural transformations, with pullbacks validating constraints and pushouts computing the economic transitions. The categorical machinery becomes from being a typing system to the specification of the actual implementation of the computations, making the mathematical structure and the program for execution identical. Category theory can serve not just as a framework for theory but as an actual implementation methodology for economic simulations and econometric estimations. In Figure 4.13 we see the time series plots of the categorical simulation and that all figures, Figure (2.2), (4.8) and (4.13), all show the same results.

4.2 Consumption Decisions

4.2.1 Consumption

Definition 4.3 (Consumption). The consumption functor $\mathcal{F}_{cons} : \mathcal{C}_{acc} \rightarrow \mathcal{C}_{flow}$ transforms account balances to consumption flows:

$$\mathcal{F}_{cons}(Res^{Bank}) = Res^{Bank} \times \rho_r \quad (26)$$

$$\mathcal{F}_{cons}(Lab^{Bank}) = Lab^{Bank} \times \rho_l \quad (27)$$

$$\mathcal{F}_{cons}(Cap^{Bank}) = Cap^{Bank} \times \rho_c \quad (28)$$

$$F_{ConsumRes} : \mathbb{R}_{\geq 0} \times [0, 1] \rightarrow \mathbb{R}_{\geq 0} \quad F_{ConsumRes}(x, \rho_r) = x \times \rho_r \quad (29)$$

$$F_{ConsumLab} : \mathbb{R}_{\geq 0} \times [0, 1] \rightarrow \mathbb{R}_{\geq 0} \quad F_{ConsumLab}(x, \rho_l) = x \times \rho_l \quad (30)$$

$$F_{ConsumCap} : \mathbb{R}_{\geq 0} \times [0, 1] \rightarrow \mathbb{R}_{\geq 0} \quad F_{ConsumCap}(x, \rho_c) = x \times \rho_c \quad (31)$$

■

Evolution across **periods 0, 1, 2** with $\rho_r = 0.8$, $\rho_l = 0.95$, $\rho_c = 0.6$.

$$\begin{aligned}\text{ConsumRes}_0 &= 0.0 \times 0.8 = 0.0 \\ \text{ConsumRes}_1 &= 208.0 \times 0.8 = 166.4 \\ \text{ConsumRes}_2 &= 273.19 \times 0.8 = 218.55\end{aligned}$$

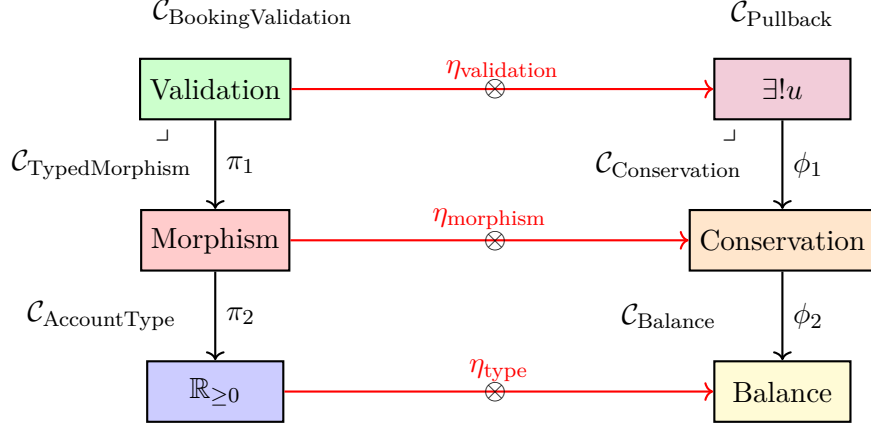


Figure 4.14: Commutative Diagram: Booking Validation Pullback Flow

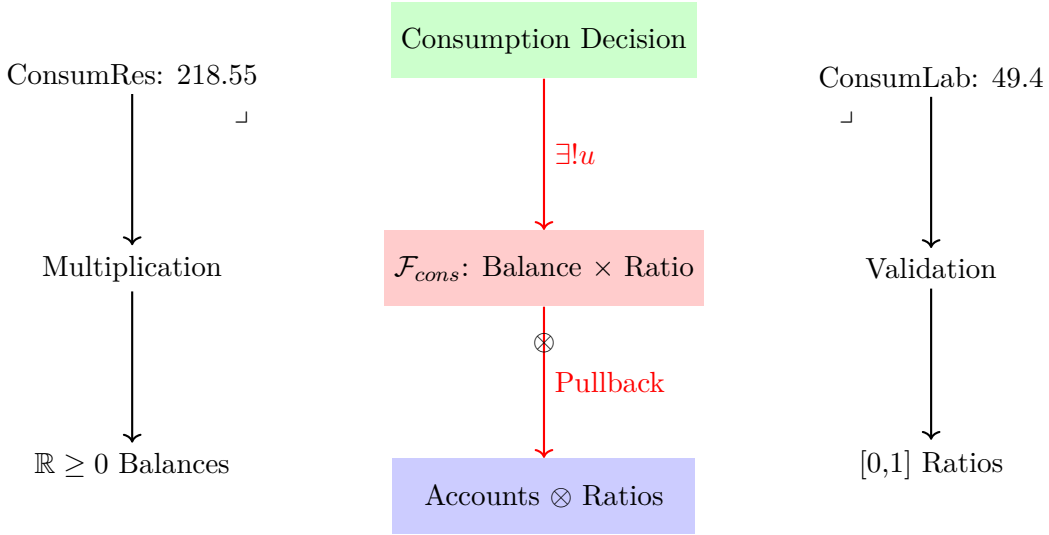


Figure 4.15: Commutative Diagram: Consumption Function Pullback Flow

Economics: Consumer Decision-Making and Consumption Choice: Figure (4.14) shows the natural transformation between booking validation layers and their pullback universal property, ensuring type-safe accounting operations. The consumption pullback captures the microeconomic foundation of consumer choice theory through categorical structures. The typed morphism $F_{\text{cons}} : \mathbb{R}_{\geq 0} \times [0, 1] \rightarrow \mathbb{R}_{\geq 0}$ transforms account balances and consumption propensities into actual consumption decisions, implicitly respecting budget constraints and preference structures.

Economically, this represents how consumers allocate their values and wealth across goods (if we were to model several) based on their consumption ratios. The pullback ensures that consumption decisions are consistent with available resources: ConsumRes (218.55) emerges from combining resource account balances with ρ_r (0.8), while ConsumLab (49.4) reflects labor's consumption with ρ_l (0.95). The categorical structure guarantees that total consumption respects budget constraints.

The diagram reveals the tensor product structure $\text{Accounts} \otimes \text{Ratios}$ at the base layer, showing how consumption decisions emerge from the categorical product of available wealth and consumption preferences. The pullback corners ensure that individual consumption choices aggregate consistently into overall demand patterns. This categorical approach to consumption theory provides a rigorous foundation for understanding

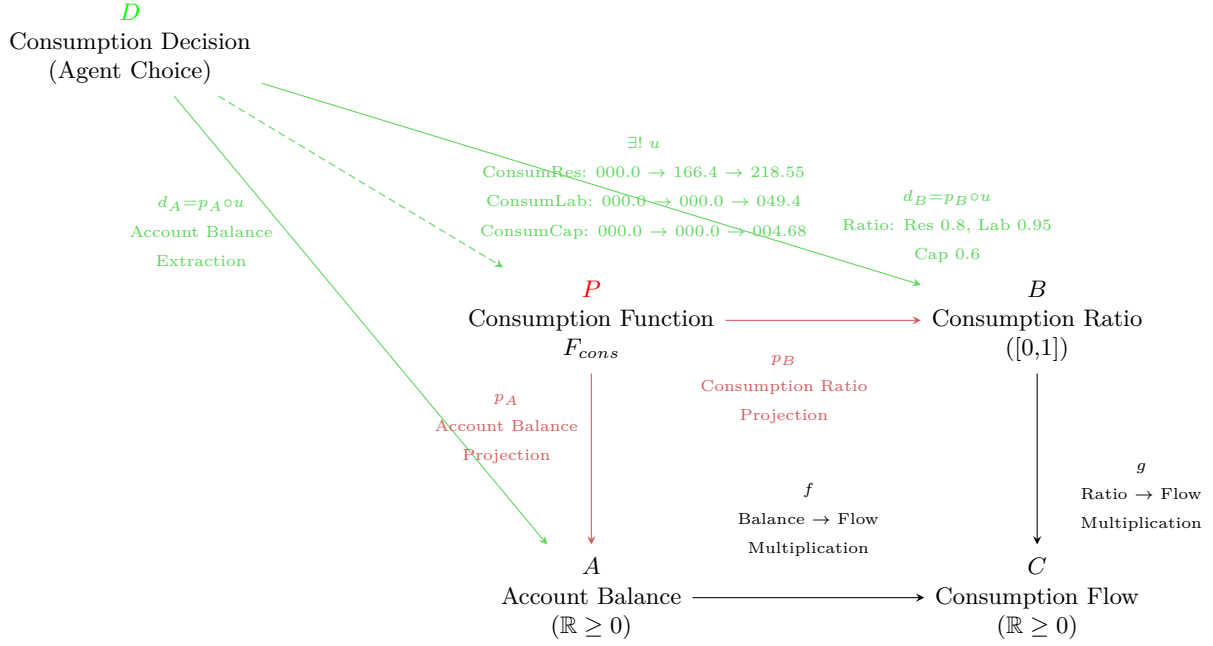


Figure 4.16: Consumption Function Pullback: The consumption decision pullback validates the multiplication of account balances with consumption ratios, ensuring proper economic behavior across agents.

how microeconomic consumer decisions compose into macroeconomic demand structures, bridging individual choice theory with aggregate economic outcomes.

4.2.2 Aggregate Demand

Definition 4.4 (Demand). The demand pushout $\mathcal{D} = \text{ConsumRes} \sqcup \text{ConsumLab} \sqcup \text{ConsumCap}$ computes total demand:

$$\text{Demand} = \text{ConsumRes} + \text{ConsumLab} + \text{ConsumCap}$$

$$\text{Pushout}_{\text{Demand}} : \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0} \quad \text{Pushout}_{\text{Demand}}(c_r, c_l, c_c) = c_r + c_l + c_c \quad (32)$$

■

Evolution across **periods 0, 1, 2**:

$$\begin{aligned} \text{Demand}_0 &= 0.0 + 0.0 + 0.0 = 0.0 \\ \text{Demand}_1 &= 166.4 + 0.0 + 0.0 = 166.4 \\ \text{Demand}_2 &= 218.55 + 49.4 + 4.68 = 272.63 \end{aligned}$$

Economics: Market Demand Formation and Aggregation Mechanisms Figure (4.17) shows the natural transformation sequence from individual consumption patterns (ConsumRes: 218.55, ConsumLab: 49.4) to market demand (272.63) through categorical pushout construction. The demand aggregation pushout embodies the fundamental principle of market formation through categorical colimits. Individual consumption decisions by resource owners (ConsumRes: 218.55) and labor (ConsumLab: 49.4) aggregate through the pushout universal property to create unified market demand (272.63), demonstrating how microeconomic choices compose into macroeconomic outcomes.

Economically, this captures the mechanism where individual consumption decisions, driven by personal utility maximisation, aggregate coherently into market signals that guide production decisions. The pushout structure ensures that the aggregation preserves economic information: the sum aggregation component maintains quantitative demand information, while the market signal component translates individual preferences into price relevant signals.

The three-layer structure of the diagram reveals the hierarchical nature of market formation: the yellow layer represents individual consumption choices, the red layer embodies the aggregation mechanism (demand aggregation), and the green layer shows the resulting market demand. The pushout corners mark where

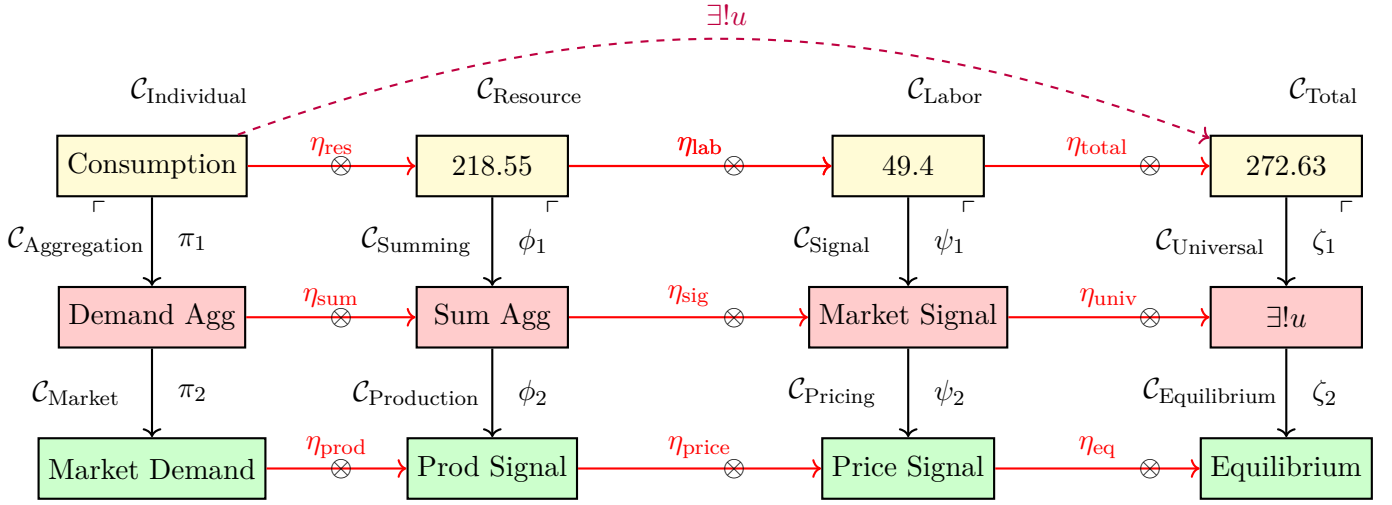


Figure 4.17: Commutative Diagram: Demand Aggregation Pushout Structure

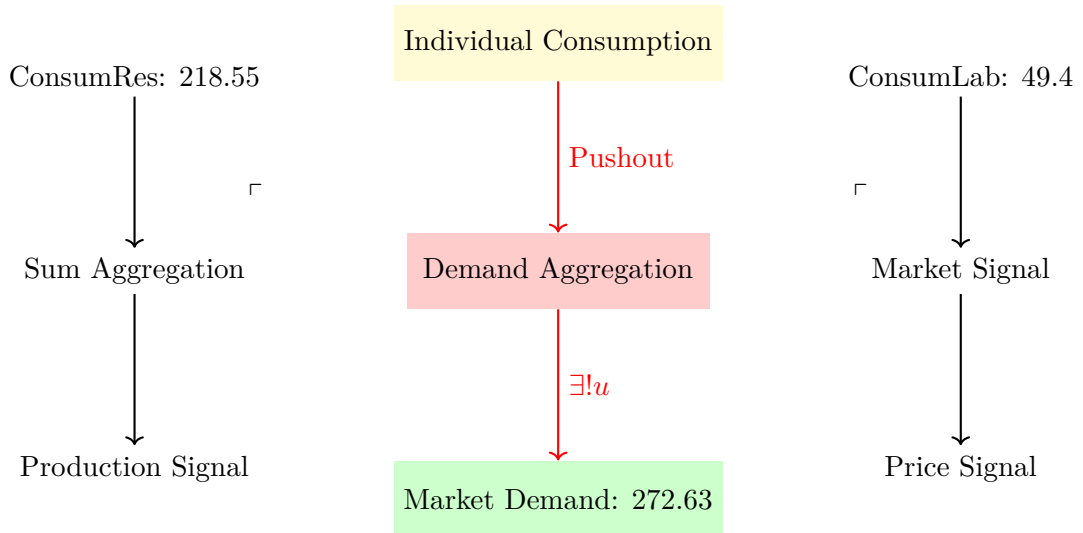


Figure 4.18: Commutative Diagram: Demand Aggregation Pushout Flow

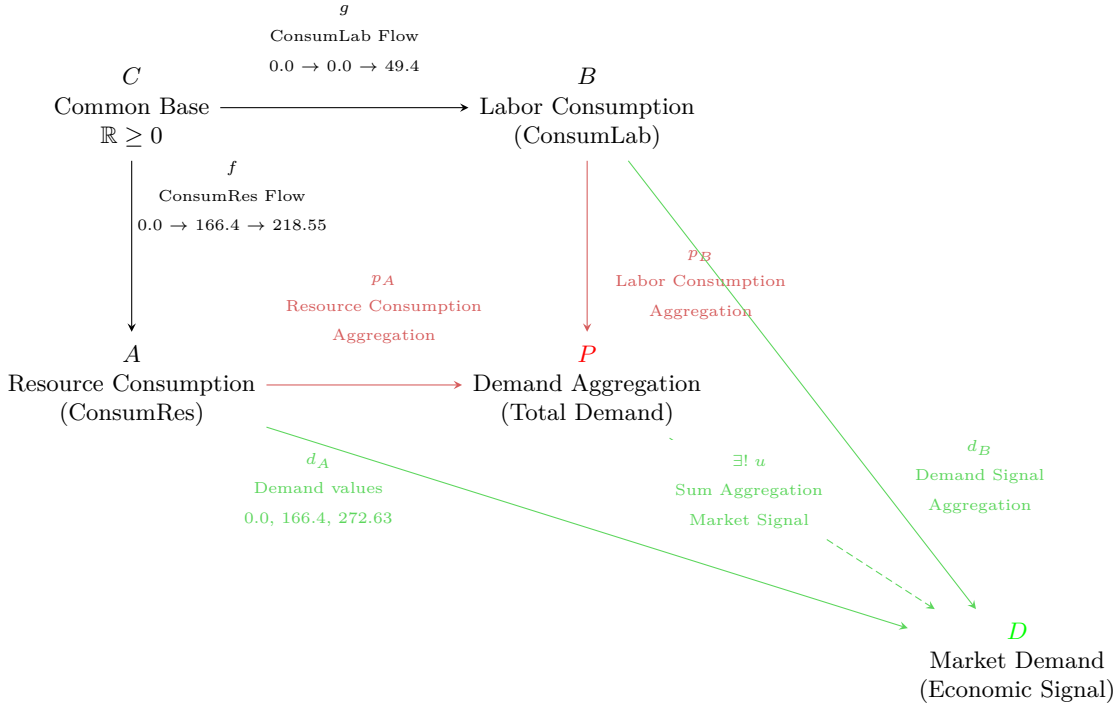


Figure 4.19: Demand Aggregation Pushout: The demand pushout aggregates individual consumption flows (ConsumeRes, ConsumLab, ConsumCap) into total market demand, providing the economic signal for production decisions.

individual consumption morphisms compose into the universal market demand morphism. This categorical approach to demand aggregation provides mathematical foundations for understanding how decentralized individual decisions create coherent market outcomes without central coordination.

4.3 Production Decisions and Price Formation

4.3.1 Cobb-Douglas Production

Definition 4.5 (Production). The production pushout aggregates labor and resource inputs according to a standard Cobb-Douglas function into the productive output of the economy:

$$\text{GoodProduction} = 1 + \alpha \times (\text{Com}^{\text{Lab}})^{\gamma} \times (\text{Com}^{\text{Res}})^{1-\gamma}$$

$$\text{Pushout}_{\text{Production}} : \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \times \mathbb{R}_{> 0} \times (0, 1) \rightarrow \mathbb{R}_{> 0} \quad \text{Pushout}_{\text{Production}}(L, R, \alpha, \gamma) = 1 + \alpha \times L^{\gamma} \times R^{1-\gamma} \quad (33)$$

where $\alpha = 0.42$ and $\gamma = 0.75$. ■

Evolution across **periods 0, 1, 2**:

$$\text{GoodProduction}_0 = 1 + 0.42 \times 110.0^{0.75} \times 20.0^{0.25} = 31.17$$

$$\text{GoodProduction}_1 = 1 + 0.42 \times 0.0^{0.75} \times 8.32^{0.25} = 1.0$$

$$\text{GoodProduction}_2 = 1 + 0.42 \times 4.33^{0.75} \times 9.26^{0.25} = 3.20$$

Economics: Production Technology and Factor Combination: Figure 4.21 shows the natural transformation sequence from production base through labor and resource inputs ($110.0 \rightarrow 0.0 \rightarrow 4.33$, $20.0 \rightarrow 8.32 \rightarrow 9.26$) to output evolution ($31.17 \rightarrow 1.0 \rightarrow 3.20$) as a categorical pushout construction. The production function pushout encapsulates the fundamental microeconomic principle of productive transformation through categorical colimits. The Cobb-Douglas production function $1 + \alpha \times L^{\gamma} \times R^{1-\gamma}$ becomes a typed morphism $F_{\text{Production}} : \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{> 0}$ that transforms labor and resource inputs into goods output while preserving technological relationships.

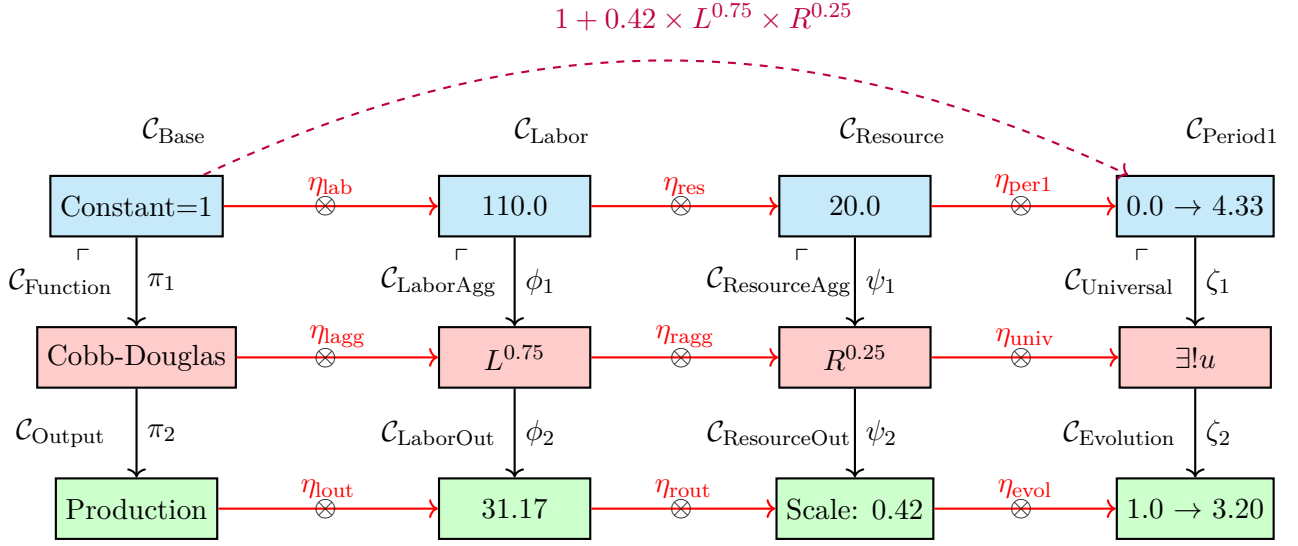


Figure 4.20: Commutative Diagram: Cobb-Douglas Production Pushout Structure

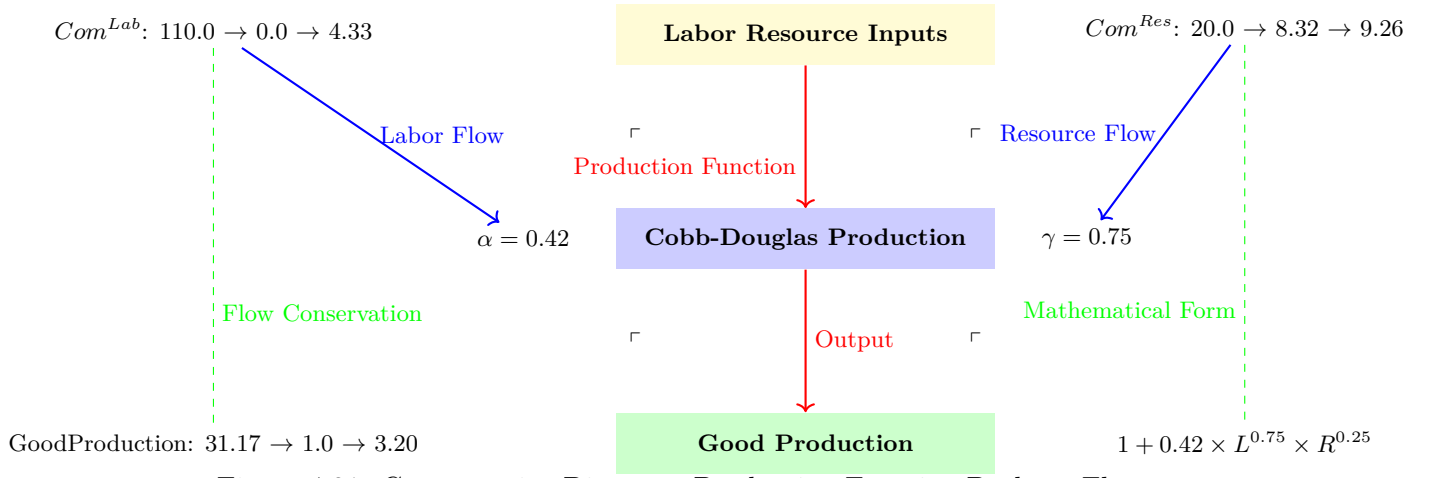


Figure 4.21: Commutative Diagram: Production Function Pushout Flow

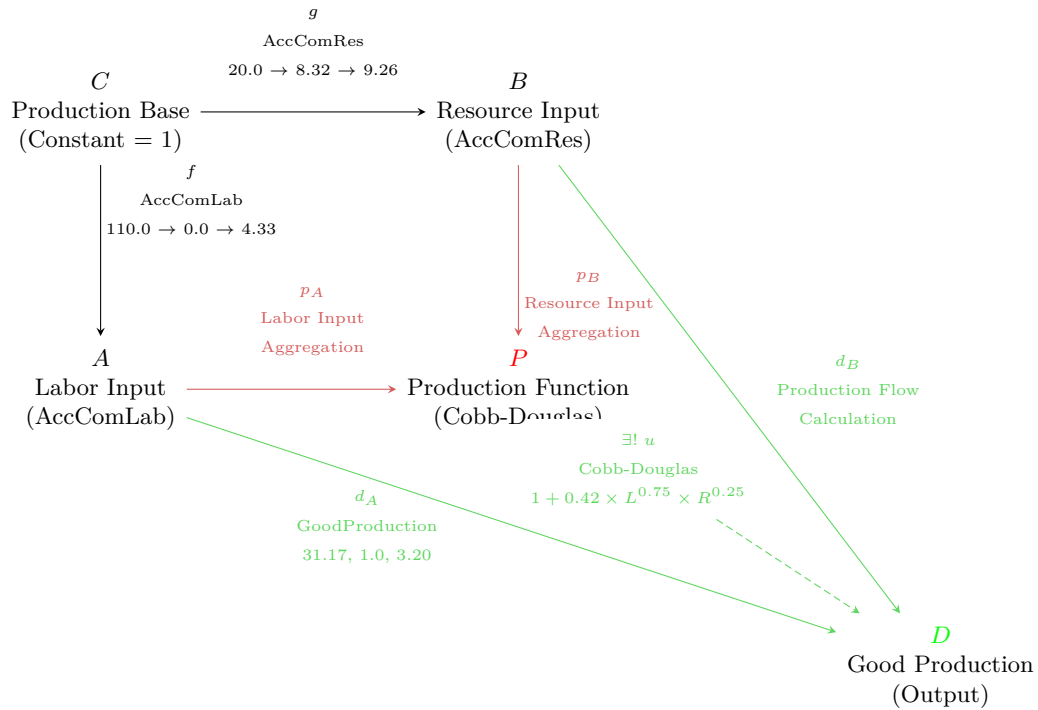


Figure 4.22: Production Function Pushout: The Cobb-Douglas production function pushout combines labor and resource inputs with scaling factor $\alpha = 0.42$ and substitution elasticity $\gamma = 0.75$, producing goods according to technology.

Economically, this captures how firms combine heterogeneous inputs (Com^{Lab} : $110.0 \rightarrow 0.0 \rightarrow 4.33$ and Com^{Res} : $20.0 \rightarrow 8.32 \rightarrow 9.26$) according to substitution possibilities characterized by the elasticity parameter (0.75) and scale factor (0.42). The pushout structure ensures that factor combination respects technological constraints while allowing for input substitution. The output varies ($GoodProduction$: $31.17 \rightarrow 1.0 \rightarrow 3.20$) and reflects how production technology responds to input availability changes and consumption decisions consistently taking place within all other events.

The three-layer architecture of diagram 4.21 reveals the categorical nature of production technology: the yellow layer represents input factor endowments, the blue layer embodies the production technology (Cobb-Douglas transformation), and the green layer shows the resulting output levels. The pushout corners mark where individual factor inputs compose into the universal production output. This categorical approach to production theory provides rigorous foundations for understanding technological relationships in production, connecting microeconomic production theory with categorical mathematics and enabling formal analysis of technological progress and factor substitution patterns. The consistent factorisations in the universal constructions are a key feature of the categorical approach to production and resource theory and to economic evolution in parallel, sequential, hierarchical and universal structures with endofunctors. Dynamics may include optimal adjustment of institutions as optimal hierarchical levels of control. Institutional evolution in universals may involve learning constitutional logical rules for example by inductive logical programming. Polycentric management of public AND private good production and consumption then forms a complete specification of economics.

4.3.2 Cost-Based Planning and Market Equilibration

The economic foundation of the price formation functor requires understanding two sequential categorical constructions: the company's cost-based revenue planning and the market's equilibration mechanism. These represent the fundamental tension between company cost structures and consumer demand patterns. The company's demand planning functor transforms contractual obligations into revenue targets that ensure cost recovery and profit generation.

Definition 4.6 (Cost-Based Planning). The cost-based planning functor F_{CostPlan} computes the company's required revenue based on contractual obligations:

$$\text{DemandPlan} = (\text{WagesPayment} + \text{RepaysPayment}) \times (1 + \mu)$$

The cost planning type signature is:

$$F_{\text{CostPlan}} : \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0} \quad F_{\text{CostPlan}}(w, r, \mu) = (w + r) \times (1 + \mu) \quad (34)$$

where $w = \text{WagesPayment}$, $r = \text{RepaysPayment}$, $\mu = \text{MarkUp}$. ■

Evolution across **periods 0,1,2**:

$$\begin{aligned} \text{DemandPlan}_0 &= (0.0 + 0.0) \times (1 + 0.5) = 0.0 \\ \text{DemandPlan}_1 &= (52.0 + 26.0) \times (1 + 0.5) = 117.0 \\ \text{DemandPlan}_2 &= (109.90 + 54.95) \times (1 + 0.5) = 247.27 \end{aligned}$$

Figure (4.23) shows the transformation from historical contractual obligations (wage and debt service memories) through cost aggregation ($0.0 \rightarrow 78.0 \rightarrow 164.85$) to revenue targets ($0.0 \rightarrow 117.0 \rightarrow 247.27$) via categorical construction. The market disequilibrium functor identifies the tension between consumer demand and company revenue requirements.

Definition 4.7 (Market Disequilibrium). The market disequilibrium functor $F_{\text{Disequilibrium}}$ computes the gap between actual and planned demand:

$$\text{DemandSurplus} = \text{Demand} - \text{DemandPlan}$$

The disequilibrium type signature is:

$$F_{\text{Disequilibrium}} : \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R} \quad F_{\text{Disequilibrium}}(d, d^*) = d - d^* \quad (35)$$

$$(36)$$

where $d = \text{Demand}$, $d^* = \text{DemandPlan}$ ■

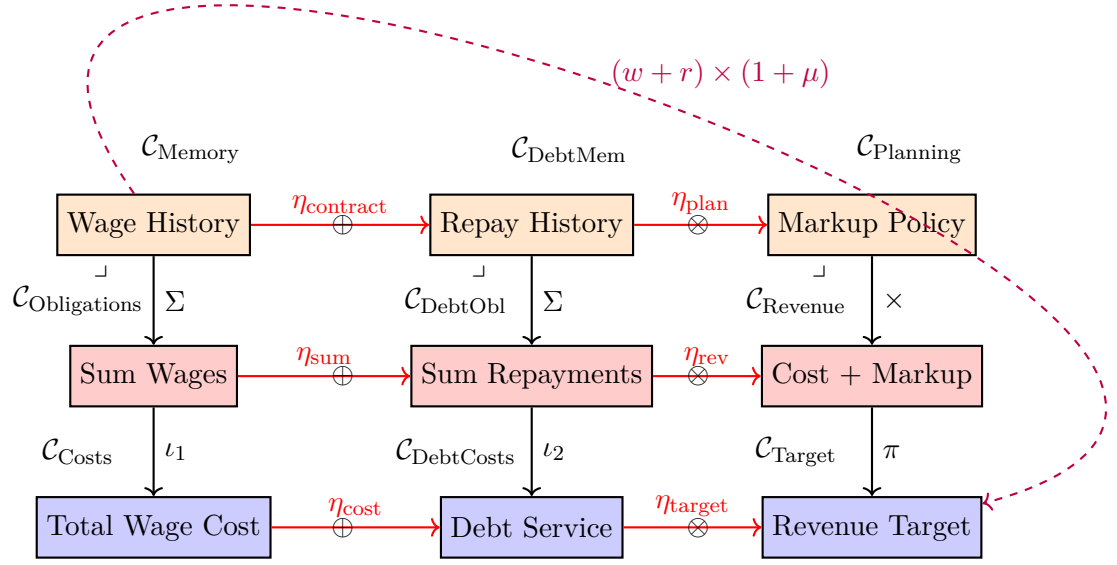


Figure 4.23: Commutative Diagram: Cost-Based Planning Structure

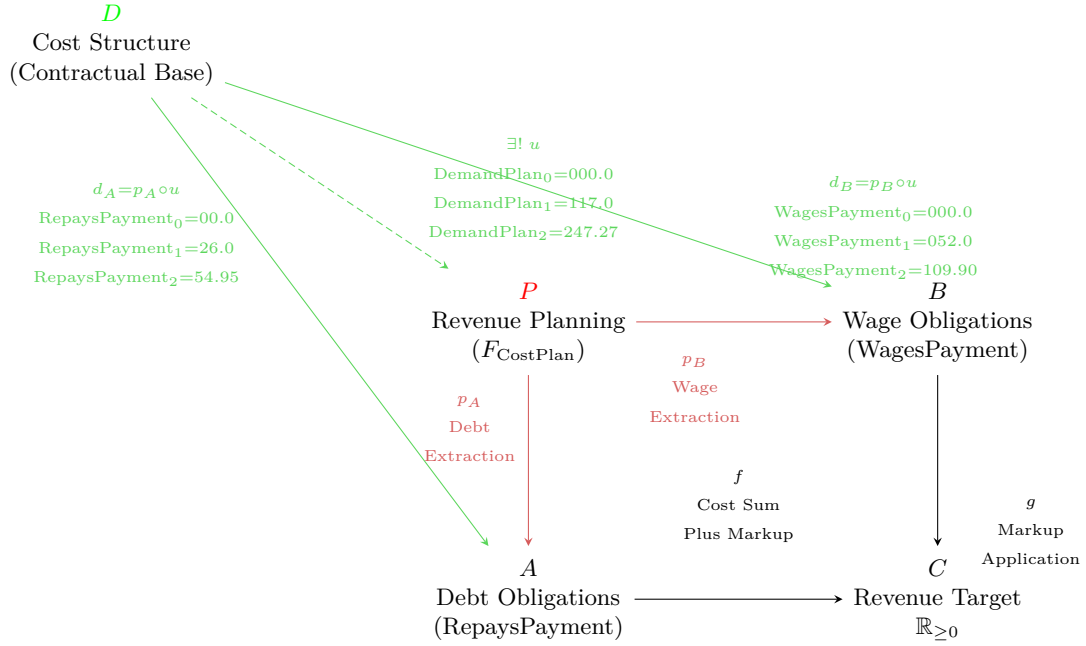


Figure 4.24: Cost-Based Planning Pullback: The cost planning pullback ensures revenue targets cover all contractual obligations (wages, debt service) plus desired profit markup.

Evolution across periods:

$$\text{DemandSurplus}_0 = 0.0 - 0.0 = 0.0$$

$$\text{DemandSurplus}_1 = 166.4 - 117.0 = 49.4$$

$$\text{DemandSurplus}_2 = 272.63 - 247.27 = 25.36$$

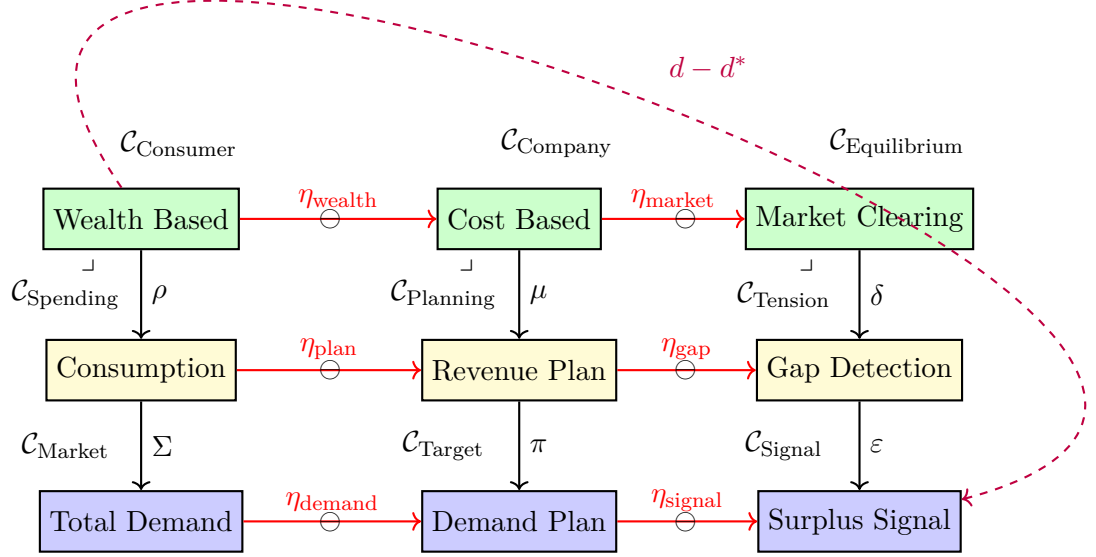


Figure 4.25: Commutative Diagram: Market Equilibration Structure

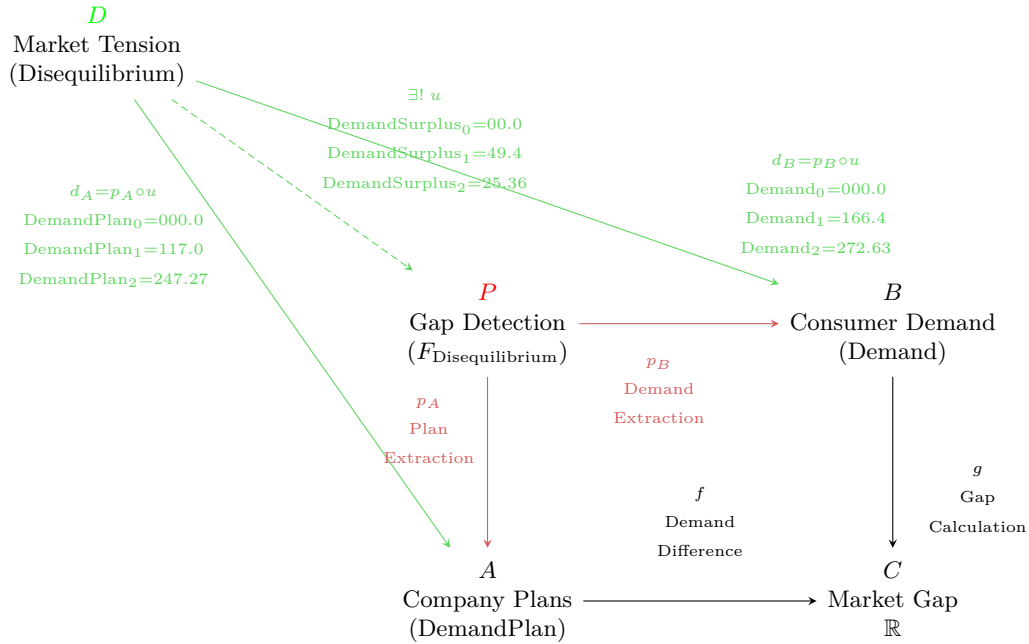


Figure 4.26: Market Disequilibrium Pullback: The disequilibrium pullback measures market tension between consumer spending capacity and company revenue requirements.

Economics: Cost Structure and Market Coordination Figure 4.25 shows the tension resolution between consumer demand ($0.0 \rightarrow 166.4 \rightarrow 272.63$) and company revenue plans ($0.0 \rightarrow 117.0 \rightarrow 247.27$) producing market signals ($0.0 \rightarrow 49.4 \rightarrow 25.36$).

The cost-based planning and market equilibration functors embody the fundamental economic tension between company survival constraints and market demand patterns. The cost planning functor $(w+r) \times (1+\mu)$ represents the minimum revenue requirement for company viability, transforming contractual obligations (wages: $0.0 \rightarrow 52.0 \rightarrow 109.90$, debt service: $0.0 \rightarrow 26.0 \rightarrow 54.95$) into survival-critical revenue targets ($0.0 \rightarrow 117.0 \rightarrow 247.27$).

The market disequilibrium functor $d - d^*$ detects coordination failures between consumer spending capacity ($0.0 \rightarrow 166.4 \rightarrow 272.63$) and company revenue requirements. The resulting demand surplus ($0.0 \rightarrow 49.4 \rightarrow 25.36$) becomes the crucial signal for investment decisions and price adjustments, representing the economy's adaptive mechanism for resolving mismatches of demand and supply.

This categorical decomposition reveals the sequential nature of market coordination: companies first determine survival critical revenue targets based on contractual obligations, then markets generate equilibrium signals through demand plan gaps. The three-layer architecture of diagram 4.25 shows how historical contractual commitments (orange layer) flow through current planning decisions (yellow layer) to generate market coordination signals (blue layer). This provides mathematical foundations for understanding dynamics like business cycles as categorical limit constructions, connecting microeconomic cost structures with macroeconomic coordination mechanisms.

4.3.3 Price Formation

Definition 4.8 (Price). The price functor F_{price} transforms demand and supply into prices. The price coordinates the turnover plan of the company with the consumption plan of the consumers mediated through production:

$$\text{GoodPrice} = \frac{\text{DemandPlan}}{\text{GoodProduction}} + \mu$$

$$F_{price} : \mathbb{R}_{\geq 0} \times \mathbb{R}_{> 0} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{> 0} \quad F_{\text{PriceFormation}}(d, q, m) = \frac{d}{q} + \mu \quad (37)$$

where $d = \text{DemandPlan}$, $q = \text{GoodProduction}$, $\mu = \text{PriceMarkup}$. ■

Evolution across *periods 0, 1, 2*:

$$\begin{aligned} \text{GoodPrice}_0 &= \frac{0.0}{31.17} + 30.0 = 30.0 \\ \text{GoodPrice}_1 &= \frac{117.0}{1.0} + 24.7 = 141.7 \\ \text{GoodPrice}_2 &= \frac{247.27}{3.20} + 12.65 = 89.94 \end{aligned}$$

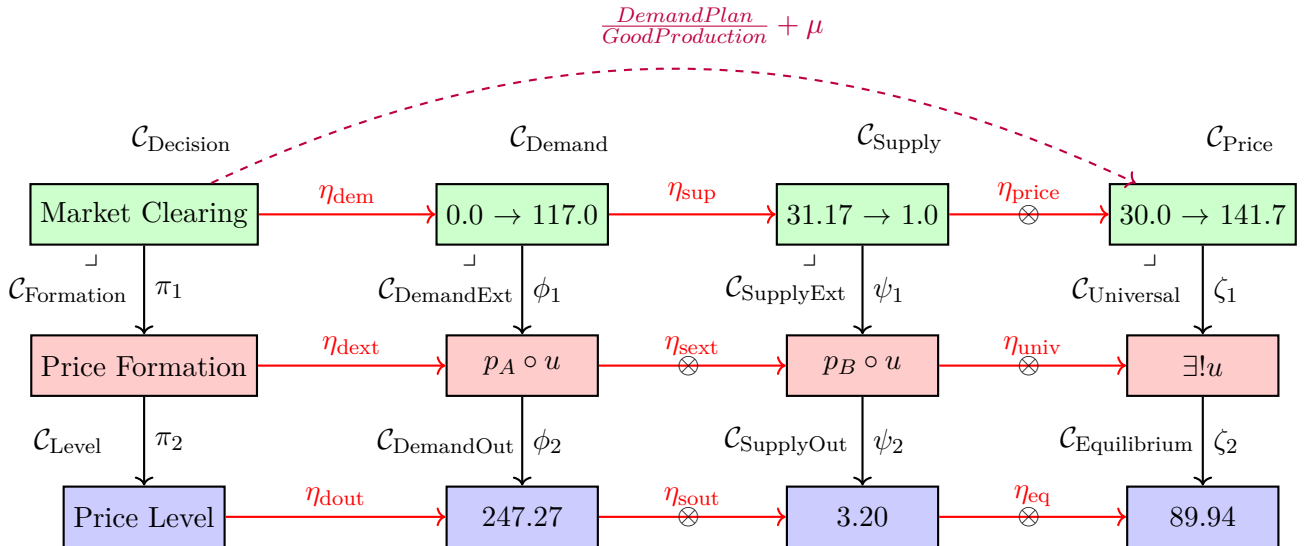


Figure 4.27: Commutative Diagram: Price Formation Pullback Structure

Economics: Market Clearing and Price Discovery Mechanisms: Figure 4.27 shows the natural transformation sequence from market clearing decision through demand and supply coordination ($0.0 \rightarrow 117.0 \rightarrow 247.27$, $31.17 \rightarrow 1.0 \rightarrow 3.20$) to price equilibrium ($30.0 \rightarrow 141.7 \rightarrow 89.94$) via categorical pullback construction.

The price formation pullback embodies the fundamental principle of market equilibrium through categorical limits. The price formation function $\text{DemandPlan}/\text{GoodProduction} + \mu$ becomes a typed morphism

$F_{price} : \mathbb{R}_{\geq 0} \times \mathbb{R}_{>0} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{>0}$ that transforms demand-supply relationships into equilibrium prices while ensuring market clearing conditions.

Economically, this captures the essential market mechanism where prices emerge from the interaction of demand plan (DemandPlan: $0.0 \rightarrow 117.0 \rightarrow 247.27$) and supply (GoodProduction: $31.17 \rightarrow 1.0 \rightarrow 3.20$) forces, mediated by markup policies (PriceMarkup: $30.0 \rightarrow 24.7 \rightarrow 12.65$). The pullback constraint ensures that price formation respects both demand constraints and supply capacities, generating equilibrium prices (GoodPrice: $30.0 \rightarrow 141.7 \rightarrow 89.94$) that clear markets, resulting in the price evolution.

The three-layer architecture of diagram 4.28 reveals the categorical nature of price discovery: the yellow layer represents market clearing conditions, the blue layer embodies the price formation mechanism, and the green layer shows the resulting equilibrium prices. The pullback corners ensure that price formation respects both demand-side constraints and supply-side constraints simultaneously. This categorical approach to price theory provides mathematical foundations for understanding market equilibrium as a categorical limit, connecting to general equilibrium theory by categorical mathematics for a formal analysis of price stability, market efficiency and the compositional, universal and consistency properties of the *general* of general equilibrium theory.

4.4 Investment Decisions

4.4.1 Investment Decision Sigmoid

Definition 4.9 (Investment Learning). The investment morphism is specified to implement a sigmoid activation function as the sectoral and overall systems learning and adaptation technology. The sigmoid function transforms *DemandSurplus* into investment:

$$\text{Investment} = \sigma_A + \frac{\sigma_B}{1 + e^{-\frac{\text{DemandSurplus}}{\sigma_C}}}$$

$$F_{\text{Investment}} : \mathbb{R} \times \mathbb{R}_{>0} \times \mathbb{R}_{>0} \times \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0} \quad (38)$$

$$F_{\text{Investment}}(\text{DemandSurplus}, \sigma_A, \sigma_B, \sigma_C) = \sigma_A + \frac{\sigma_B}{1 + e^{-\frac{\text{DemandSurplus}}{\sigma_C}}} \quad (39)$$

with $\sigma_A = 20.0$, $\sigma_B = 480.0$, $\sigma_C = 200.0$: ■

Evolution across *periods* 0, 1, 2:

$$\begin{aligned} \text{Investment}_0 &= 20.0 + \frac{480.0}{1 + \exp(-0.0/200.0)} = 260.0 \\ \text{Investment}_1 &= 20.0 + \frac{480.0}{1 + \exp(-49.4/200.0)} = 289.49 \\ \text{Investment}_2 &= 20.0 + \frac{480.0}{1 + \exp(-25.36/200.0)} = 275.20 \end{aligned}$$

Economics: Investment Decision-Making: Figure 4.32 shows the natural transformation sequence from investment decision through demand surplus ($0.0 \rightarrow 49.4 \rightarrow 25.36$) and parameters ($\sigma_A = 20.0, \sigma_B = 480.0, \sigma_C = 200.0$) to investment levels ($260.0 \rightarrow 289.49 \rightarrow 275.20$) via categorical pullback construction.

The investment sigmoid pullback captures the corporate investment decisions through categorical structures. The sigmoid transformation $\sigma_A + \sigma_B/(1 + \exp(-\text{DemandSurplus}/\sigma_C))$ becomes a typed morphism $F_{\text{Investment}} : \mathbb{R} \times \mathbb{R}_{>0} \times \mathbb{R}_{>0} \times \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0}$ that transforms demand surplus signals into investment levels while incorporating behavioral response patterns.

Economically, this represents how firms respond to market signals (DemandSurplus: $0.0 \rightarrow 49.4 \rightarrow 25.36$) through some rationality and gradual adjustment and learning mechanisms captured by the sigmoid parameters ($\sigma_A = 20.0, \sigma_B = 480.0, \sigma_C = 200.0$). The pullback ensures that investment decisions emerge from the interaction of market conditions and firm behavioral parameters, generating investment patterns (Investment: $260.0 \rightarrow 289.49 \rightarrow 275.20$) that avoid extreme responses and maintaining sensitivity to economic conditions.

The three-layer architecture of diagram 4.31 reveals the categorical nature of behavioral investment theory: the yellow layer represents investment decision triggers, the blue layer embodies the behavioral

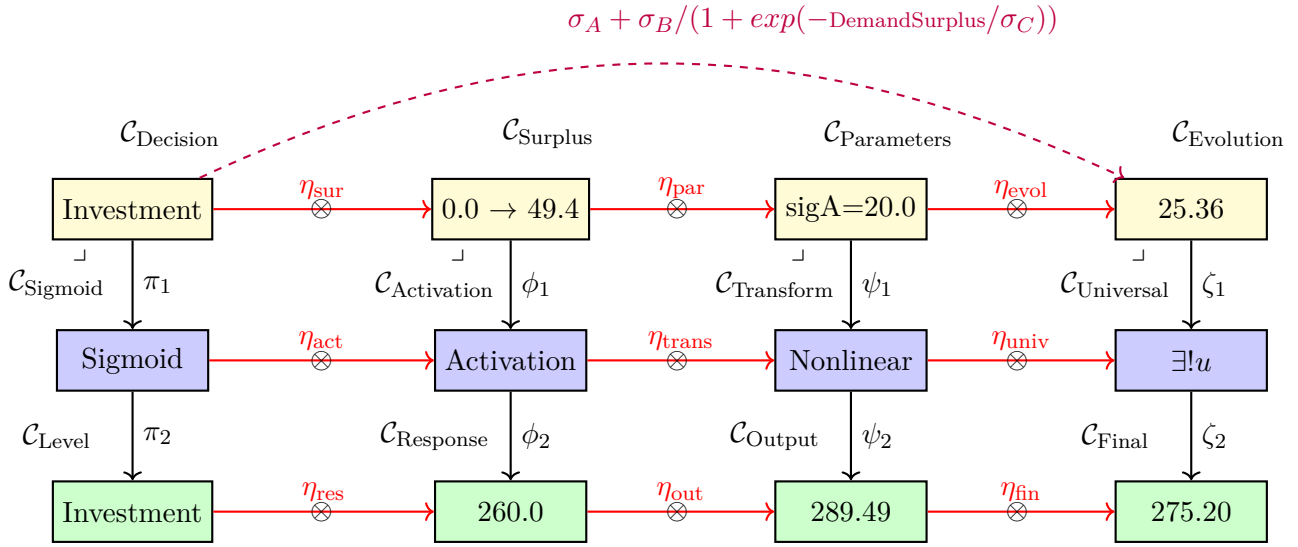


Figure 4.30: Commutative Diagram: Investment Sigmoid Pullback Structure

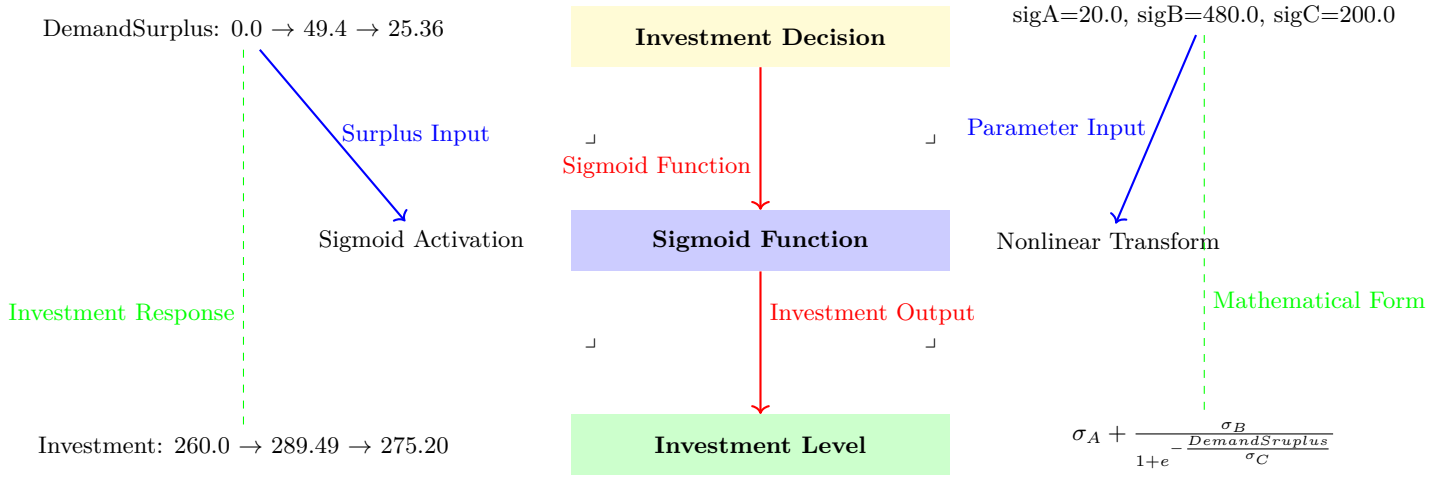


Figure 4.31: Commutative Diagram: Investment Sigmoid Pullback Flow

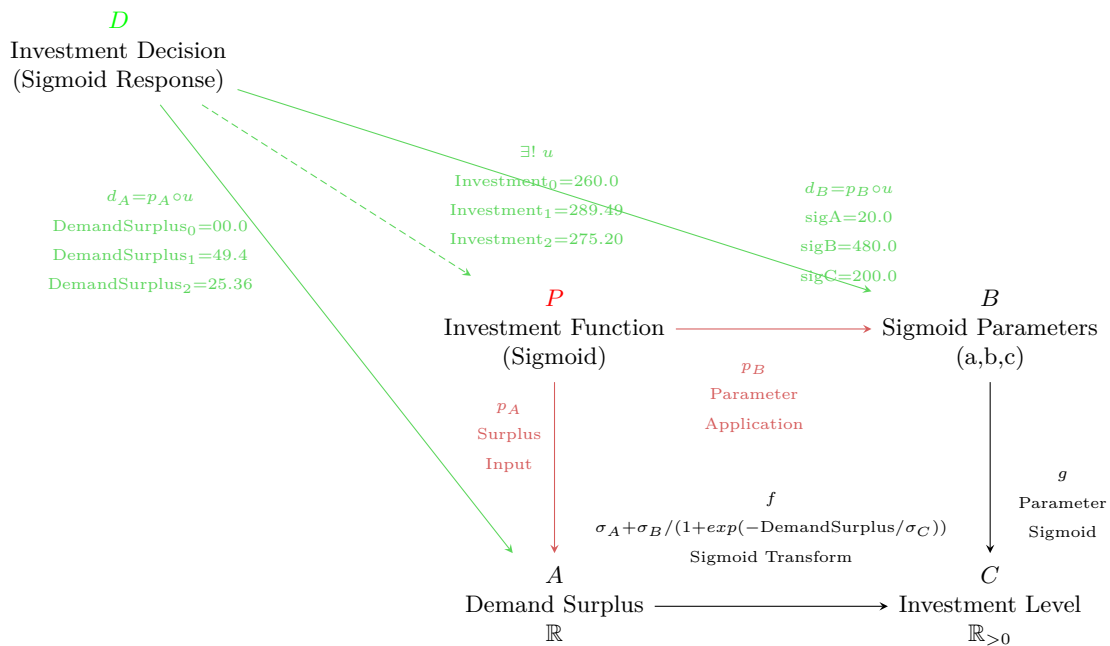


Figure 4.32: Investment Sigmoid Pullback: The investment decision pullback applies the sigmoid function to demand surplus, with parameters ($\sigma_A = 20.0, \sigma_B = 480.0, \sigma_C = 200.0$) governing the response curve.

response mechanism (sigmoid activation), and the green layer shows the resulting investment levels. The pullback corners ensure that investment responses respect both market signal constraints and behavioral parameter constraints. This categorical approach to investment theory bridges microeconomic investment theory with behavioral finance, providing mathematical foundations for understanding how market signals translate into capital formation through firm behavior patterns that may incorporate bounded rationality or adjustment costs.

4.4.2 Investment Allocation

Definition 4.10 (Investment Allocation). The investment allocation functors distribute investment between resources and labor.

$$\text{InvestmentRes} = \text{Investment} \times \lambda \quad (40)$$

$$\text{InvestmentLab} = \text{Investment} \times (1 - \lambda) \quad (41)$$

$$F_{\text{InvestRes}} : \mathbb{R}_{>0} \times [0, 1] \rightarrow \mathbb{R}_{\geq 0} \quad F_{\text{InvestRes}}(i, s) = i \times s \quad (42)$$

$$F_{\text{InvestLab}} : \mathbb{R}_{>0} \times [0, 1] \rightarrow \mathbb{R}_{\geq 0} \quad F_{\text{InvestLab}}(i, s) = i \times (1 - s) \quad (43)$$

with constraint $s_{\text{res}} + s_{\text{lab}} = 1$ and $\lambda = 0.8$ and $1 - \lambda = 0.2$: ■

Evolution across **periods 0, 1, 2**:

$$\text{InvestmentRes}_0 = 260.0 \times 0.8 = 208.0$$

$$\text{InvestmentRes}_1 = 289.49 \times 0.8 = 231.59$$

$$\text{InvestmentRes}_2 = 275.20 \times 0.8 = 220.16$$

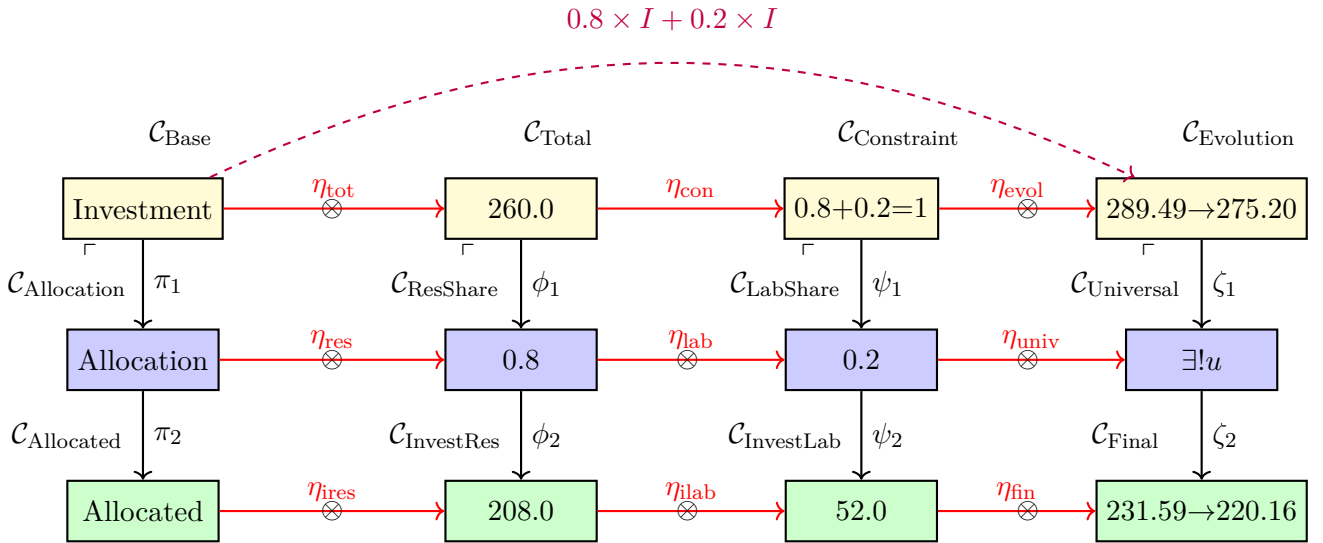


Figure 4.33: Commutative Diagram: Investment Allocation Pushout Structure

Economics: Capital Allocation and Portfolio Theory: Diagram 4.33 shows the natural transformation sequence from investment base through total investment ($260.0 \rightarrow 289.49 \rightarrow 275.20$) and share constraint ($0.8+0.2=1$) to resource and labor allocation ($208.0 \rightarrow 231.59 \rightarrow 220.16$, $52.0 \rightarrow 57.90 \rightarrow 55.04$) via the categorical pushout construction.

The investment allocation pushout embodies the fundamental principle of capital budgeting through categorical colimits. The allocation functions $F_{\text{InvestRes}}$ and $F_{\text{InvestLab}}$ become typed morphisms that distribute total investment capital (Investment: $260.0 \rightarrow 289.49 \rightarrow 275.20$) across factor-specific investments while respecting portfolio constraint that allocation shares sum to unity ($0.8 + 0.2 = 1$).

Economically, this captures how firms allocate scarce investment capital across heterogeneous productive factors based on factor-specific investment shares ($\lambda = 0.8$, $1 - \lambda = 0.2$). The pushout structure ensures that allocation decisions respect both capital constraints and factor complementarity relationships, generating

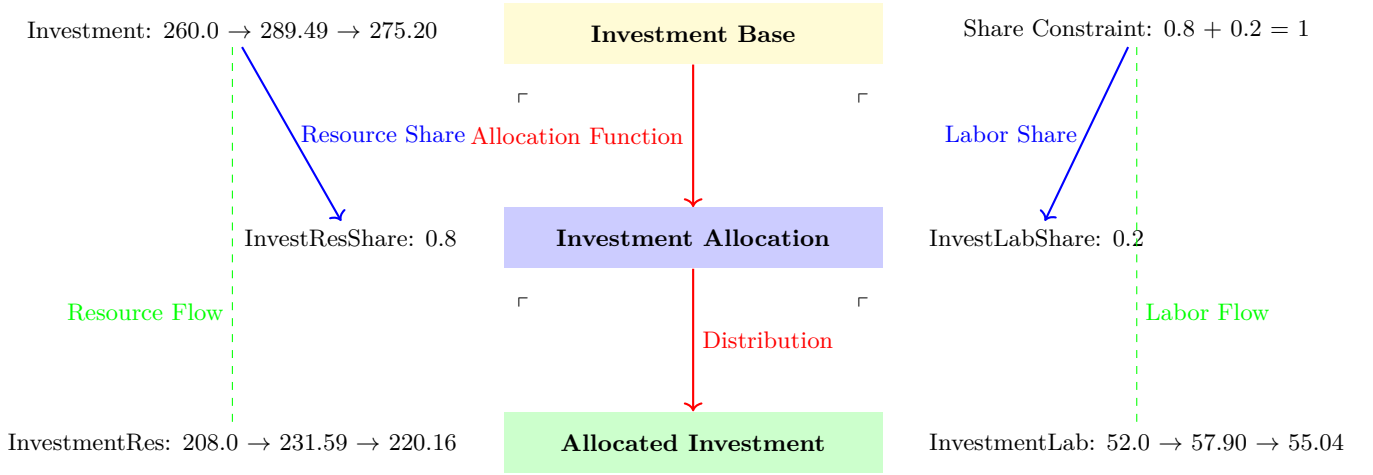


Figure 4.34: Commutative Diagram: Investment Allocation Pushout Flow

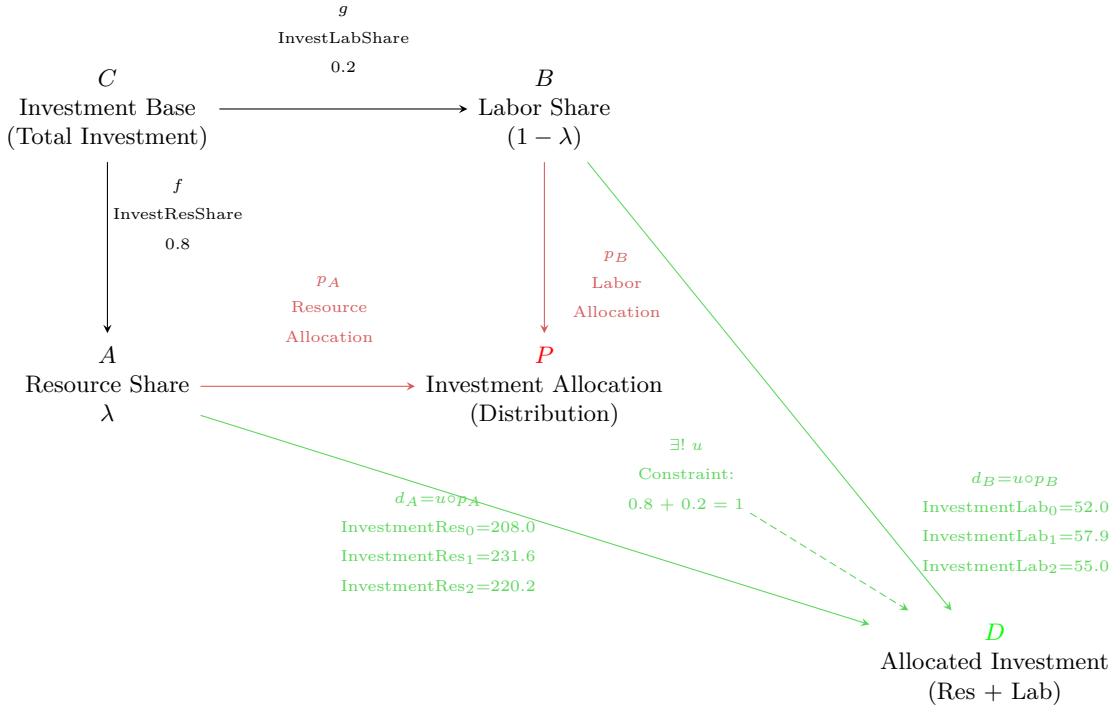


Figure 4.35: Investment Allocation Pushout: The investment allocation pushout distributes total investment between resource and labor shares (0.8 and 0.2), ensuring the constraint that shares sum to 1.

balanced investment flows (InvestmentRes: 208.0 $\rightarrow$ 231.59 $\rightarrow$ 220.16 and InvestmentLab: 52.0 $\rightarrow$ 57.90 $\rightarrow$ 55.04) that maintain productive factor balance while adapting to changing economic conditions.

The three-layer architecture of diagram 4.34 reveals the categorical nature of capital allocation theory: the yellow layer represents total investment capacity, the blue layer embodies the allocation mechanism, and the green layer shows the resulting factor-specific investment flows. The pushout corners mark where total investment morphisms compose into the universal factor allocation morphism. This categorical approach to capital budgeting connects microeconomic investment theory with portfolio optimisation theory, providing mathematical foundations for understanding how firms balance competing investment opportunities while maintaining operational efficiency and responding to changing factor price relationships.

4.4.3 Intertemporal Contract Memory Updates

Investment decisions create future contractual obligations that must be tracked through categorical state updates. This memory mechanism represents the intertemporal dimension of economic contracts, where current investment decisions generate future payment obligations.

The memory update functors manage the intertemporal contract obligations created by investment decisions.

Definition 4.11 (Memory Update). The memory update functors $F_{\text{MemoryUpdate}}$ maintain FILO (First In, Last Out) stacks of contractual obligations:

$$\mathcal{H}_{wage}^{t+1} = \text{p2H}(\mathcal{H}_{wage}^t, \text{InvestmentLab}^t) \quad (44)$$

$$\mathcal{H}_{repay}^{t+1} = \text{p2H}(\mathcal{H}_{repay}^t, \text{Repayment}^t) \quad (45)$$

The FILO update operation is defined as:

$$F_{\text{FILO}} : \mathbb{R}^\tau \times \mathbb{R} \rightarrow \mathbb{R}^\tau \quad F_{\text{FILO}}([h_1, h_2, \dots, h_\tau], x) = [x, h_1, h_2, \dots, h_{\tau-1}] \quad (46)$$

where $\tau = 10$ is the memory length.

The memory update is implemented as imperative state modification: `p2H(hist, newelem) = [newelem; hist[1:end-1]]`

`stateNew.wageHist = p2H(state.wageHist, InvestmentLab)`

`stateNew.repayHist = p2H(state.repayHist, Repayment)`

■

The categorical formulation integrates memory updates as natural transformations that preserve temporal ordering while maintaining categorical structure through state morphisms. Evolution of wage memory across **periods 0, 1, 2**:

$$\mathcal{H}_{wage,0} = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]$$

$$\mathcal{H}_{wage,1} = [52.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]$$

$$\mathcal{H}_{wage,2} = [57.90, 52.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]$$

Evolution of repayment memory across **periods 0, 1, 2**:

$$\mathcal{H}_{repay,0} = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]$$

$$\mathcal{H}_{repay,1} = [26.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]$$

$$\mathcal{H}_{repay,2} = [28.95, 26.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]$$

Economics: Temporal Consistency and Contract Theory: The memory update mechanism creates a crucial *intertemporal closure and feedback loop* in the economic system. Today's investment decisions become tomorrow's cost structure. This closure arises from the closure of debt relations in loans and repayments and the closure of production and consumption relations. Figure 4.36 shows how investment decisions create future contractual obligations through FILO memory updates, transforming current investment flows (52.0 $\rightarrow$ 57.90 $\rightarrow$ 55.04, 26.0 $\rightarrow$ 28.95 $\rightarrow$ 27.52) into temporally ordered payment streams via categorical memory functors.

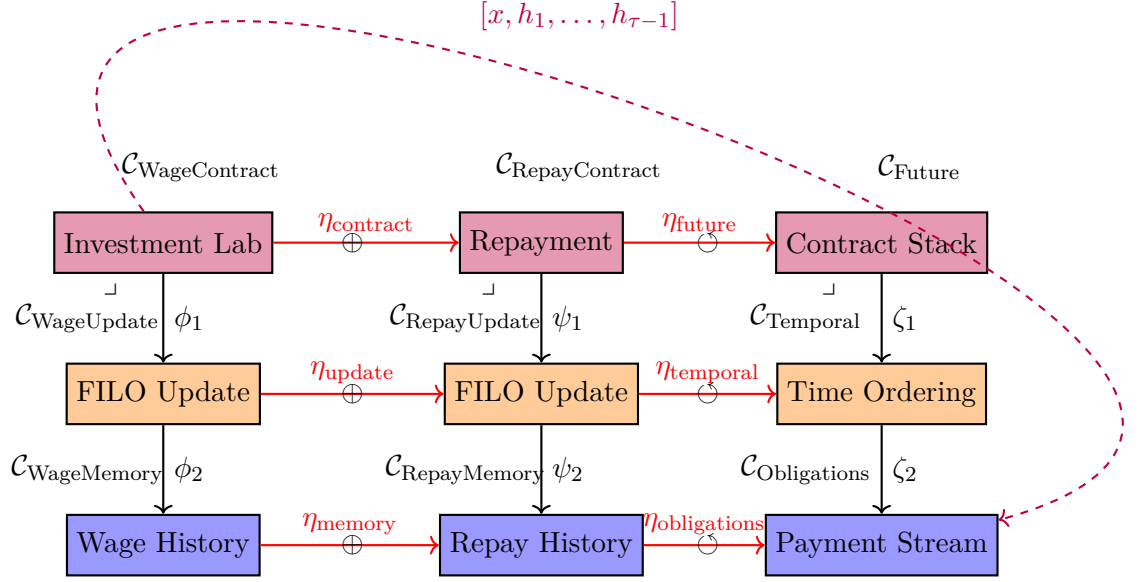


Figure 4.36: Commutative Diagram: Intertemporal Contract Memory Structure

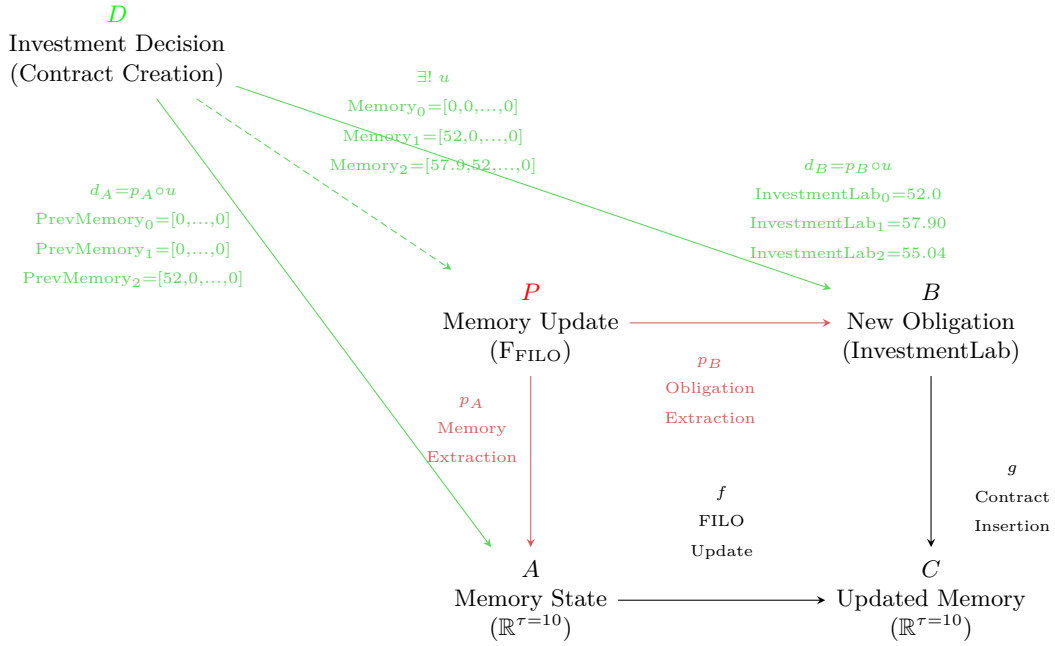


Figure 4.37: Memory Update Pullback: The memory update pullback ensures intertemporal contract obligations are correctly inserted into FILO stacks, preserving temporal ordering of contractual commitments.

$$\text{Investment}^t \rightarrow \text{FutureObligation}^{t+1, \dots, t+\tau} \quad (47)$$

$$\text{FutureObligation}^{t+1} \rightarrow \text{CostStructure}^{t+1} \quad (48)$$

$$\text{CostStructure}^{t+1} \rightarrow \text{DemandPlan}^{t+1} \quad (49)$$

The two intertemporal memory updates on the labor and investment contracts embody an important economic principle that current investment decisions create future contractual obligations. The FILO structure $F_{\text{FILO}} : \mathbb{R}^\tau \times \mathbb{R} \rightarrow \mathbb{R}^\tau$ represents how investment commitments (InvestmentLab: 52.0 $\rightarrow$ 57.90 $\rightarrow$ 55.04, Repayment: 26.0 $\rightarrow$ 28.95 $\rightarrow$ 27.52) generate temporally ordered payment obligations that constrain future economic decisions.

The types of the memory reveal how economic systems maintain *temporal consistency* through contract enforcement mechanisms. The memory updates ensure that investment decisions today (generating investment flows) automatically become cost constraints tomorrow (through WagesPayment and RepaysPayment aggregation), creating an essential feedback mechanism that connects investment cycles with cost structures across time periods. The diagrams show how categorical memory functors preserve temporal ordering while maintaining economic consistency: investment decisions (purple layer) flow through FILO updates (orange layer) to create structured payment obligations (blue layer). This provides mathematical foundations for understanding how modern economies balance current investment opportunities with future payment obligations, connecting investment theory with contract theory through categorical time structures. It also shows how to construct contracts as executed in verified and consistently aggregated macroeconomic accounting systems.

The modal logical aspects of contracts over time can be modelled in coalgebras for equation (47) for finit and infinite τ for institutions, companies, corporates or sectors. In computer science the categorical coalgebraic approach is used for programming language design and their semantics (DSL). A fundamental paper by Rutten [2000] shows the categorical constructions used for dynamical systems from a computer scientific point of view. In Abramsky and Winschel [2015] the coalgebraic approach is used for infinite game theory. Coalgebras are also useful for the stochastic extension and game theoretical branching logics, like trees and branching time in Kripke structures or modalities of deontic, temporal or other forms of logic. An economic logic can be added on top of the memory of the system or subsystems. On the practical side coalgebras are infinite data structures that allow to express infinite streams as in infinite horizon econometric theories like (Winschel and Krätzig [2010]). The extraction of calculations from these obviously never completely computable structures, is to evaluate them in a lazy way where calculations are only promised to be done, and only calculated if needed. Calculations is always an extraction of some first finite numbers from the infinite streams located in final coalgebras. The expressivity of the DSL increases very much with coalgebraic operations for example provided as stream calculus. The calculus allows to express high level economic concepts in a compositional way and let the compiler compose the meaning as the calculation of the whole program from the behavioural meaning of the calculated parts. This reduces the high level concepts into the low level implementation, which we do not want to be concerned with, with the consistency of the calculations being enforced by the compiler. The lazily evaluated finite parts of the infinite stream are then the simulations of a finite sample from an infinite stream to which the parameters of the simulations map in the coalgebras. Convergence in numerical methods and estimators or Monte Carlo simulations are mathematically typed as coalgebras. So, it is useful to accompany the initial algebras of usual calculations and recursions by the coalgebraic final coalgebras.

The legal abstraction principle used in Menéndez and Winschel [2025] to disentangle the nature and type of money will be typed as contra variant and covariant functors as the *tit for tat* in legal value exchanges. The economic and legal content of contracts as programs in high level DSL languages are constructions on final structures complementing initial algebras. The products of real and nominal categories should be analysed from the points of view and functors of denotic (denotic modalities formalise legal musts and can haves) and temporal (like eventually, surely, maybe, once ...) modalities of legal contracts in equations (47), (48) and (49).

The memory of MoMaT can be extended in many more ways towards the ultimate goal of a consciousness of the economy as a whole. It can contain a categorical state as a current theory an agent maintains about the economy as a whole or subparts. The informational state of agents and aggregated levels entities is a functor that is kept consistent in natural transformations with the overall system information state. The category of the economy $\mathcal{C}_{\text{Economy}}$ can be modularised in any subbranching depths and aspects of reflexive

agents and levels, see Abramsky et al. [2015] for a categorical semantics of reflexive economics. In this workshop, we have laid out the foundations of this paper's categorical types of macroeconomic systems, wherein econometric agents can be modelled.

A memory that emerges into consciousness of biological and cognitive systems has been modelled categorically in terms of colimits as binding and emergence of memory evolutive systems (MES) by Ehresmann and Vanbreemersch [2007]. In this paper we present a memory of contracts, macroeconomics as a colimit (albeit not as colimits arranged which create emergent properties in the aggregated entity as in MES), and an endofunctor implementing a categorical evolution. The coregulators of MES can be modelled as open games of Ghani et al. [2018] for a polycentric management which needs a compositional econometric approach as (sub)system learners. Emergence in MES is about reusing objects and concepts with emergent properties in various contexts as their defining property. In economics of course this can be interpreted as reuse of creatable, copyable and deletable resources in initial algebras and final coalgebras. The multiplicity property of complex objects with emergent properties in MES models the reuse of concepts as colimits. The biological concepts of MES of binding, emergence and concept formation are useful for economic policy implementations and institutional design. The multiplicity property is about reuse of concepts in different contexts which is what makes them valuable. There is an economic value of pulling concepts, as in biological conscious systems, in front of the paranthesis. This is very naturally an economic principle or even the economic principle, that a composition, the whole, is more valuable, i.e. is more than the parts. Coregulators of MES formalise something very similar to polycentrism of optimal hierarchical control levels as a top level systemic economic policy question on decentralisation. The memory can encompass as concept formation, as in MES, the causal detections of structural econometric theories as in Winschel and Krätzig [2010] which can be categorified as adjunctions.

4.4.4 Dividend Decision

Definition 4.12 (Dividend). The dividend morphism transforms company profits into dividend payments:

$$\text{Diff} = \text{GoodProduction} \times \text{GoodPrice} - \text{WagesPayment} - \text{RepayPayment} \quad (50)$$

$$\text{DividendDecision} = \max(0, \text{Diff} \times \text{DivRateDiff}) + \text{AccComBank} \times \text{DivRateBank} \quad (51)$$

$$F_{\text{Diff}} : \mathbb{R}_{>0} \times \mathbb{R}_{>0} \times \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R} \quad F_{\text{Diff}}(q, p, w, r) = q \times p - w - r \quad (52)$$

$$F_{\text{Dividend}} : \mathbb{R} \times \mathbb{R}_{\geq 0} \times [0, 1] \times [0, 1] \rightarrow \mathbb{R}_{\geq 0} \quad F_{\text{Dividend}}(d, b, r_d, r_b) = \max(0, d \times r_d) + b \times r_b \quad (53)$$

■

With $\delta_c = 0.15$ and $\delta_b = 0.4$:

$$\text{Diff}_0 = 52.0 \quad (54)$$

$$\text{Diff}_1 = 138.50 \quad (55)$$

$$\text{Diff}_2 = 121.25 \quad (56)$$

The dividend payments across **periods 0, 1, 2**:

$$\text{DividendDecision}_0 = \max(0, 52.0 \times 0.15) + 0.0 \times 0.4 = 7.8$$

$$\text{DividendDecision}_1 = \max(0, 138.5 \times 0.15) + 52.0 \times 0.4 = 41.6$$

$$\text{DividendDecision}_2 = \max(0, 121.3 \times 0.15) + 190.5 \times 0.4 = 94.4$$

Economics: Corporate Finance and Dividend Policy: The dividend decision pullback embodies the fundamental principle of corporate finance through categorical limits. The dividend function $\max(0, d \times r_d) + b \times r_b$ becomes a typed morphism $F_{\text{Dividend}} : \mathbb{R} \times \mathbb{R}_{\geq 0} \times [0, 1] \times [0, 1] \rightarrow \mathbb{R}_{\geq 0}$ that transforms profit differentials and bank balances into dividend payments while respecting non-negativity constraints and corporate finance policy.

Economically, this captures how firms balance profit distribution to shareholders with operational cash management through systematic dividend policies (DivRateDiff: 0.15, DivRateBank: 0.4). The pullback ensures that dividend decisions emerge from the interaction of profit performance (Diff: 52.0 $\rightarrow$ 138.50 $\rightarrow$ 121.25) and financial reserves (AccComBank: 0.0 $\rightarrow$ 52.0 $\rightarrow$ 190.50), generating dividend patterns (DividendDecision: 7.8 $\rightarrow$ 41.57 $\rightarrow$ 94.39) that reflect both profitability and financial prudence.

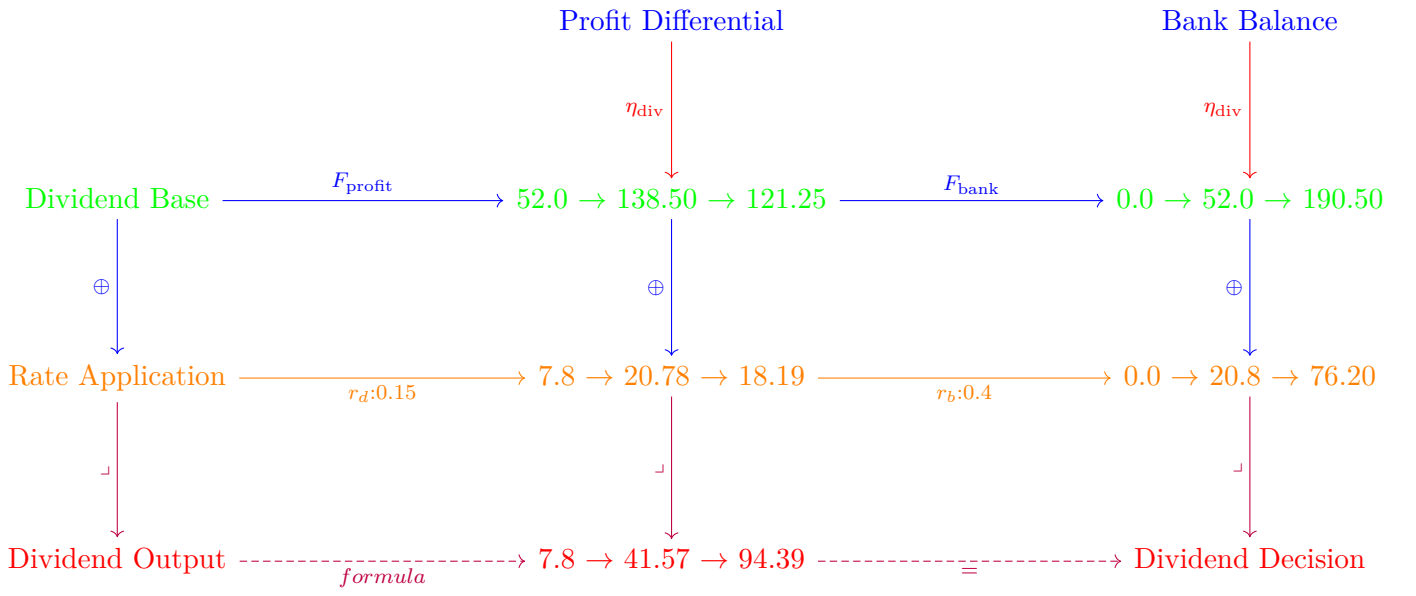


Figure 4.38: Commutative Diagram: Dividend Decision Natural Transformation with pullback structure

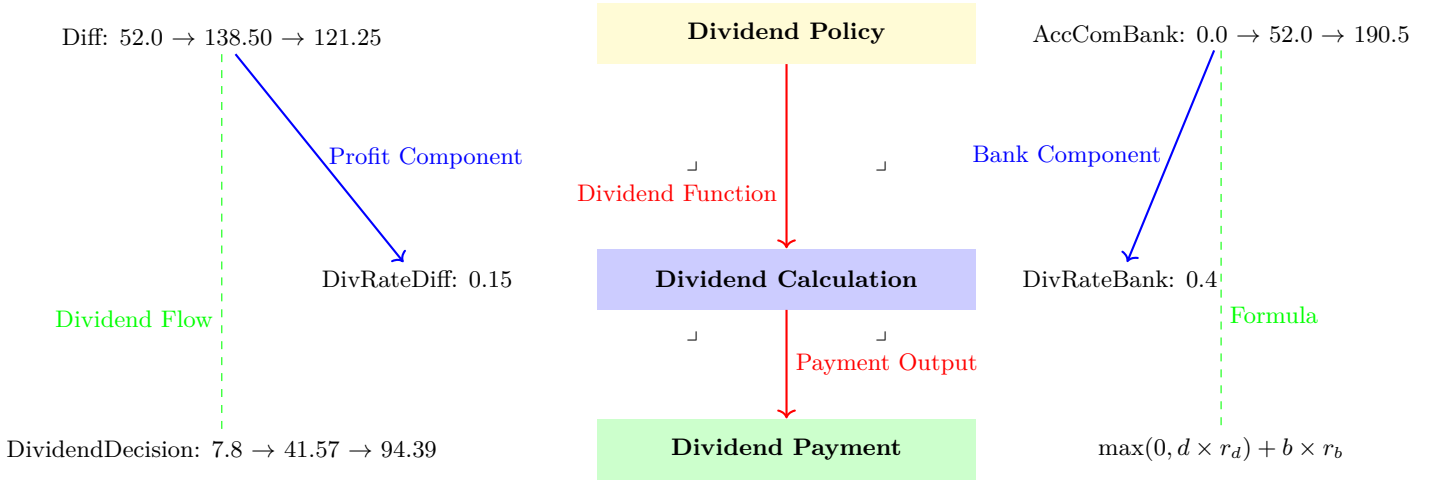


Figure 4.39: Commutative Diagram: Dividend Decision Pullback Flow

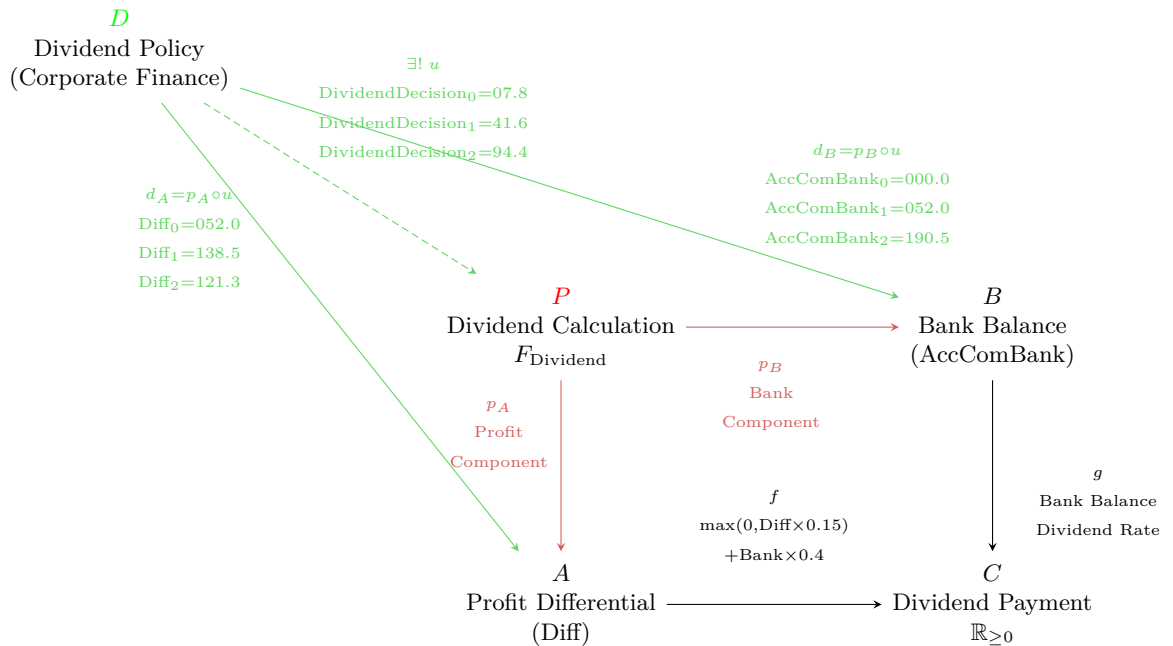


Figure 4.40: Dividend Decision Pullback: The dividend decision pullback combines profit differential and bank balance with respective dividend rates, ensuring non-negative dividend payments.

The three-layer architecture of diagram 4.39 reveals the categorical nature of corporate finance theory: the yellow layer represents dividend policy foundations, the blue layer embodies the dividend calculation mechanism, and the green layer shows the resulting dividend payments. The pullback corners ensure that dividend decisions respect both profit constraints and liquidity constraints simultaneously. This categorical approach to corporate finance provides mathematical foundations for understanding how firms balance shareholder returns with operational requirements, connecting corporate finance theory with categorical mathematics and enabling formal analysis of optimal dividend policies under uncertainty.

4.5 Universal Constructions

Universal constructions in mathematics and category theory are what in economics often the argmax of a maximisation is used for. For an intuition of the vast generalisation universals are over maximisation, we can think about them to also encompass what economists call institutions: structural contextual rules of finding good rules like constitutions for politics or institutional dynamics and diversity, like in polycentrism Ostrom [2005], or the selection functions of open games in Ghani et al. [2018] as higher order functions where the max operator is one of any possible. Institutions are what in economics is a top level structure and is rather awkward to understand and to formulate them as the result of a simple maximisation of some function over reals only. Of course in economics, we discuss *optimal institutions* as an outcome of optimisation but in what sense optimal, what types are involved is to be worked out as part of the theory maintained. A maximisation of a function over reals or vector spaces can take advantage of universal constructions as a first step towards adjunctions. By that we get for economics a general approach to formulate outcomes having some best, maximal, most congruent, unique or whatever universality we need to use to describe reality in theory. Universals as pullbacks and pushouts can be derived from an even more general principle of adjoint functors which are said to be a most important contribution of category theory to mathematics. We do not use adjointness of functors explicitly but only as pushouts and pullbacks but adjoint functors as expressing the optimality itself should be kept in mind when looking at the universal constructions from an economic perspective. For example, for modelling an agent in the theory being an econometrician using a local or globally valid theory of the economic data. The very fact that the universal constructions are themselves constructions of more general categorical patterns hints into the possibility to formulate the economists being in the theory an agent with the same abilities as the agent in the theory, namely building theory including constructing the optimisation itself. Endogenous and more importantly, compositional econometric models are the key to the compositional and endogenous nature of hierarchical control. The compositional properties of econometric theories have to show the available inverse informations of the data in and about the hierachical levels of the system.

4.5.1 Pullbacks for Constraint Validation

Pullbacks implement constraint validation before economic calculations. In case of the investment decision, we categorify the validation property of the banking sector being responsible for granting loans to companies and making sure to get the repayments from the hopefully successful production by the producer and purchase and payment by their customers.

Definition 4.13 (Investment). The investment pullback as a universal construction validates loan creation by the bank for the company. The production process of banking is a universal optimisation in the following diagram.

$$\begin{array}{ccc}
 \text{InvestmentDecision} & \xrightarrow{p_2} & \text{BankLiquidity} \\
 \downarrow p_1 & & \downarrow g \\
 \text{CompanyCapacity} & \xrightarrow{f} & \text{ValidInvestment}
 \end{array}$$

The validation of the pullback is part of the design of investment contracts of the banking and financial sector. This may include econometric or AI methods to validate the properties of the companies and banks involved as a more complex algorithms then the sigmoid learning we have for now. Similarly, sensitivity analysis like in Harenberg et al. [2019] is part of validation of regulatory needs. The universal construction allows to formulate validation rules for the company like looking at its bank account. Also bank's liquidity

contracts can be a complex structure of validations of international business and financial plans in the supply decision of liquidity at the bank. With central banks and governments the validation rules become even more complex. Any algorithm that improves the validation process can be build in as a maximisation, universal construction or adjunction as needed.

$$\text{Pullback}_{\text{Investment}} : \mathbb{R}_{>0} \times \mathbb{R}_{\geq 0} \rightarrow \{0, 1\} \quad \text{Pullback}_{\text{Investment}}(i, c) = \begin{cases} 1 & \text{if } i \leq c + \text{CreditLimit} \\ 0 & \text{otherwise} \end{cases} \quad (57)$$

where

$$\begin{aligned} p_1 &: \text{InvestmentDecision} \rightarrow \text{CompanyCapacity} \\ p_2 &: \text{InvestmentDecision} \rightarrow \text{BankLiquidity} \\ f &: \text{CompanyCapacity} \rightarrow \text{ValidInvestment} \\ g &: \text{BankLiquidity} \rightarrow \text{ValidInvestment} \end{aligned}$$

■

Investment validation evolution in **periods 0, 1, 2**.

$$\begin{aligned} \text{Investment}_0 &= 260.0 \text{ (validates against initial state)} \\ \text{Investment}_1 &= 289.49 \text{ (validates against period 1 liquidity)} \\ \text{Investment}_2 &= 275.20 \text{ (validates against period 2 liquidity)} \end{aligned}$$

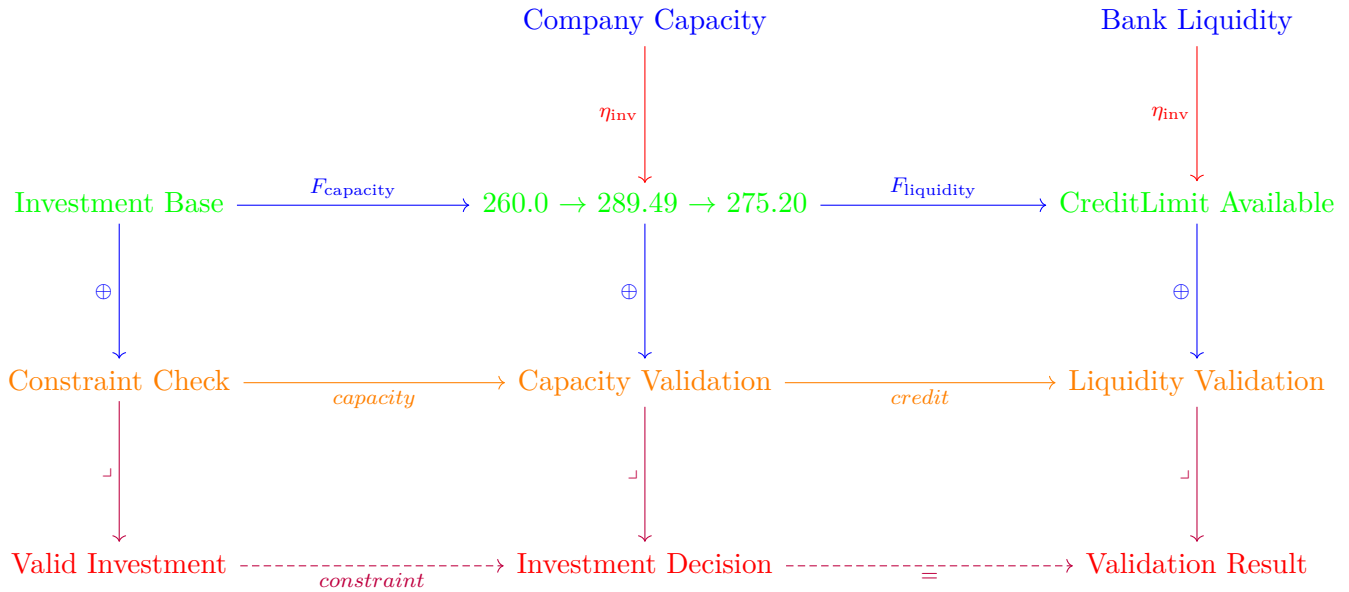


Figure 4.41: Commutative Diagram: Investment Validation Natural Transformation with pullback structure

The calculation taking place in Figure 4.41 in the commutative diagram of the natural transformation are the investment decisions and validation results. In the hierarchical commutative diagram 4.42 we can see the investment validation pullback corners, as another point of view on the compositions of the processes involved.

Economics: Credit Constraints and Financial Intermediation The investment validation pullback embodies the fundamental principle of credit constraint enforcement through categorical limits. A validation function implementing $i \leq c + \text{CreditLimit}$ becomes a typed morphism $\text{Pullback}_{\text{Investment}} : \mathbb{R}_{>0} \times \mathbb{R}_{\geq 0} \rightarrow \{0, 1\}$ that ensures investment decisions respect both company capacity constraints and bank liquidity limits before execution.

Economically, this captures the essential role of financial intermediation in validating economic feasibility of investment projects (Investment: 260.0 → 289.49 → 275.20) against available credit capacity and company

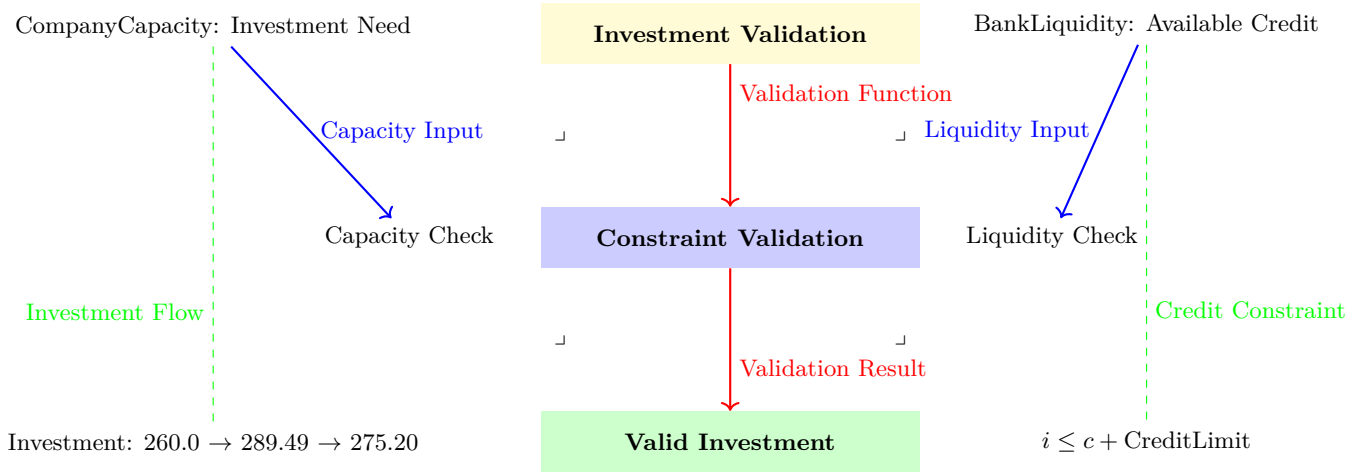


Figure 4.42: Commutative Diagram: Investment Validation Pullback Flow

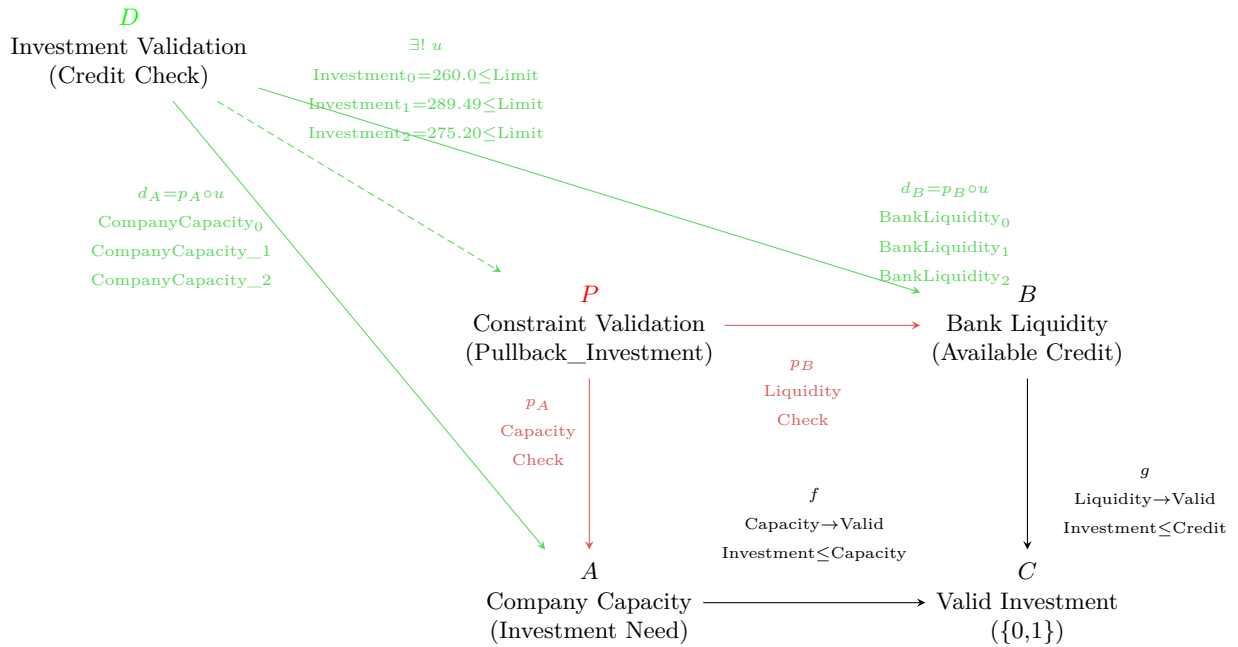


Figure 4.43: Investment Validation Pullback: The investment validation pullback ensures that investment decisions satisfy both company capacity constraints and bank liquidity limits across all periods.

financial capacity. The pullback ensures that investment validation respects both microeconomic level firms' constraints and macroeconomic banking system's constraints, preventing excessive leverage while enabling productive capital formation within resource aware realistic limits.

The three-layer architecture of diagram 4.42 reveals the categorical nature of financial constraint validation. The yellow layer represents investment validation requirements, the blue layer embodies the constraint validation mechanism. The green layer shows the resulting validation decisions. The pullback corners ensure that validation respects both company capacity constraints and bank liquidity constraints simultaneously. This categorical approach to credit constraint theory provides mathematical foundations for understanding financial intermediation as a categorical limit process, connecting banking theory with investment theory. It enables formal analysis of credit cycles and financial stability mechanism for the issues of Marx, Keynes or as in MoMaT, the desynchronized payments of the producer to suppliers now and only later back from clients, synchronised by a loan now and repayments later. The universal constructions shows this to be the macroeconomic accounting solution for the banking and monetary sector. The real accounts can be used to record the production and distribution of the common product, the GDP. Hence, the hierarchical risk-sharing solution in this paper for the labor sharing decentralized and parallel economies can be extended to also implement the GDP sharing in the product, initial and final objects of categories of real and nominal accounts. The macroeconomic accounting is the institutional answer to trace hierarchically risk from companies, to banks and ultimately to central banks and governments. The micro- and macroeconomic levels in the five agent MoMaT theory can be extended to include the central banks at the macroeconomic level and the banks at the then mesoeconomic level. Such a system includes gold based warehouse receipts as bank notes from former centuries, to paper money since 1970ies up to digital and crypto and quantum validated IDs of money being a means of loan repayments or dedebting in a validation process. This includes fractional and other financial contracts of macroeconomic accounting agents like central banks, interbanking markets or governments.

4.5.2 Pushouts for Economic Computation

Definition 4.14 (Accounts Update). Pushouts perform the actual economic calculations and aggregations. The account update pushout implements the 8 bookings.

$$\begin{array}{ccc}
 \text{Transaction} & \xrightarrow{g} & \text{Target} \\
 \downarrow f & & \downarrow q_2 \\
 \text{Source} & \xrightarrow{q_1} & \text{UpdatedAccounts}
 \end{array}$$

$$\text{Pushout}_{\text{Account}} : \quad \mathbb{R}_{\geq 0} \times \mathbb{R} \rightarrow \mathbb{R}_{\geq 0} \quad \text{Pushout}_{\text{Account}}(a, \Delta) = \max(0, a + \Delta) \quad (58)$$

$$(59)$$

where

$$\begin{aligned}
 f &: \text{Transaction} \rightarrow \text{Source} \\
 g &: \text{Transaction} \rightarrow \text{Target} \\
 q_1 &: \text{Source} \rightarrow \text{UpdatedAccounts} \\
 q_2 &: \text{Target} \rightarrow \text{UpdatedAccounts} \\
 \text{with constraint } \quad \Sigma_i \Delta_i &= 0 \text{ (conservation)}
 \end{aligned} \quad (60)$$

■

Account evolution through pushout computations:

$$\begin{aligned}
 \text{Period } 0 \rightarrow 1: \quad \text{AccResBank} &: 0.0 \rightarrow 208.0 \\
 \text{AccComBank} &: 0.0 \rightarrow 52.0 \\
 \text{AccComLoan} &: 0.0 \rightarrow 260.0
 \end{aligned}$$

4.5.3 Categorical Types of the Eight Macro Bookings

The categorical typing of the 8 macroeconomic bookings reveals how category theory disentangles the complex bidirectional flows between real and nominal accounts in the eight macroeconomic bookings of equations (3). Each booking is a morphism in the product category $\mathcal{C}_{\text{Real}} \times \mathcal{C}_{\text{Nominal}}$ where real flows and nominal flows move in opposite directions through categorical composition:

Booking	Economic Exchange	Agents	Real Flow Category	Nominal Flow Category
1	Lab sells Lab to Com	3	$Lab_{\mathcal{R}}^{Lab} \rightarrow Com_{\mathcal{R}}^{Lab}$	$Com_{\mathcal{N}}^{Bank} \rightarrow Lab_{\mathcal{N}}^{Bank}$
2	Lab buys Good from Com	3	$Com_{\mathcal{R}}^{Good} \rightarrow Lab_{\mathcal{R}}^{Good}$	$Lab_{\mathcal{N}}^{Bank} \rightarrow Com_{\mathcal{N}}^{Bank}$
3	Res sells Res to Com	3	$Res_{\mathcal{R}}^{Res} \rightarrow Com_{\mathcal{R}}^{Res}$	$Com_{\mathcal{N}}^{Bank} \rightarrow Res_{\mathcal{N}}^{Bank}$
4	Res buys Good from Com	3	$Com_{\mathcal{R}}^{Good} \rightarrow Res_{\mathcal{R}}^{Good}$	$Res_{\mathcal{N}}^{Bank} \rightarrow Com_{\mathcal{N}}^{Bank}$
5	Com gets Loan from Bank	2	$\emptyset$	$Bank_{\mathcal{N}}^{Loan} \leftrightarrow Com_{\mathcal{N}}^{Loan}$
6	Com pays Div to Cap	3	$\emptyset$	$Com_{\mathcal{N}}^{Bank} \rightarrow Cap_{\mathcal{N}}^{Bank}$
7	Com repays Loan to Bank	2	$\emptyset$	$Com_{\mathcal{N}}^{Bank} \rightarrow Bank_{\mathcal{N}}^{Loan}$
8	Cap buys Good from Com	3	$Com_{\mathcal{R}}^{Good} \rightarrow Cap_{\mathcal{R}}^{Good}$	$Cap_{\mathcal{N}}^{Bank} \rightarrow Com_{\mathcal{N}}^{Bank}$

Table 1: Categorical Types of the 8 Macroeconomic Bookings: Real vs Nominal Flow Categories

Categorical Commentary on Real/Nominal Disentanglement: The categorical approach resolves the typing complexity by recognizing that each booking operates simultaneously in two categories:

1. **Real Flow Category $\mathcal{C}_{\text{Real}}$:** Objects are real accounts (Lab, Res, Good), morphisms are physical transfers measured in units [h], [kg], [G]
2. **Nominal Flow Category $\mathcal{C}_{\text{Nominal}}$:** Objects are monetary accounts (Bank, Loan, Div), morphisms are monetary transfers measured in [EU]

The key categorical insight is that bookings 1-4,6,8 are contravariant functors $F : \mathcal{C}_{\text{Real}}^{op} \rightarrow \mathcal{C}_{\text{Nominal}}$ where real flows and nominal flows move in opposite directions, preserving the double-entry principle categorically. Bookings 5,7 are pure nominal morphisms representing financial contracts without real commodity exchange. The three-agent bookings (1,2,3,4,6,8) involve Bank as a mediating functor that ensures the categorical composition $Lab \leftrightarrow Com$ factors through $Lab \rightarrow Bank \rightarrow Com$, maintaining the categorical pullback property of banking intermediation. The two-agent bookings 5,7 represent morphisms in the loan contract category. This categorical typing eliminates the confusions of the recursive formulation of the eight bookings about the types of the bookings and the types of the variables in the bookings by properly recognizing that each booking is not a simple morphism but a natural transformation between functors that preserves both real and nominal categorical structures while ensuring conservation laws through categorical limits.

Following the categorical typing from equation 3, each booking is a typed morphism:

$$\begin{aligned}
\text{Booking}_1 : & \quad Lab_{\mathcal{R}}^{Lab} \times Com_{\mathcal{N}}^{Bank} \rightarrow Lab_{\mathcal{N}}^{Bank} \times Com_{\mathcal{R}}^{Lab} \\
\text{Booking}_2 : & \quad Lab_{\mathcal{N}}^{Bank} \times Com_{\mathcal{R}}^{Good} \rightarrow Lab_{\mathcal{R}}^{Good} \times Com_{\mathcal{N}}^{Bank} \\
\text{Booking}_3 : & \quad Res_{\mathcal{R}}^{Res} \times Com_{\mathcal{N}}^{Bank} \rightarrow Res_{\mathcal{N}}^{Bank} \times Com_{\mathcal{R}}^{Res} \\
\text{Booking}_4 : & \quad Res_{\mathcal{N}}^{Bank} \times Com_{\mathcal{R}}^{Good} \rightarrow Res_{\mathcal{R}}^{Good} \times Com_{\mathcal{N}}^{Bank} \\
\text{Booking}_5 : & \quad Bank_{\mathcal{N}}^{Loan} \rightarrow Com_{\mathcal{N}}^{Loan} \times Bank_{\mathcal{N}}^{Loan} \\
\text{Booking}_6 : & \quad Com_{\mathcal{N}}^{Bank} \rightarrow Cap_{\mathcal{N}}^{Div} \times Bank_{\mathcal{N}}^{Cap} \\
\text{Booking}_7 : & \quad Com_{\mathcal{N}}^{Bank} \times Bank_{\mathcal{N}}^{Loan} \rightarrow Com_{\mathcal{N}}^{Loan} \times Bank_{\mathcal{N}}^{Loan} \\
\text{Booking}_8 : & \quad Cap_{\mathcal{N}}^{Bank} \times Com_{\mathcal{R}}^{Good} \rightarrow Cap_{\mathcal{R}}^{Good} \times Com_{\mathcal{N}}^{Bank}
\end{aligned}$$

where $\mathcal{R}$ denotes real flow categories and $\mathcal{N}$ denotes nominal flow categories, with each morphism preserving the contravariant relationship between real and nominal flows as categorical functors.

Economic Interpretation of Categorical Booking Types: The categorical morphism types reveal the fundamental economic structure of market exchanges and financial intermediation. Bookings 1 and 3 represent factor market transactions where labor $Lab_{\mathcal{R}}^{Lab}$ and resources $Res_{\mathcal{R}}^{Res}$ flow to the company while monetary compensation flows in the opposite direction, implementing the economic principle of quid pro quo through contravariant functors. These flows are to be typed in a legal sense and modalities as discussed

in Menéndez and Winschel [2025] as the abstraction principle to disentangle into two opposingly flowing values of exchange in a quid pro quo game. Bookings 2, 4, and 8 represent product market transactions where goods Com_R^{Good} flow from company to consumers *Lab, Res, Cap* while payments flow back, completing the circular flow of the economy. The pure nominal bookings 5 and 7 capture the temporal dimension of financial intermediation, where Booking 5 creates future obligations Com_N^{Loan} in exchange for present liquidity, while Booking 7 extinguishes those obligations through repayment—these represent the banking sector’s core function of intertemporal coordination. Booking 6 implements profit distribution, transferring corporate earnings to capitalists through dividend payments, representing the return to risk-taking and capital provision, which we have not modelled as a tit for tat flow of values. The categorical typing thus captures the complete economic cycle: production factor acquisition (1,3), consumption fulfillment (2,4,8), intertemporal financial intermediation (5,7), and profit distribution (6), with each morphism preserving the fundamental economic law that real and monetary flows must move in opposite directions to maintain market equilibrium, quid pro quo games and double-entry accounting consistency as verified properties in the pullback. Combining the possibility to protocol the processes in categorical terms opens a way to rewind the debt processes during liquidation of banks. The restitution in German law is a contra variant functor for rewinding debt contracts of the agents intermediated by the bank. In the case the bank is to be resolved, agents debts are to be restituted and reconnected in a new way, without the bank as an intermediary in the restituted debt contracts.

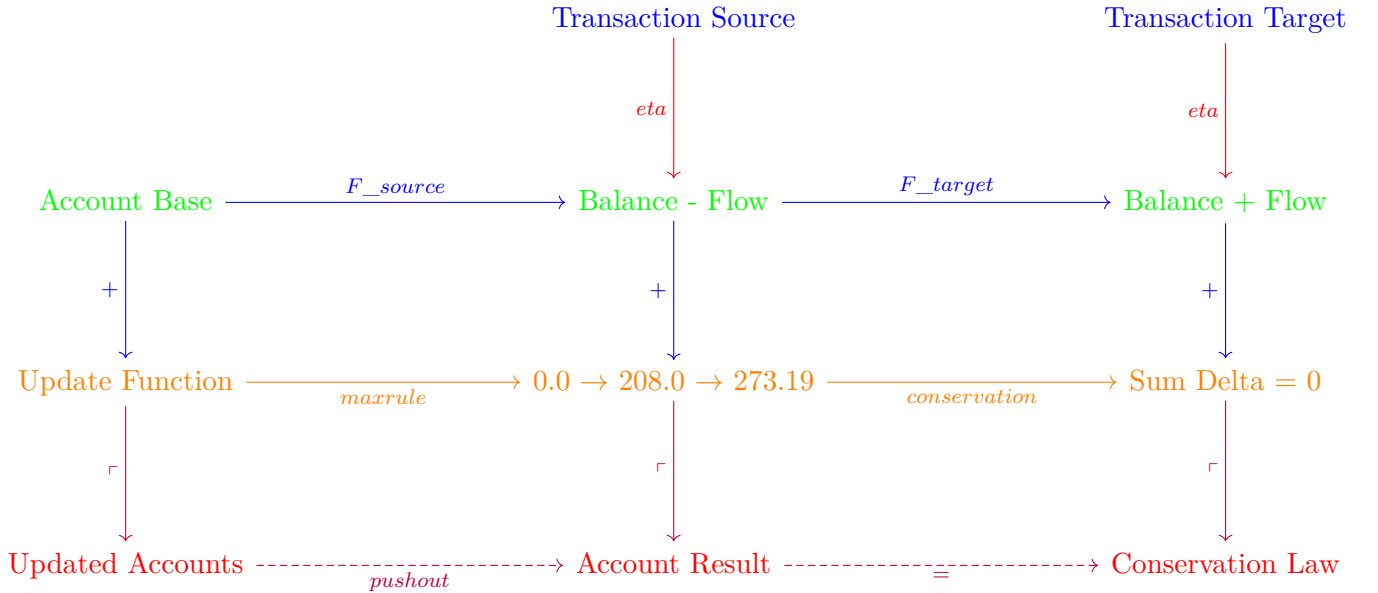


Figure 4.44: Commutative Diagram: Account Update Natural Transformation

In Figure 4.44, we see how money flows through the economy as a consistent compositionality of processes of bookings on accounts. When a transaction occurs, like when Labor provides services to a Company, two things happen simultaneously: The real flow moves from source to target - Labor’s services flow to the Company. At the same time, the nominal (money) flow moves in the opposite direction - from Company’s bank account to Labor’s bank account. The commutative diagram captures this dual movement through its categorical structure. The top layer shows the transaction participants - a source account (like Labor’s bank account) and a target account (like Company’s bank account). The middle layer implements the core economic principle that values cannot be created or destroyed - they can only move between accounts. This is enforced by the update function that ensures when money leaves one account it must appear in another.

The bottom layer shows the final updated account balances and verifies that the fundamental law of accounting is preserved: the sum of all changes must equal zero. The categorical machinery (through morphisms like η and F_{source}, F_{target}) ensures this rule to be mathematically guaranteed. For example, when Labor’s bank account grows from 0.0 to 208.0 to 273.19, we know with mathematical certainty that this money representing some value came from somewhere else in the system - it wasn’t created from nothing. The pushout structure forces all money flows to be properly balanced, making the double-entry accounting

principle a mathematical necessity.

This categorical view shows economic transactions as parallel synchronized flows of real goods and money that must always balance, governed by the mathematical laws of category theory rather than just accounting conventions. The legal modalities that appear in these contracts can be lifted to the institutional level as a deontic logic as in the polycentric governance and management structures in Ostrom [2005]. A categorical formulation of modal logic would add to the abstract syntax of the compiler and is a well researched path in programming language design. A DSL for the modalities of legal contracts would be operating on the memory structures that collects and represents in our categorical setup the legal contracts.

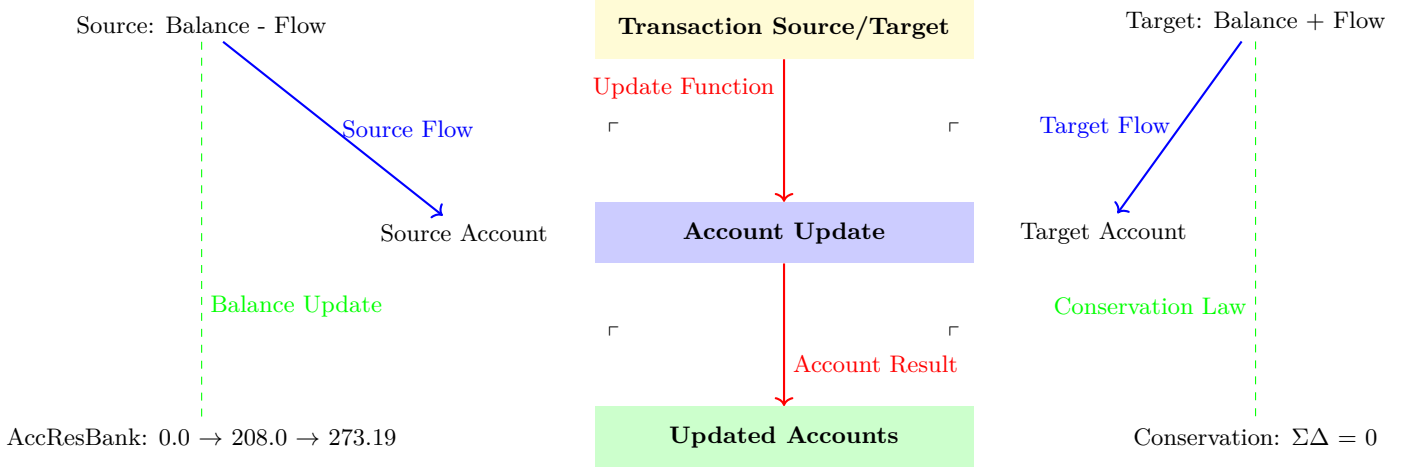


Figure 4.45: Commutative Diagram: Account Update Pushout Flow

This diagram 4.45 tells the economics of how money moves through the economy as the interaction of three levels: At the top level (yellow), we have two, a source and a target account, like when a company pays a worker's salary. The middle level (blue) shows the actual transaction happening - money leaves the company's account (source) and enters the worker's account (target). The bottom level (green) reveals the final result - we can track how the accounts changed (like the bank balance growing from 0 to 208 to 273.19), while making sure no money appears or disappears ($\sum \delta = 0$) by the conservation law. The mathematical machinery of pushouts (shown by the corner marks $\lceil$) ensures this as a mathematical guarantee of the program that money or values, i.e. the unit of account is conserved, like energy in physics, we enforce a conservation law. The arrows show how everything connects: yellow arrows track the transaction's progress, blue arrows show the money flows, and green dashed lines connect the initial balances to their final states. The categorical structure transforms banking from a set of rules into mathematically engineered consistent programs implementing economic universals.

In Figure 4.46 we see the account update pushout that implements the 8 macroeconomic bookings through proper double-entry accounting. The diagram shows a pushout as a universal construction that combines information from multiple sources. Objects are C (Transaction Source): The account that money flows from (e.g., Lab's bank account). B (Transaction Target): The account that money flows to (e.g., Com's bank account). A (Transaction Origin): The original financial flow/transaction. P (Account Update): The red pushout object that processes the update. D (Updated Accounts): The green final state with balanced books. Morphisms are f : Source Update: subtracts the flow from the source account (Balance - Flow). g : Target Update: adds the flow to the target account (Balance + Flow). p_A, p_B : Project the updates to the pushout object. u : The unique universal morphism ensuring mathematical consistency. The pushout structure provides three critical Economic Guarantees: Conservation: $\sum \delta = 0$ (total money in the system remains constant), Non-negativity: $\max(0, \text{account} + \Delta)$ (accounts cannot go below zero), Double-entry consistency: Every debit has a corresponding credit. The Concrete Example shows actual values: Res_N^{Bank} : $0.0 \rightarrow 208.0 \rightarrow 273.19$ (Resource owner's bank account grows), Com_N^{Bank} : $0.0 \rightarrow 52.0 \rightarrow 190.50$ (Company's bank account grows), Com_N^{Loan} : $0.0 \rightarrow 260.0 \rightarrow 523.49$ (Company's loan liability grows).

The categorical approach ensures that all 8 bookings respect these mathematical constraints. The pushout's universal property guarantees that there exists a unique way (morphism u) to update accounts that satisfies all constraints simultaneously, making errors impossible by construction rather than requiring manual checking. This is the mathematical foundation that makes the categorical formulation more expressive, robust and compositionally extendable and maintainable than the recursive computational for-

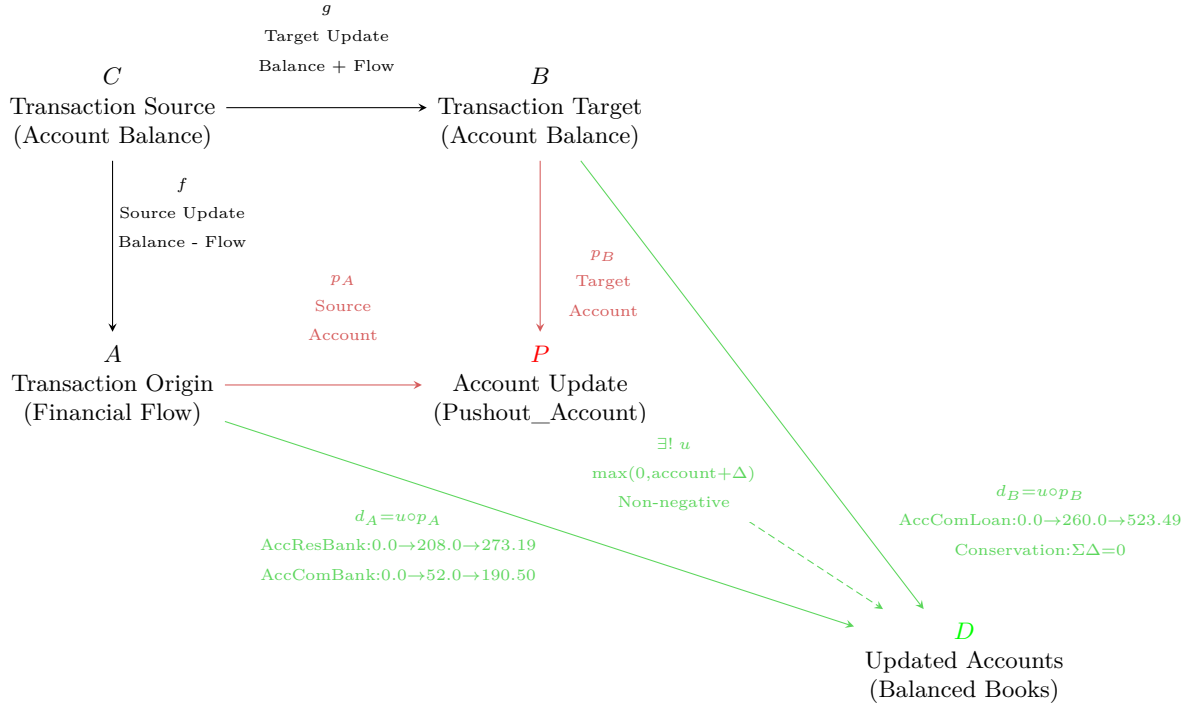


Figure 4.46: Account Update Pushout: The account update pushout implements the 8 bookings through proper double-entry accounting, ensuring conservation of the unit of account ($\sum \delta = 0$) and non-negative account balances.

malisation only.

Economics: Double-Entry Accounting and Money Conservation The account update pushout embodies the fundamental principle of double-entry accounting through categorical colimits. The account update function $\max(0, a + \Delta)$ becomes a typed morphism $\text{Pushout}_{\text{Account}} : \mathbb{R}_{\geq 0} \times \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$ that transforms transaction flows into account balance updates while preserving the essential conservation law $\sum_i \delta_i = 0$ that ensures units of account are neither created nor destroyed.

Economically, this captures the fundamental accounting identity that underlies all monetary transactions: every debit has a corresponding credit, ensuring that economic value flows are conserved across the entire system. The pushout structure guarantees that individual account updates ($\text{AccResBank}: 0.0 \rightarrow 208.0 \rightarrow 273.19$) aggregate coherently into system-wide conservation, maintaining macroeconomic consistency as a macroeconomic foundation of microeconomic transactions.

The three-layer architecture of diagram 4.45 reveals the categorical nature of accounting theory: the yellow layer represents transaction sources and targets, the blue layer embodies the account update mechanism, and the green layer shows the resulting updated account balances. The pushout corners mark where individual transaction morphisms compose into the universal account update morphism. This categorical approach to accounting theory provides mathematical foundations for understanding monetary systems as categorical structures that preserve conservation laws, connecting accounting theory with category theory and enabling formal analysis of monetary consistency in complex economic systems.

4.6 Categorical Evolution

The categorical state transition integrates all universal constructions. This is by stating that the model closes or that we have an endofunctor that is a monad. We can trace the mathematical structure of the theory through the state transition functor. Side effects as they are called in computer science are properly internalised or endogeneized as economists say. The endofunctor implements the iteration of the categorical simulation in endo loops of the universal pullbacks of verification and pushouts of computation. The result is a mathematically verified program and evolution of the system.

The monad is what is missing in the categorically typed recursive program in the Appendix A.2.2. The natural transformation helper functions `add_component!` and `add_transformation!` are implemented as a function definition but are not used for running and implementing the simulation. The categorical program is finally a program where evolution is a natural transformation.

Definition 4.15 (Categorical Evolution). The categorical evolution **endofunctor** $T : \mathcal{C}_t \rightarrow \mathcal{C}_{t+1}$ preserves all categorical structures:

$$T(\text{period}) = \text{Pullback}_{\text{Validation}} \circ \text{Pushout}_{\text{Computation}} \circ \text{Functor}_{\text{Transformation}}(\text{period}) \quad (61)$$

$$T : \mathcal{C}_{\text{Economy}} \rightarrow \mathcal{C}_{\text{Economy}} \quad (62)$$

$$T : (\mathcal{C}_{\text{acc}} \times \mathcal{C}_{\text{flow}} \times \mathcal{C}_{\text{par}}) \rightarrow (\mathcal{C}_{\text{acc}} \times \mathcal{C}_{\text{flow}} \times \mathcal{C}_{\text{par}}) \quad T(A, F, P) = (A', F', P') \quad (63)$$

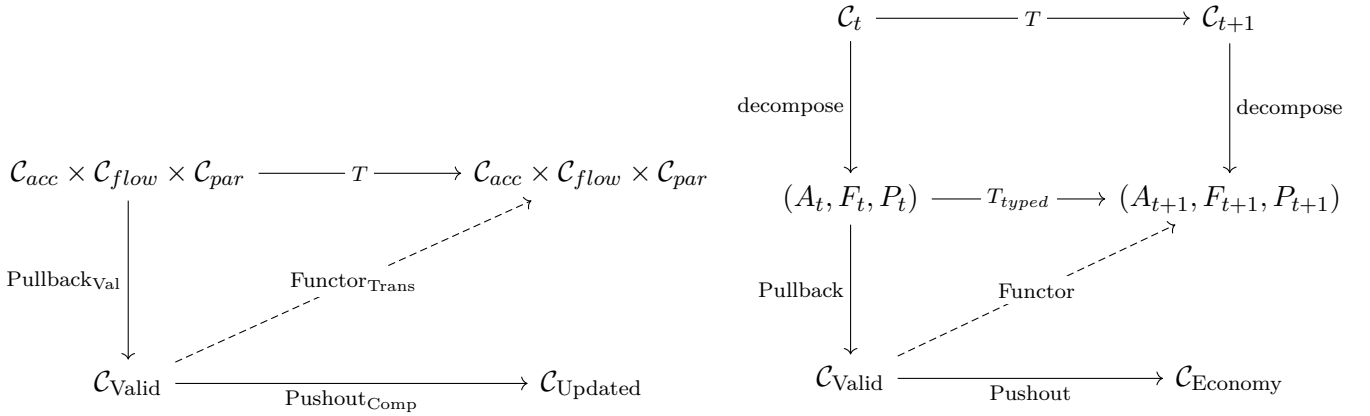
where

$$A' = \text{Pushout}_{\text{acc}}(\text{Pullback}_{\text{Validation}}(A, F, P))$$

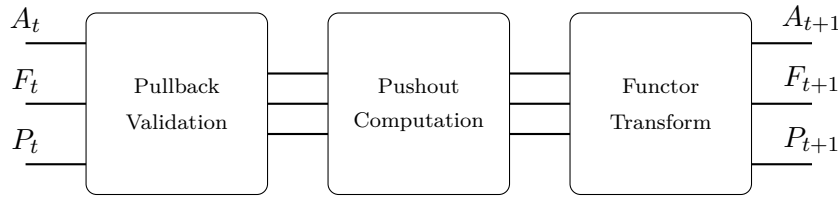
$$F' = \text{Functor}_{\text{flows}}(A', P)$$

$$P' = \text{NatTrans}_{\text{Evolution}}(P)$$

The commutative diagrams shows the categorical state transition as the temporal evolution of the system.



The composition of operations of the endofunctor of state transition is shown in the following string diagram:



■

The complete economic evolution as a commutative diagram 4.47 is showing the parallel operations as needed for typing the eight bookings of the macroeconomic accounting systems. The diagram illustrates the categorical structure of economic evolution through three interconnected layers. At the highest level, the Parameters Layer contains the fundamental economic ratios that govern behavior - consumption ratios and scale production factors. These parameters are transformed through natural transformations (η) to influence the Flows Layer, where actual economic activities like demand and production materialize. The flows represent the economic transactions, showing how demand grows from 0 to 166.4 to 272.63 units while production fluctuates from 31.17 to 1.0 to 3.20. Finally, these flows manifest in the concrete Account Layer through functorial transformations, tracking how Labor accounts grow from 0 to 52.0 units and Company accounts expand from 0 to 190.50 units. The categorical machinery of functors (F_{params} , F_{flows}) and natural transformations ensures consistency between layers, while tensor products and pushouts capture the parallel evolution and aggregation of economic quantities. This categorification reveals the mathematical structures underlying economic transformations - from abstract parameters to concrete account balances - while preserving the essential relationships between different aspects of the economy.

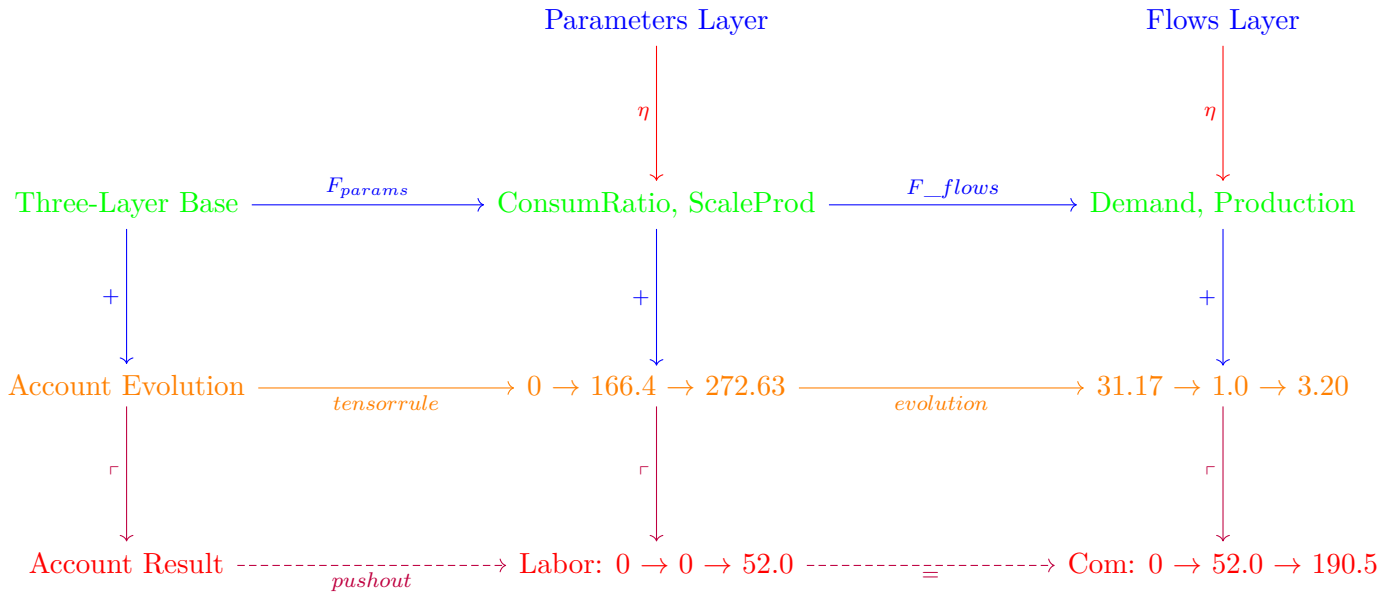


Figure 4.47: Commutative Diagram: Three-Layer Economic Evolution Natural Transformation

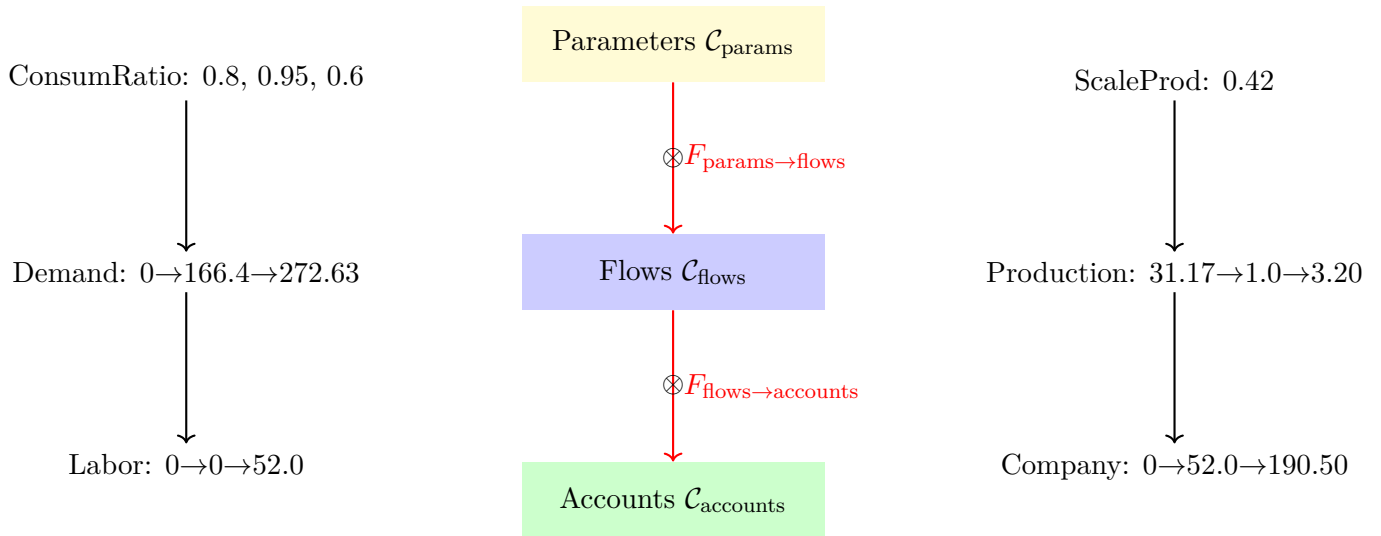
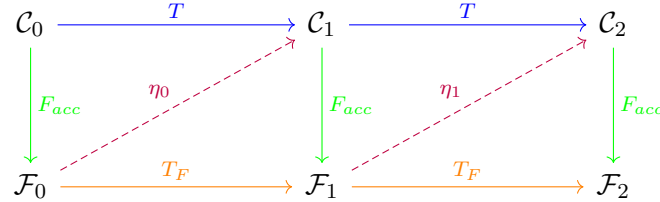


Figure 4.48: String Diagram: Parameters $\otimes$ Flows $\otimes$ Accounts Evolution

The diagram 4.48 illustrates the three-layer categorical structure through a tensor product perspective. The Parameters layer ($\mathcal{C}_{\text{params}}$) contains the fundamental economic parameters - ConsumRatio evolving through values 0.8, 0.95, 0.6 and ScaleProd fixed at 0.42. These parameters are transformed through the functor $F_{\text{params} \rightarrow \text{flows}}$ into the Flows layer ($\mathcal{C}_{\text{flows}}$), where economic flows materialize as Demand (growing from 0 to 166.4 to 272.63) and Production (changing from 31.17 to 1.0 to 3.20). Finally, through $F_{\text{flows} \rightarrow \text{accouts}}$, these flows manifest in the Accounts layer ($\mathcal{C}_{\text{accouts}}$) as concrete account balances - Labor accounts evolving from 0 to 0 to 52.0 and Company accounts from 0 to 52.0 to 190.50. The tensor products ($\otimes$) indicate parallel operations occurring simultaneously at each layer, capturing the concurrent nature of economic transformations. This categorical structure ensures that changes in parameters propagate consistently through flows to account balances while maintaining parallel processing capabilities.



Period Evolution Values:

Company Bank: 0.0 $\rightarrow$ 52.0 $\rightarrow$ 190.50

Labor Bank: 0.0 $\rightarrow$ 0.0 $\rightarrow$ 52.0

Resource Bank: 0.0 $\rightarrow$ 208.0 $\rightarrow$ 273.19

Investment: 260.0 $\rightarrow$ 289.49 $\rightarrow$ 275.20

Demand: 0.0 $\rightarrow$ 166.4 $\rightarrow$ 272.63

Figure 4.49: Commutative Diagram: Period Evolution Natural Transformation

The diagram 4.49 tells the economic story through categorical structures. The evolution occurs across three time periods, represented by categories C_0 , C_1 , and C_2 in the top row. These categories capture the complete economic state at each moment. Below them, categories F_0 , F_1 , and F_2 represent the actual flows of money and goods between economic agents. The blue arrows T show how the economy moves forward in time, while orange arrows T_F track how flows evolve. The green vertical arrows F_{acc} are functors that translate flows into account balance changes. Most importantly, the purple dashed arrows η are natural transformations that ensure consistency between different ways of viewing the same economic evolution - whether through direct state changes or through accumulated flows. We can see this evolution quantitatively in the account values: The Company Bank account grows substantially from 0 to 190.50 units as business activity increases. The Labor Bank shows an interesting pattern - staying at 0 for two periods before jumping to 52 units, reflecting delayed wage payments. Meanwhile, Resource Bank builds up reserves from 0 to 273.19 units, and Investment fluctuates around 260-290 units while Demand steadily grows from 0 to 272.63 units.

The diagram in Figure 4.51 describes the evolution in time by categorical constructions. At any moment, the economy has two main aspects: the accounts that show what everyone owns (represented in category $\mathcal{C}_{\text{accounts}}$), and the flows that show how money and goods move between agents (captured in $\mathcal{C}_{\text{flows}}$). As time passes from period t to $t + 1$, both accounts and flows change - wages get paid, goods are bought and sold, and investments are made. These changes don't happen independently - when money flows from one account to another, both accounts must update consistently. Pushouts verify that all changes fit together properly. The final result is a single mathematical object - an endofunctor T - that takes the entire economic state and transforms it into the next period's state while preserving all the important relationships between accounts and flows. This mathematical framework helps us understand how individual economic actions combine to create the overall evolution of the economy as a macroeconomic foundation of microeconomic transactions.

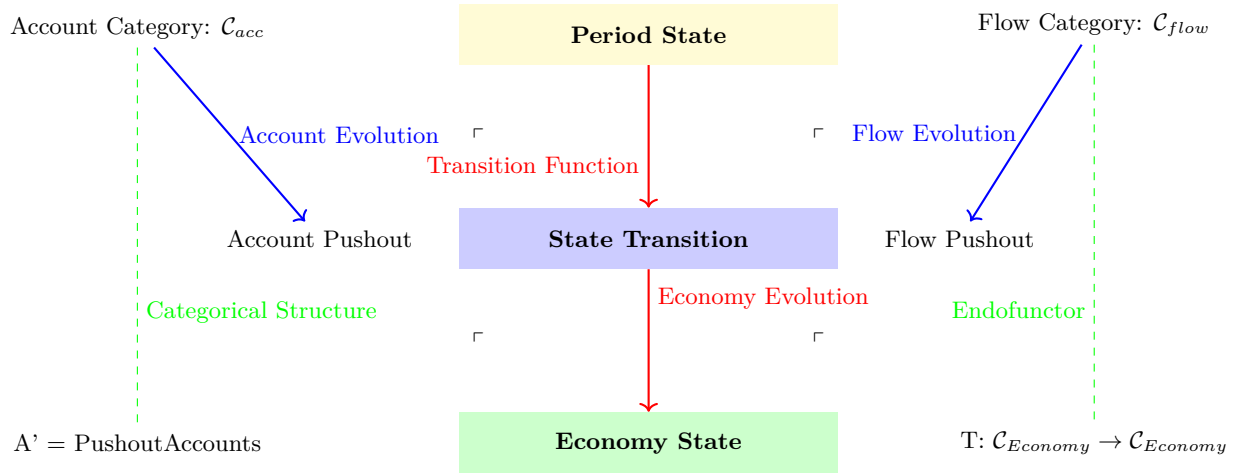


Figure 4.50: Commutative Diagram: Categorical State Transition Pushout Flow

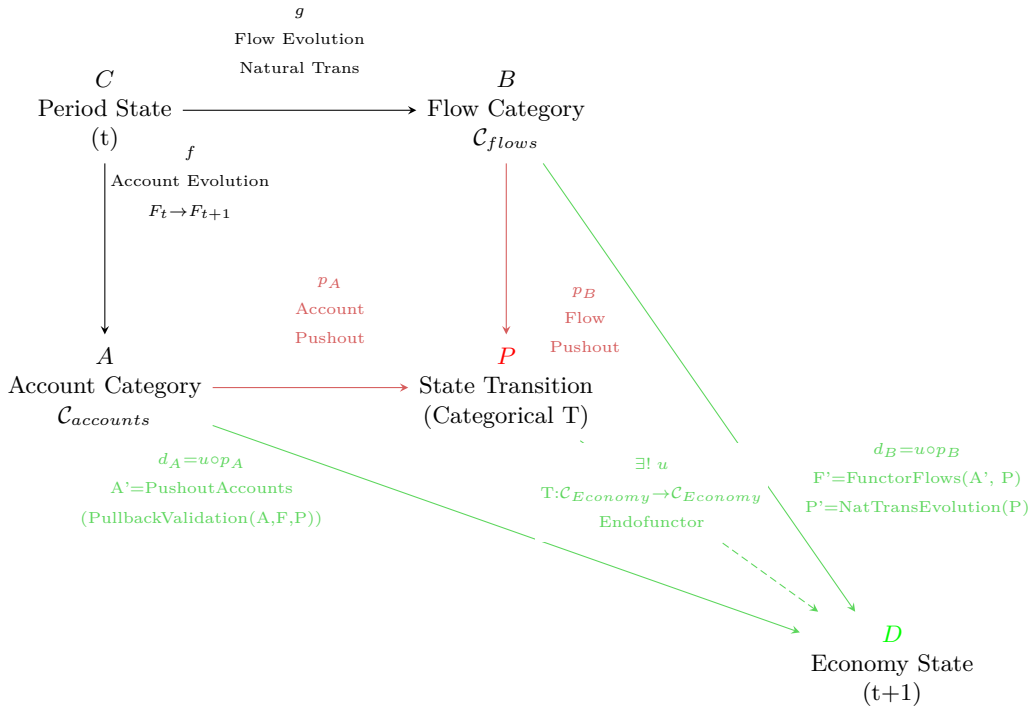


Figure 4.51: Categorical State Transition Pushout

Economics: Complete Economic Evolution and Systemic Integration: The categorical state transition pushout represents the ultimate synthesis of all economic mechanisms through categorical colimits. The endofunctor

$$T : \mathcal{C}_{Economy} \rightarrow \mathcal{C}_{Economy} \quad (64)$$

integrates pullback validation, pushout computation, and functor transformation into a single categorical morphism that preserves all economic structures while enabling systemic evolution.

Economically, this captures the complete economic evolution process where individual categorical structures (Account Category: $\mathcal{C}_{acc}$, Flow Category: $\mathcal{C}_{flow}$) compose through state transitions to generate coherent economic development. The pushout ensures that economic evolution respects both microeconomic level constraints and macroeconomic consistency, creating new economic states ($A' = \text{PushoutAccounts}$) that emerge naturally from previous states while maintaining categorical properties. The three-layer architecture of diagram 4.50 reveals the categorical nature of complete economic systems: the yellow layer represents current period's economic state, the blue layer embodies the state transition mechanism, and the green layer shows the evolved economy state. The pushout corners mark where individual categorical structures compose into the universal economic evolution morphism. This categorical approach to economic system theory provides mathematical foundations for understanding economic development as a categorical process, connecting all economic theory through categorical mathematics and enabling formal analysis of economic systems as evolving categorical structures that preserve essential economic relationships while adapting to changing conditions and maintaining systemic coherence. The complete set of economic variables across periods 0, 1, 2 is in Table 2.

Variable	$t = 0$	$t = 1$	$t = 2$	Variable	$t = 0$	$t = 1$	$t = 2$
ConsumRes	0.0	166.4	218.55	AccLabBank	0.0	0.0	52.0
ConsumLab	0.0	0.0	49.4	AccLabLab	0.0	100.0	195.67
ConsumCap	0.0	0.0	4.68	AccLabGood	0.0	0.0	0.0
Demand	0.0	166.4	272.63	AccResBank	0.0	208.0	273.19
DemandPlan	0.0	117.0	247.27	AccResRes	0.0	91.68	182.42
DemandSurplus	0.0	49.4	25.36	AccResGood	0.0	0.0	1.17
GoodProduction	31.17	1.0	3.20	AccCapBank	0.0	0.0	7.8
GoodPrice	30.0	141.7	89.94	AccCapDiv	0.0	7.8	41.57
Investment	260.0	289.49	275.20	AccCapGood	0.0	91.68	182.42
InvestmentRes	208.0	231.59	220.16	AccComBank	0.0	52.0	190.50
InvestmentLab	52.0	57.90	55.04	AccComLoan	0.0	260.0	523.49
Repayment	26.0	28.95	27.52	AccComDiv	0.0	7.8	41.57
WagesPayment	0.0	52.0	109.90	AccComRes	20.0	8.32	9.26
RepaysPayment	0.0	26.0	54.95	AccComLab	110.0	0.0	4.33
Diff	52.0	138.50	121.25	AccComGood	0.0	31.17	30.99
DividendDecision	7.8	41.57	94.39	AccBankComLoan	0.0	260.0	523.49
DividendPayment	0.0	7.8	41.57	AccBankComBank	0.0	52.0	190.50
				AccBankLabBank	0.0	0.0	52.0
				AccBankResBank	0.0	208.0	273.19
				AccBankCapBank	0.0	0.0	7.8

Table 2: Complete Economic Variable Evolution for Periods 0, 1, 2

5 Conclusion

The main economic result of this paper is the construction of a stable monetary accounting system. Money primarily serves as a means to measure and repay debts and liabilities by transferring ownership of physical tokens like paper bills and coins, or references to them such as checking account balances representing defined units of account. These monetary instruments are necessary not only for loan contracts but also for any exchange contracts where goods are traded for money, tit for tat, with the debt being denominated in terms of both the good and the money transferred to fulfill the contract.

Our MoMaT demonstrates that monetary systems are essential for economies organised by a division of labour. The hierarchical nature of these systems provides a risk-sharing technology: risk originates at the microeconomic company level and is ultimately socialised at the central bank level through the banking sector. However, a critical missing piece has been an accounting consistency technology at the

macroeconomic level that can integrate consistent microeconomic double-entry accounting systems. This paper’s core contribution is accordingly providing the mathematical tools for these macroeconomic systems.

The ultimate goal of sharing in economies and societies concerns GDP distribution, beginning with the organisation of labour sharing in modern societies. Effective economic policy and distributional theory require a deep understanding of the underlying processes. This has been the primary aim of our paper, which we hope to have achieved to some extent. We believe this work represents important progress toward enabling the economics profession to better address these fundamental societal challenges.

A more abstract perspective on monetary systems in modern economies reveals that hierarchical levels form an institutional framework designed to synchronise payment flows. This synchronisation is crucial because input providers, who are risk-averse, must be paid before companies receive payments from consumers of their products. This timing challenge parallels the modern biological understanding of cognitive systems described in Ehresmann and Vanbreemersch [2007], where hierarchical synchronisation of time scales occurs across different organisational levels. For instance, top-level corporate management must operate with much longer planning horizons compared to assembly line workers or middle management. Our categorical approach to economic modelling, both at the foundational accounting level and through strategic game theory in open games in Ghani et al. [2018], is particularly well-suited for analysing decentralised control and management of private and common pool resources, as explored in polycentric management by Elinor Ostrom [2005]. We demonstrate how category theory provides rigorous mathematical tools for economic modelling and simulation programming. Our categorical formulation of macroeconomic accounting shows that complex economic phenomena - from local individual decisions to global macroeconomic systemic outcomes - can be effectively understood through categories, functors, and natural transformations in universal constructions.

We have shown in our MoMaT that eight fundamental macroeconomic bookings naturally arise as typed morphisms in product categories $\mathcal{C}_{\text{Real}} \times \mathcal{C}_{\text{Nominal}}$, revealing the deep categorical structure underlying double-entry bookkeeping. The contravariant relationship between real and nominal flows emerges not as an accounting convention, but as a mathematical necessity encoded in the categorical structure itself. String diagrams provide visualisations of economic evolution, offering economists a new language for understanding temporal dynamics through functors and natural transformations.

The categorical approach delivers concrete computational benefits: pullbacks ensure constraint validation before transactions occur, pushouts guarantee consistency in state transitions, and natural transformations preserve economic relationships across time periods. Unlike the recursive formulation, our categorical approach provides type safety for all economic variables and universal property guarantees for all calculations.

The categorical perspective reveals previously hidden economic structures. Investment decisions emerge as universal constructions with functors transforming demand surplus into capital allocation, while market equilibration appears as pullback constructions ensuring supply-demand consistency. The three-layer categorical structure (Parameters $\rightarrow$ Flows $\rightarrow$ Accounts) shows how abstract economic behavioural specifications materialize into concrete account balances through functorial transformations, providing a mathematical foundation for understanding the propagation and communication of economic decisions through hierarchical layers.

This work establishes category theory as more than a mathematical abstraction in economics. It becomes a practical tool for economic analysis and policy design, gives economic theories a formal semantics expressible in a domain specific language (DSL) for economics as calculations and information processing of double-entry bookkeeping programs. Our approach suggests that many classical economic problems, from stability analysis to welfare optimisation, might benefit from categorifications which amounts to a typing discipline for units of accounts. The framework opens new research directions in computational economics, monetary theory, and macroeconomic modelling.

While our theory captures key macroeconomic dynamics, it currently focuses on a simplified five-sector-agent economy with a bank as the banking sector which is usually about risk management. However, extensions with central banks are now compositional and for consistency and decision structures it is clear how to integrate them into the categorical framework. Extensions to incorporate heterogeneous agents, complex financial instruments, and international trade represent important avenues for future research. Additionally, the relationship between our categorical constructions and established economic theories requires deeper investigations by using the language of categories to formulate them and check for consistencies and inconsistencies of different points of view.

Categories, functors and natural transformations, as shown by Lawvere, categorify the concepts of theory and models with homomorphic consistency in Tarski’s logic, ready to use for the epistemology of *econom(etr)ics*. Categorical universals generalise maximisation in economics and econometrics which is

useful to unite operational and institutional dynamics. A monad is an endofunctor as well and formalises side effects in programs which is useful to formally capture institutional dynamics versus operational dynamics within the institutional setup. Category theory's anecdotal *abstract nonsense* does make sense when syntax and semantics are designed for domain-specific programming languages (DSL) and their compilers. In economic systems syntax and semantics have to be separated to type the observing and observed agents in the structural econometrics that emerged after the Lucas Critique. Custom units of account and their creation are definable by functors and natural transformations. The national accounting system is extendable compositionally to also include central banks and governments in institutional hierarchies. New agents can be added with accounts, hierarchies, bookings and consistency conditions to design, program and control monetary institutional contractual evolutions. An earlier categorically programmed double-entry system can be composed with the macroeconomic accounting of this paper by suitably specified type theories to industrially scalable programs for managing many agents' accounts, decisions, hierarchies of strategic game theoretic, econometric or multi-agent systems as digital twins of holdings, platforms, banks, crypto currencies, stable coins, central banks or at the top level of monetary economics as exchange rate regime dynamics booked in the bank of central banks.

Leibniz might have probably said *calculemus*, which is Latin for *let us calculate*. When he used this term, he was not only referring to mathematical calculations in the modern sense. He was expressing a much broader vision for resolving all forms of human disputes. In our case major economic disputes are about the distribution of the GDP, ever since Marx's *struggle of the classes*. Leibniz wished that when people had a disagreement on philosophical, legal, political, or scientific questions they could sit down, take out their pens, and say to each other, *Calculemus!* and then work through the problem logically to see who was right. In essence, *calculemus* called for:

- Objectivity: It was his belief that logic and reason, if applied systematically, could transcend subjective opinions and emotional arguments.
- Automation: It was a precursor to modern ideas of computation and artificial intelligence, where complex reasoning could be automated through formal, rule-based processes.
- Notation: It was a testament to his deep conviction that the right notation (like the one he developed for calculus) could make complex ideas clear and tractable.

In this sense, we try to lay out an approach to *calculemus* for economics. Who owes what of the common product, the GDP, to whom in the production process of an economy organised by a division of labour and hierarchical risk sharing.

A Appendix

A.1 Category Theory

We use in this paper the three fundamental categorical data structures of category, functor and natural transformation. Categories contain objects and morphisms, like a graph with vertices and edges, while the morphisms in a category are closed under composition. This means that a structure to qualify as a category, for all morphisms which are composable, the composed morphism must be part of the category as well. In short, categories are structures where composable morphisms are composed. It is the mathematics of composable structures and thus perfect for the study of complex systems, like mathematics itself. Examples are formed by the category of Sets as objects with morphisms being functions - linear or continuous or relations - between them. Other categories contain topological spaces, dynamical systems, strategic games and other subjects that can be made into a category with suitable definitions of what morphisms and their compositions are or should be for some modelling purpose.

Category theory allows to switch between an original category with morphisms between objects to a meta category with the original morphisms being objects and the morphisms are then morphisms between the morphisms of the original category. Hence, we can switch from an *object oriented* view to a *processes oriented* view of the structures being analysed. Categories are for consistently bookkeeping morphisms and meta morphisms in mathematics. We have made them available for macroeconomic bookkeeping as well. Another view is that category theory is the grammar of mathematics and accordingly in our macroeconomic accounting system categories seem to be useful as the grammar of economies and economics.

We have shown that exchanges of goods and money are naturally expressed in categories. The resulting morphisms between morphisms bind opposing processes in value transfers in economies subject to the legal principles of *tit for tat* or *quid pro quo*, for example underlying the restitution claims in paragraph §812 of the BGB (German Civil Code), see more details in Menéndez and Winschel [2025].

Category theory provides an extension of set based mathematics where modelling takes place in terms of custom categories. Functors can be used to map them for example into the standard category of sets for a usual mathematical representation. But not only algebra is inherently categorical but also geometry and logic, see Abramsky et al. [2014] for a survey. The economic models are typed, decomposed and recomposed in a compositional and modular way and equipped with a programming language for economics. They help in understanding, programming, simulating or learning parameters and patterns from data. Modern software architectures and compiler technology can not be thought without category theory nowadays and accordingly our paper is a step to unite computer science and economics for designing and implementing the informational structures of digital twins of complex real world systems.

A topos, a top level categorical construction, has an internal language, which is a formal language that is interpreted within the topos itself. This is often an intuitionistic higher-order logic. From a computer scientific view category theory is useful in designing different semantics of functional and domain specific (DSL) programming languages, see Abramsky et al. [2014]. This concept is a powerful idea that allows one to treat a topos as a universe where one can do mathematics and define new objects and relationships. The *internal* nature means that the language's syntax and semantics are completely defined by the categorical structure of the topos, rather than being imposed from the outside. In this sense this paper is ultimately aiming to the define the topos of economics. The invariance based approach to complex systems mimicks the *Erlangen program* of Felix Klein to study geometries by their invariants. Similarly the *Emmy Noether program* is to understand symmetries and invariants of physical systems by understanding what is conserved during transformations. In economics values are conserved during bookings. Accordingly, a value based money theory, as is often attempted in order to understand money, is from our point of view doomed to fail.

Using categories as states with functors as generalised functions and natural transformations as consistency conditions and equations over composed functions and universals as adjunctions for expressing optimality is a rather general and powerful approach to modelling. The power comes from using categories for keeping consistency in local to global structures and evolutionary transitions over time. This is where category theory shines and which is the core of using them in macroeconomic accounting as an example of a complex system with a whole and local parts. It is a language for describing and reasoning about complex systems and their interactions.

Functors map objects and morphisms of one category to another category preserving the identities and the compositional structure. Natural transformations are maps between functors, also preserving the identities and the compositional structure. We use category theory as the mathematics of function composition. The usual elements of sets $x \in X$ can be represented as a function from a singleton set like $\{*\}$ to X picking the element $x \in X$. Function application $f(x)$ then becomes function composition $f \circ x : \{*\} \rightarrow X$, making the usual set and element based mathematics into a special case of a categorical morphism based mathematics. Categories can also be conceived as the module system of mathematics being able to represent algebraic, geometric and logical structures and how they morph into each other.

For economics another aspect seems to be most powerful - namely that category theory provides a calculus for morphisms where systems of equations are solved for functions, processes, morphisms or relations and not only numbers. This high-levelness provides much expressivity for economic concepts which are often more appropriately described by processes and not by objects, much in line with the ideas usually attributed to Heraklit - the world is a flux of change.

Pullbacks and pushouts are the tools we use for verifying consistency conditions and composing calculations, respectively. They are examples of limits and (dual) colimits which are universal constructions. The universality can represent uniqueness, optimality, best, biggest or similar concepts. For economics they massively generalise economic principles of optimality, see Abramsky et al. [2015]. The categorical machinery comes with formal and visual 2-dimensional mathematical tools like string and commutative diagrams, see Selinger [2009] and Selinger [2011] where they are used in a categorical reformulation of physics. The results are now used in compilers for quantum computers, for example at the company Quantinuum, based on Bob Coecke's and Samson Abramsky's original work in Abramsky and Coecke [2004] or its results later in Coecke and Kissinger [2017]. This work has been a direct inspiration for developing open games for economic applications in Ghani et al. [2018] and this paper's tools.

Diagrams can be used to visually represent systems of equations of functions. String diagrams are in a

way the process oriented visualisations of functions while commutative diagrams are visual representations of equations. Commuting diagrams show paths of sequentially composed morphisms as arrows. Their commuting property is that different paths and by that composed functions must be equal. String diagrams can do the same but turn morphisms into the object of interest by boxing them while the domain and codomains (types) are taken to be the wires. Usual pictures of functions $\boxed{A} \xrightarrow{f} \boxed{B}$ are an object oriented view while string diagrams $\xrightarrow{A} \boxed{f} \xrightarrow{B}$ focus on the morphisms as a process oriented view. Function composition in formulas is denoted by a circle in $f \circ p_A$ and reads as *apply f after p_A*. Equality on paths is then written as $f \circ p_A = g \circ p_B$ which is what commutative diagrams represent in a visual way. For some people equations are more intuitive while others prefer visual representations in diagrams.

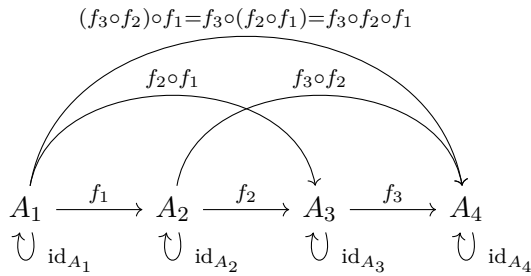
We now define formulas, commuting and string diagrams of categories, functors, natural transformations, pullbacks and pushouts in their abstract form which we apply to macroeconomics in the main text.

A.1.1 Category, Functor, Natural Transformation

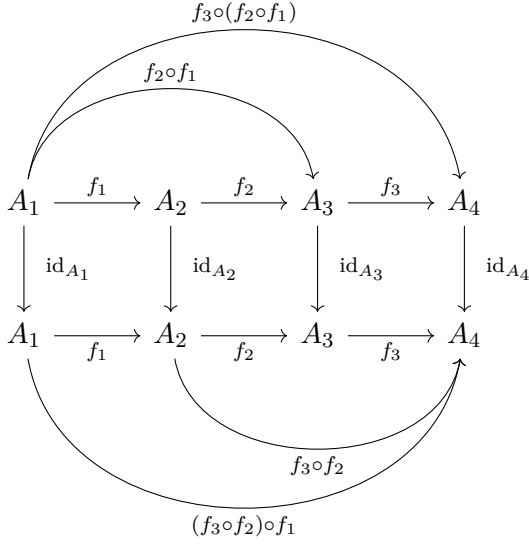
A category contains objects and morphisms being functions or relations between them. A usual set is a category with objects without any morphisms between them. By that we can add relations and morphisms between objects and take these structures into account which allows us to model emergent effects of geometric and macroeconomic wholenesses. These concepts are modelable in a very elegant way in categories, functors and universals as optimal decisions in spaces where morphisms and relations between objects are to be composed. The important point of category theory is that properties of objects are not defined by their internal structure which one needs to look up within the set. Instead mathematical properties are defined by their behaviour at their interface when being used by their context in operations. Mathematics becomes by that a kind of social science of relational behaviours at interfaces of systems. These features make categories into attractive representations of states, relations or structures in a social science like economics.

Definition A.1 (Category). A category $\mathcal{A}$ in the category of categories $\mathcal{A} \in \mathcal{C}$ has the following structure.

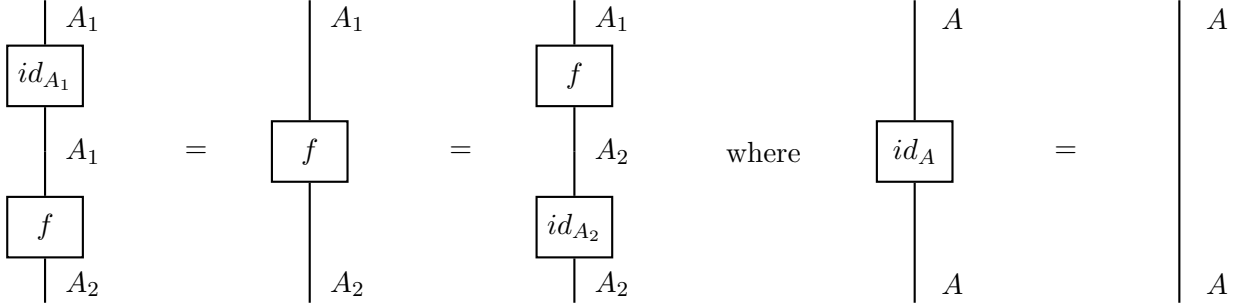
- Objects Ob : objects of $\mathcal{A}$ have a unique identity morphism, $\forall A \in \text{Ob}(\mathcal{A}), \exists ! \text{id}_A \in \text{Hom}(\mathcal{A}) :: A \rightarrow A$
- Morphisms Hom : morphisms of $\mathcal{A}$ from objects to objects, $\forall f \in \text{Hom}(\mathcal{A}) :: A_s \rightarrow A_t, A_s, A_t \in \text{Ob}(\mathcal{A})$
- Composition Hom : that are composable, $\forall f_1, f_2, f_3, \circ :: \text{Hom}(A_2, A_3) \times \text{Hom}(A_1, A_2) \rightarrow \text{Hom}(A_1, A_3)$
- Identity of $\circ$: can be composed obeying to identity law, $f \circ \text{id}_{A_1} = f = \text{id}_{A_2} \circ f$
- Associativity of $\circ$: and associativity law, $(f_3 \circ f_2) \circ f_1 = f_3 \circ (f_2 \circ f_1) = f_3 \circ f_2 \circ f_1$.
- Diagram of category with identities and associative compositions.



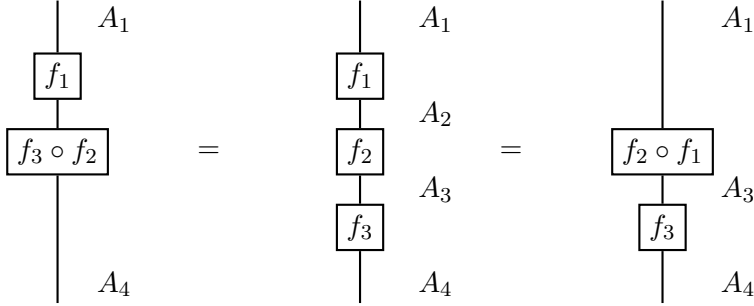
- Commutative diagram of associativity and identity law of composition.



- String diagram of preservation of identity law for identity morphisms.



- String diagram of preservation of associativity law for composition of morphisms.

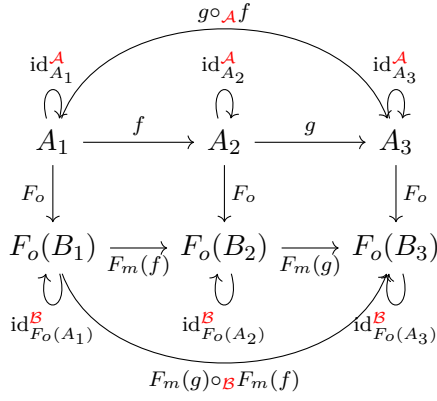


Functors are views on categories. They allow us to focus on parts, build the blocks of composites with natural transformations between functors making sure that these composites are consistent in a definable way. We use functors to lift all sectoral decisions of the economy from being a function to being a functor which also take care of the domain and codomain maps for managing units of accounts and units in economic systems. Domains and codomains also change when objects are created and deleted, formalisable by initial and final objects. By that any parts of the economy can be composed and made consistent by suitable natural transformations to the functors.

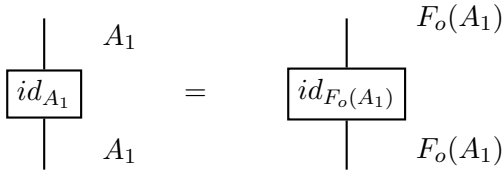
Definition A.2 (Functor). A functor $F : \mathcal{A} \rightarrow \mathcal{B}$ between categories $\mathcal{A}$ and $\mathcal{B}$ consists of:

- An object mapping: $F_o : \text{Ob}(\mathcal{A}) \rightarrow \text{Ob}(\mathcal{B})$
- A morphism mapping: $F_m : \text{Hom}_{\mathcal{A}}(A_1, A_2) \rightarrow \text{Hom}_{\mathcal{B}}(F_o(A_1), F_o(A_2))$
- which preserve identity: $F_m(id_A^{\mathcal{A}}) = id_{F_o(A)}^{\mathcal{B}}, \forall A$
- and composition: $F_m(g \circ_{\mathcal{A}} f) = F_m(g) \circ_{\mathcal{B}} F_m(f), \forall A, f, g$

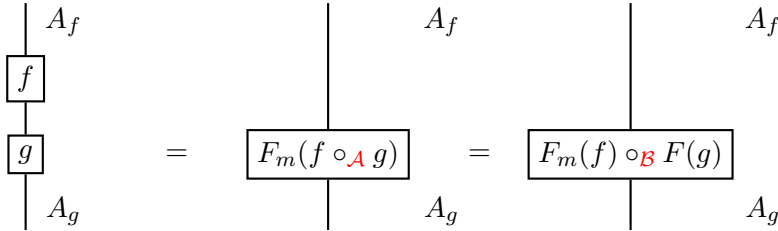
- Commutative diagram of identity and composition preservation:



- String diagram of identity preservation:



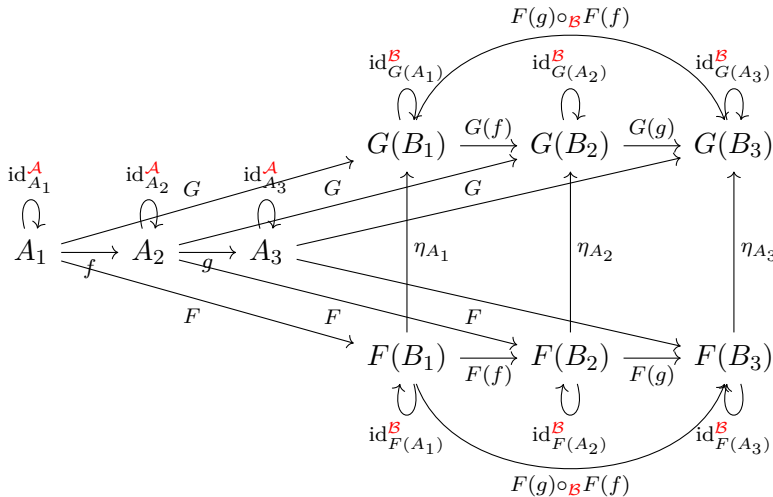
- String diagram of composition preservation:



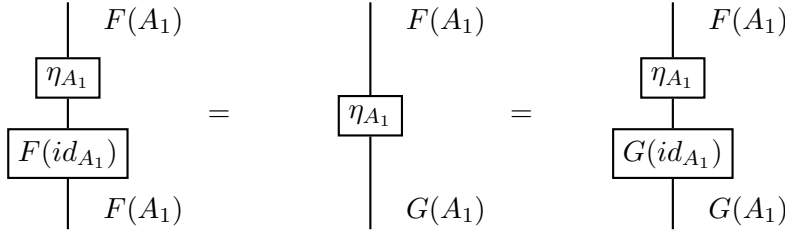
A natural transformation is a morphism between functors. It preserves identities and composition meaning they keep the systems consistent. We can choose in what way they are to be kept consistent. In economics usually some optimality or decision is formulated as a maximisation of some object or flow by some agent. In category theory these are then the universals which formulate in what sense the choice is best, maximal or what ever way optimality is useful or optimal to be defined.

Definition A.3 (Natural Transformation). A natural transformation $\eta : F \Rightarrow G$ between functors $F, G : \mathcal{A} \rightarrow \mathcal{B}$ consists of:

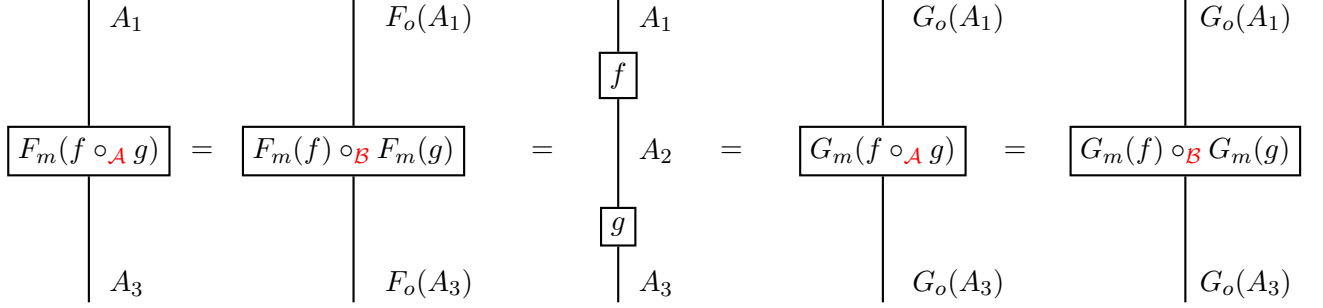
- For each object a morphism: $\eta_A : F(A) \rightarrow G(A)$ for each object $A \in \mathcal{A}$,
- which preserves identities $\eta_{A_1} \circ_B F(id_{A_1}^A) = \eta_{A_1} = G(id_{A_1}^A) \circ_B \eta_{A_1}$
- and composition $\eta_{A_1} \circ_B F(g \circ_A f) = \eta_{A_1} \circ_B F(g) \circ_B F(f)$
- Commutative diagram of identity and composition preservation:



- String diagram of identity preservation:



- String diagram of composition preservation:

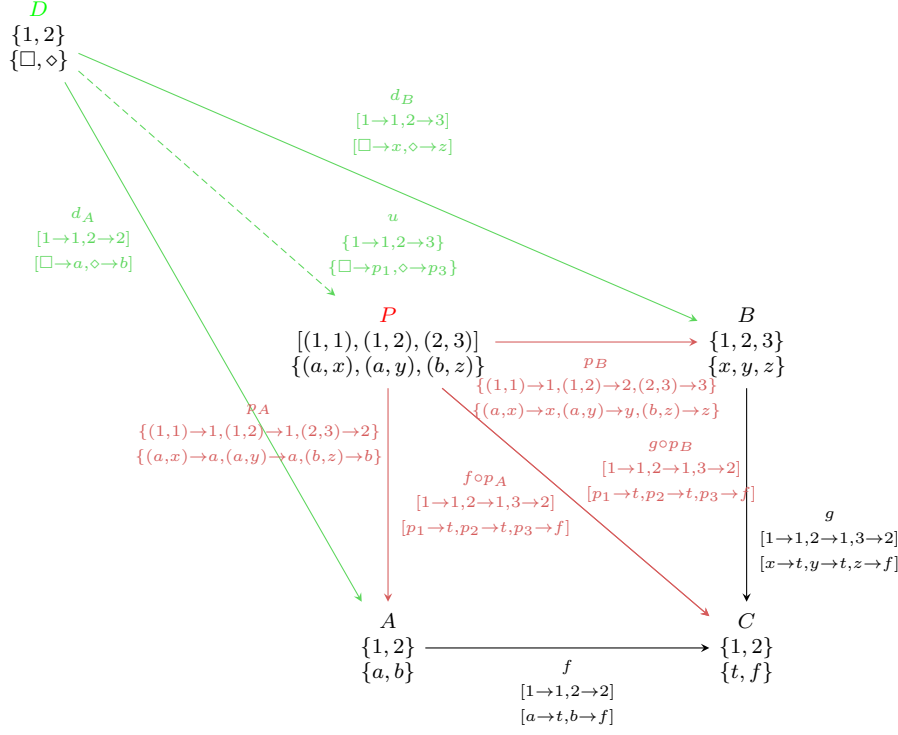


A.1.2 Pullback and Pushout

Universals are the categorical way to say optimal, or best or what ever. It is to be formulated including for all or there exists a unique, i.e. universal object with some properties of interest. We use pullback and pushout as the validation, calculation and aggregation in a compositional way for macroeconomic accounting. Universal constructions can be motivated economically as contracts between the analyst (of the model, theory, product, business model) ABC in black, the **calculator** P in red and a green **user** (client or customer) D . In the diagrams the objects A, B, C, D, P are sets defined by $\{\}$. In the first line we have the elements numbered, meaning that the actual elements do not matter. In category theory, once we know that an id function exists for each object, the objects actually do not matter anymore. Then only the behaviours of the elements matter once the id is composed with the environment of the object, i.e. once the inside is exposed to interaction with outside morphisms and other behaviours. The universals then pick out those behaviours and optimality conditions of interest. In the second line under the objects we have the elements labeled like $\{a, b\}$, for human readability. The morphisms $f, g, p_A, p_B, d_A, d_B, u$ are functions defined by $\square$ being lists of mappings of elements from domain to elements of the codomain, like $[a \rightarrow t, \dots]$ mapping a to t as part of the function.

Definition A.4 (Pullback). Given morphisms $f : A \rightarrow C$ and $g : B \rightarrow C$, a pullback consists of

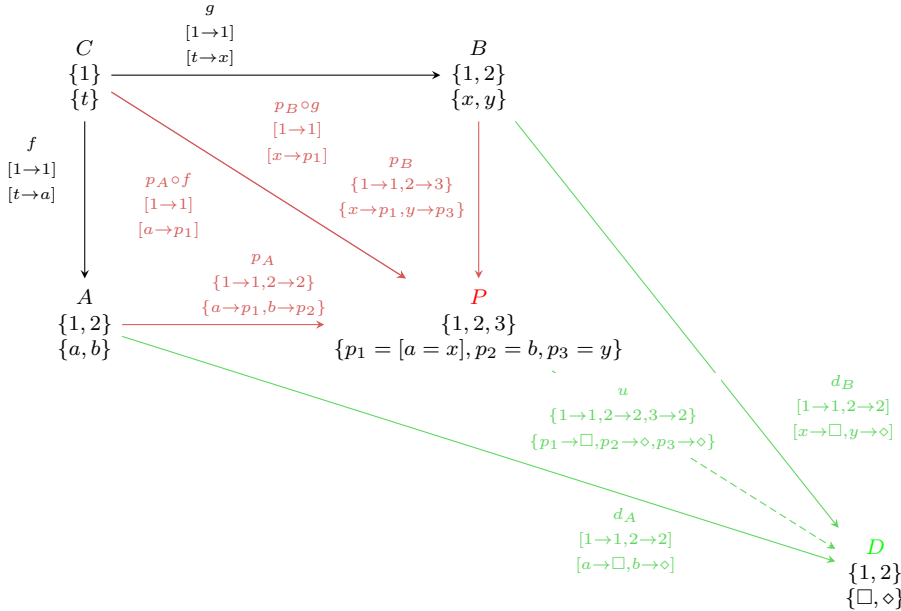
- An object P and morphisms $p_A : P \rightarrow A$, $p_B : P \rightarrow B$
- with the universal property: $\forall D, d_A : D \rightarrow A, d_B : D \rightarrow B, \exists! u : D \rightarrow P$
- Commutative diagram of the pullback:



The pullback contract is that P can internalise any connection of a user D to the model ABC through the universal construction. The pullback factorises and deconstructs, it shows where f and g are equal over C . Accordingly P are those pairs of the cartesian product of A and B where f and g are equal.

Definition A.5 (Pushout). Given morphisms $f : C \rightarrow A$ and $g : C \rightarrow B$, a pushout consists of

- An object P and morphisms $p_A : A \rightarrow P$, $p_B : B \rightarrow P$
- with the universal property: $\forall D, d_A : A \rightarrow D, d_B : B \rightarrow D, \exists! u : P \rightarrow D$
- Commutative diagram of the pushout:



The pushout is dual to the pullback which in category theory means all arrows being reversed. The pushout contract is that P gives equal elements of composites of the model ABC a name or ID specific to the user D . The pushout is about aggregation or concept formation while the pullback deconstructs and factorises. In our application the pullback validates and the pushout computes. The pullback check all parts to compose and the pushout applies the morphisms to the calculations of the whole from the parts they represent. Any connections of a user D can be replaced by the **calculator's** constructions, namely $d_A = p_A \circ u$ and $d_B = p_B \circ u$. The pushout is the construction to be precise about when a magical miracle occurs in the transition from micro- to macroeconomic. The colimit is for constructing

emergent phenomena, see Ehresmann and Vanbreemersch [2007], that humans do in knowledge formation, consciousness, production and assembly lines, and processes of social institutions or in using macroeconomic accounting in a consistent way to create money for the loan and repayment relations in human groups where trading and producing occurs over time. The pushout P is the quotient of the disjoint union of A and B over an equivalence relation over C . The whole is glued over C in a consistent way, which is what we use for the *macro* of macroeconomics. The concept formation can be seen in P where equal objects in A and B are equated over C , and by that given a name, p_a , where $a = x$. Pullbacks and pushouts are like pulling common factors in front of the paranthesis before the parts are calculated into the whole calculations. In economics it is like pulling a rule into an institution in front of the *paranthesis* of the local agent deciding inside the paranthesis - hopefully resulting in a universal welfare gain, like a universally verified Pareto gain, by a pullback, for all, at the macroeconomic level, in a pushout.

A.2 Programs

A.2.1 Recursive Program

```

1  using DataFrames
2  using CSV
3  using Parameters
4  using Plots
5
6  # -----
7  #                                     --- Parameters
8  # -----
9  @with_kw struct SystemParams
10     InvestmentLen::Int64 = 10          # 1. length and number of investments
11     LabResourceRatio::Float64 = 0.2    # 2. investment ratio wages resources
12     ConsumRatioRes::Float64 = 0.8      # 3. ratio resources consumption
13     ConsumRatioLab::Float64 = 0.95     # 4. ratio wages consumption
14     ConsumRatioCap::Float64 = 0.6      # 5. ratio dividends consumption
15     Markup::Float64 = 0.5             # 6. markup
16     DivRateDiff::Float64 = 0.15        # 7. dividend rate companies
17     DivRateBank::Float64 = 0.4         # 8. dividend rate banks
18     WindFallProfit::Float64 = 0.5      # 9. windfall profit
19     sigA::Float64 = 20.0               # 10. sigA decide investment from demand gap
20     sigB::Float64 = 480.0              # 11. sigB decide investment from demand gap
21     sigC::Float64 = 200.0              # 12. sigC decide investment from demand gap
22     ResourcePrice::Float64 = 25.0      # 13. price resources
23     LaborPrice::Float64 = 12.0          # 14. price labor, wages
24     InitialGoodPrice::Float64 = 30.0   # 15. initial price goods
25     LabResSubstProd::Float64 = 0.75    # 16. elasticity of substitution
26     ScaleProd::Float64 = 0.42          # 17. production scalar
27     DecayGoodLab::Float64 = 0.95       # 18. depreciation labor
28     DecayGoodRes::Float64 = 0.7        # 19. depreciation resources
29     DecayGoodCap::Float64 = 0.6        # 20. depreciation capital
30     ReNewRes::Float64 = 100.0           # 21. new resources each period
31     ReNewLab::Float64 = 100.0          # 22. new labor each period
32 end
33
34 # -----
35 #                                     --- State
36 # -----
37 @with_kw mutable struct State
38     Parameters::Any
39     wageHist::Vector{Float64} = zeros(Parameters.InvestmentLen)
40     repayHist::Vector{Float64} = zeros(Parameters.InvestmentLen)
41     AccLabBank::Float64 = 0.0           # Labor, Asset, Nominal
42     AccLabLab::Float64 = 0.0            # Asset, Real
43     AccLabGood::Float64 = 0.0           # Asset, Real
44     AccResBank::Float64 = 0.0           # Resource, Asset, Nominal
45     AccResRes::Float64 = 0.0            # Asset, Real
46     AccResGood::Float64 = 0.0           # Asset, Real
47     AccComBank::Float64 = 0.0           # Company, Asset, Nominal
48     AccComLoan::Float64 = 0.0           # Liability, Nominal

```

```

49 AccComDiv:Float64 = 0.0 # Liability, Nominal
50 AccComRes:Float64 = 20.0 # Asset, Real
51 AccComLab:Float64 = 110.0 # Asset, Real
52 AccComGood:Float64 = 0.0 # Asset, Real
53 AccCapBank:Float64 = 0.0 # Capital, Asset, Nominal
54 AccCapDiv:Float64 = 0.0 # Asset, Nominal
55 AccCapGood:Float64 = 0.0 # Asset, Real
56 AccBankComLoan:Float64 = 0.0 # Bank, Liability, Nominal
57 AccBankComBank:Float64 = 0.0 # Liability, Nominal
58 AccBankLabBank:Float64 = 0.0 # Liability, Nominal
59 AccBankResBank:Float64 = 0.0 # Liability, Nominal
60 AccBankCapBank:Float64 = 0.0 # L, N
61 end
62
63 # -----
64 # Helper function for history of wages and repayments (FILO stack)
65 # -----
66 function push_to_history(hist:Vector{Float64}, newelem:Float64)
67     return [newelem; hist[1:end-1]]
68 end
69
70 # -----
71 # --- Dynamics
72 # -----
73 function StateTransition(sim, state, period)
74     p2H(hist, newelem) = [newelem; hist[1:end-1]]
75     pars = state.Parameters
76     stateNew = State(Parameters=pars)
77     # -----
78     # --- Invariances
79     # -----
80     InvCapBank = state.AccCapBank - state.AccBankCapBank
81     InvResBank = state.AccResBank - state.AccBankResBank
82     InvComBank = state.AccComBank - state.AccBankComBank
83     InvComLoan = state.AccBankComLoan - state.AccComLoan
84     InvLabBank = state.AccLabBank - state.AccBankLabBank
85     InvMacro = InvCapBank + InvResBank + InvComBank + InvComLoan + InvLabBank
86     # -----
87     # --- Decisions
88     # -----
89     WagesPayment = sum(state.wageHist) # Labour
90     RepaysPayment = sum(state.repayHist) # Loan
91     # -----
92     ConsumRes = state.AccResBank * pars.ConsumRatioRes # Consumption
93     ConsumLab = state.AccLabBank * pars.ConsumRatioLab
94     ConsumCap = state.AccCapBank * pars.ConsumRatioCap
95     # -----
96     Demand = ConsumRes + ConsumLab + ConsumCap # Demand
97     # -----
98     GoodProduction = # Production
99         1 + pars.ScaleProd * state.AccComLab^pars.LabResSubstProd *
100         state.AccComRes^(1 - pars.LabResSubstProd)
101     DemandPlan = (WagesPayment + RepaysPayment) * (1 + pars.MarkUp) # Plan
102     DemandSurplus = Demand - DemandPlan # Error
103     # -----
104     GoodPrice = # Price
105         ((period == 0 ? pars.InitialGoodPrice : 0.0)
106         + DemandPlan / GoodProduction
107         + ((DemandSurplus > 0.0)
108         ? (DemandSurplus * pars.WindFallProfit) : 0.0))
109     # -----
110     Investment = pars.sigA + pars.sigB / # Investment
111         (1.0 + exp(-DemandSurplus / pars.sigC))
112     InvestmentRes = Investment * (1 - pars.LabResourceRatio)
113     InvestmentLab = Investment * pars.LabResourceRatio
114     # -----
115     Repayment = Investment / pars.InvestmentLen # Repayment
116     # -----
117     stateNew.wageHist = p2H(state.wageHist, InvestmentLab)

```

```

118 stateNew.repayHist = p2H(state.repayHist, Repayment)
119 # -----
120 # --- Bookings
121 # -----
122 inAccLabBank = WagesPayment # Labour
123 outAccLabBank = ConsumLab
124 stateNew.AccLabBank = state.AccLabBank + inAccLabBank - outAccLabBank
125 # -----
126 inAccLabLab = pars.ReNewLab
127 outAccLabLab = WagesPayment / pars.LaborPrice
128 stateNew.AccLabLab = state.AccLabLab + inAccLabLab - outAccLabLab
129
130 inAccLabGood = ConsumLab / GoodPrice
131 outAccLabGood = state.AccLabGood * pars.DecayGoodLab
132 stateNew.AccLabGood = state.AccLabGood + inAccLabGood - outAccLabGood
133 # -----
134 inAccResBank = InvestmentRes # Resource
135 outAccResBank = ConsumRes
136 stateNew.AccResBank = state.AccResBank + inAccResBank - outAccResBank
137 # -----
138 inAccResRes = pars.ReNewRes
139 outAccResRes = InvestmentRes / pars.ResourcePrice
140 stateNew.AccResRes = state.AccResRes + inAccResRes - outAccResRes
141 # -----
142 inAccResGood = ConsumRes / GoodPrice
143 outAccResGood = state.AccResGood * pars.DecayGoodRes
144 stateNew.AccResGood = state.AccResGood + inAccResGood - outAccResGood
145 # -----
146 DividendPayment = state.AccComDiv # Company
147 inAccComBank = (Investment + ConsumLab + ConsumRes + ConsumCap)
148 outAccComBank = (InvestmentRes + WagesPayment + DividendPayment + RepaysPayment)
149 Diff = inAccComBank - outAccComBank
150 stateNew.AccComBank = state.AccComBank + Diff
151 # -----
152 DividendDecision =
153     (Diff > 0.0
154     ? (Diff * pars.DivRateDiff) : 0.0) +
155     ((state.AccComBank > 0)
156     ? (state.AccComBank * pars.DivRateBank) : 0.0)
157 inAccComDiv = DividendDecision
158 outAccComDiv = DividendPayment
159 stateNew.AccComDiv = state.AccComDiv + inAccComDiv - outAccComDiv
160 # -----
161 inAccComLoan = Investment
162 outAccComLoan = RepaysPayment
163 stateNew.AccComLoan = state.AccComLoan + inAccComLoan - outAccComLoan
164 # -----
165 inAccComRes = InvestmentRes / pars.ResourcePrice
166 outAccComRes = state.AccComRes
167 stateNew.AccComRes = state.AccComRes + inAccComRes - outAccComRes
168 # -----
169 inAccComLab = WagesPayment / pars.LaborPrice
170 outAccComLab = state.AccComLab # labor usage in production
171 stateNew.AccComLab = state.AccComLab + inAccComLab - outAccComLab
172 # -----
173 inAccComGood = GoodProduction
174 outAccComGood = (ConsumRes + ConsumLab + ConsumCap) / GoodPrice
175 stateNew.AccComGood = state.AccComGood + inAccComGood - outAccComGood
176 # -----
177 DividendIncome = state.AccCapDiv # Capitalist
178 inAccCapDiv = DividendDecision
179 outAccCapDiv = DividendIncome
180 stateNew.AccCapDiv = state.AccCapDiv + inAccCapDiv - outAccCapDiv
181 # -----
182 inAccCapBank = DividendIncome
183 outAccCapBank = ConsumCap
184 stateNew.AccCapBank = state.AccCapBank + inAccCapBank - outAccCapBank
185 # -----
186 inAccCapGood = ConsumCap / GoodPrice

```

```

187 outAccCapGood = state.AccCapGood * pars.DecayGoodCap
188 stateNew.AccCapGood = state.AccCapGood + inAccCapGood - outAccCapGood
189 # -----
190 inAccBankComLoan = Investment # Bank
191 outAccBankComLoan = RepaysPayment
192 stateNew.AccBankComLoan = state.AccBankComLoan + inAccBankComLoan - outAccBankComLoan
193 # -----
194 inAccBankComBank = Investment + ConsumRes + ConsumLab + ConsumCap
195 outAccBankComBank = InvestmentRes + WagesPayment + DividendPayment + RepaysPayment
196 stateNew.AccBankComBank = state.AccBankComBank + inAccBankComBank - outAccBankComBank
197 # -----
198 inAccBankLabBank = WagesPayment
199 outAccBankLabBank = ConsumLab
200 stateNew.AccBankLabBank = state.AccBankLabBank + inAccBankLabBank - outAccBankLabBank
201 # -----
202 inAccBankResBank = InvestmentRes
203 outAccBankResBank = ConsumRes
204 stateNew.AccBankResBank = state.AccBankResBank + inAccBankResBank - outAccBankResBank
205 # -----
206 inAccBankCapBank = DividendIncome
207 outAccBankCapBank = ConsumCap
208 stateNew.AccBankCapBank = state.AccBankCapBank + inAccBankCapBank - outAccBankCapBank
209 # -----
210 push!(sim, (Period=period,
211     InvCapBank=InvCapBank,
212     InvResBank=InvResBank,
213     InvComBank=InvComBank,
214     InvComLoan=InvComLoan,
215     InvLabBank=InvLabBank,
216     InvMacro=InvMacro,
217     ConsumRes=ConsumRes,
218     ConsumLab=ConsumLab,
219     ConsumCap=ConsumCap,
220     Demand=Demand,
221     DemandPlan=DemandPlan,
222     DemandSurplus=DemandSurplus,
223     GoodProduction=GoodProduction,
224     GoodPrice=GoodPrice,
225     Investment=Investment,
226     InvestmentRes=InvestmentRes,
227     InvestmentLab=InvestmentLab,
228     Repayment=Repayment,
229     WagesPayment=WagesPayment,
230     RepaysPayment=RepaysPayment,
231     Diff=Diff,
232     DividendDecision=DividendDecision,
233     DividendPayment=DividendPayment,
234     DividendIncome=DividendIncome,
235     AccLabBank=state.AccLabBank,
236     AccLabLab=state.AccLabLab,
237     AccLabGood=state.AccLabGood,
238     AccResBank=state.AccResBank,
239     AccResRes=state.AccResRes,
240     AccResGood=state.AccResGood,
241     AccCapBank=state.AccCapBank,
242     AccCapDiv=state.AccCapDiv,
243     AccCapGood=state.AccCapGood,
244     AccComBank=state.AccComBank,
245     AccComLoan=state.AccComLoan,
246     AccComDiv=state.AccComDiv,
247     AccComRes=state.AccComRes,
248     AccComLab=state.AccComLab,
249     AccComGood=state.AccComGood,
250     AccBankComLoan=state.AccBankComLoan,
251     AccBankComBank=state.AccBankComBank,
252     AccBankLabBank=state.AccBankLabBank,
253     AccBankResBank=state.AccBankResBank,
254     AccBankCapBank=state.AccBankCapBank)
255 )

```



```

256     return sim, stateNew
257 end
258
259 # Run simulation and save results
260 function run_and_save_simulation(nperiods::Int64, output_file::String)
261     sim = DataFrame()
262     state = State(Parameters=SystemParams())
263
264     for period in 0:nperiods
265         sim, state = StateTransition(sim, state, period)
266     end
267
268     # Replace near zero values with 0.0
269     eps = 1e-12
270     for col in names(sim)
271         if eltype(sim[:, col]) <: Number
272             sim[:, col] = map(x -> abs(x) < eps ? 0.0 : x, sim[:, col])
273         end
274     end
275
276     # Save results
277     CSV.write(output_file, sim)
278     println("Recursive simulation results saved to: $output_file")
279
280     # Create visualizations
281     create_time_series_plots(sim, replace(output_file, ".csv" => ".plots.png"))
282
283     return sim
284 end
285
286 # Create comprehensive time series plots
287 function create_time_series_plots(sim::DataFrame, output_file::String)
288     # Get all variable names except Period
289     var_names = filter(x -> x != :Period, names(sim))
290     n_vars = length(var_names)
291
292     # Calculate grid dimensions (aim for roughly square layout)
293     n_cols = ceil{Int, sqrt}(n_vars)
294     n_rows = ceil{Int, n_vars / n_cols}
295
296     # Create subplots
297     plots_array = []
298
299     for (i, var) in enumerate(var_names)
300         # Determine if this is one of the five invariance variables (compare strings)
301         is_inv = string(var) in ["InvCapBank", "InvResBank", "InvComBank", "InvComLoan", "InvLabBank", "
302             InvMacro"]
303
304         # Calculate y-axis limits
305         y_lims = if is_inv
306             # For invariance variables, center around zero
307             max_abs = maximum(abs.(sim[:, var]))
308             # Handle case where all values are zero
309             if max_abs == 0.0
310                 (-1.0, 1.0) # Set a default range when all values are zero
311             else
312                 (-max_abs * 1.1, max_abs * 1.1) # Add 10% padding, centered at zero
313             end
314         else
315             # For other variables, use actual min/max with padding
316             min_val = minimum(sim[:, var])
317             max_val = maximum(sim[:, var])
318             range_val = max_val - min_val
319             if range_val == 0.0
320                 # Handle constant values
321                 (min_val - 1.0, max_val + 1.0)
322             else
323                 # Add 10% padding to both sides
324                 padding = range_val * 0.1

```

```

324         (min_val - padding, max_val + padding)
325     end
326 end
327
328 # Create plot with centered zero
329 plot_args = (sim.Period, sim[:, var])
330 plot_kwargs = Dict(
331     :title => string(var),
332     :xlabel => "Period",
333     :ylabel => string(var),
334     :legend => false,
335     :linewidth => 2,
336     :titlefontsize => 8,
337     :labelfontsize => 6,
338     :tickfontsize => 5,
339     :color => is_inv ? :red : :blue,
340     :ylims => y_lims
341 )
342
343 p = plot(plot_args...; plot_kwargs...)
344
345 # Add zero line if zero is within the visible range
346 if y_lims[1] <= 0 <= y_lims[2]
347     hline!(p, [0], color=:black, linestyle=:dash, alpha=0.5)
348 end
349
350 push!(plots_array, p)
351 end
352
353 # Combine all plots
354 combined_plot = plot(plots_array...,
355     layout=(n_rows, n_cols),
356     size=(1600, 1200),
357     margin=2Plots.mm)
358
359 # Save the plot
360 savefig(combined_plot, output_file)
361 println("Recursive time series plots saved to: $output_file")
362 end
363
364 run_and_save_simulation(100, joinpath(@__DIR__, "money.theory.rec.cat.csv"))

```

A.2.2 Categorically Typed Recursive Program

```
1 using DataFrames
2 using CSV
3 using Parameters
4 using Plots
5
6 include("money.theory.rec.cat.jl")
7
8 mutable struct Vertex
9     id::Int
10    name::String
11    category::String
12    amount::Float64
13 end
14
15 struct Edge
16     source::Int
17     target::Int
18     weight::Float64
19 end
20
21 struct MonetaryCategory
22     vertices::Vector{Vertex}
23     edges::Vector{Edge}
24     name::String # e.g., "Accounts", "Flows", "Parameters"
25 end
26
27 # Constructor
28 function MonetaryCategory(name::String)
29     return MonetaryCategory(Vertex[], Edge[], name)
30 end
31
32 struct MonetaryFunctor
33     source::MonetaryCategory
34     target::MonetaryCategory
35     vertex_map::Dict{Int,Int} # Maps source vertex IDs to target vertex IDs
36     name::String # e.g., "PriceFunctor", "FlowFunctor"
37 end
38
39 struct NaturalTransformation
40     source_functor::MonetaryFunctor
41     target_functor::MonetaryFunctor
42     component_map::Dict{Int,Edge} # Maps vertex IDs to transformation edges
43     name::String # e.g., "PriceAdjustment", "FlowAggregation"
44 end
45
46 function create_pullback(cat1::MonetaryCategory, cat2::MonetaryCategory,
47     common::MonetaryCategory,
48     f1::MonetaryFunctor, f2::MonetaryFunctor)
49     # Create pullback category
50     pullback = MonetaryCategory("Pullback")
51
52     # Add vertices that satisfy the pullback condition
53     vertex_map = Dict{Int,Int}()
54     for v1 in cat1.vertices
55         for v2 in cat2.vertices
56             if f1.vertex_map[v1.id] == f2.vertex_map[v2.id]
57                 new_id = add_vertex!(pullback, "$(v1.name)_$(v2.name)",
58                     v1.category, v1.amount)
59                 vertex_map[v1.id, v2.id] = new_id
60             end
61         end
62     end
63
64     return pullback, vertex_map
65 end
```

```

66
67 function create_pushout(cat1::MonetaryCategory, cat2::MonetaryCategory,
68   common::MonetaryCategory,
69   f1::MonetaryFunctor, f2::MonetaryFunctor)
70   # Create pushout category
71   pushout = MonetaryCategory("Pushout")
72
73   # Add vertices from both categories
74   vertex_map = Dict{Int,Int}()
75   for v in cat1.vertices
76     new_id = add_vertex!(pushout, v.name, v.category, v.amount)
77     vertex_map[v.id] = new_id
78   end
79   for v in cat2.vertices
80     if !haskey(vertex_map, v.id)
81       new_id = add_vertex!(pushout, v.name, v.category, v.amount)
82       vertex_map[v.id] = new_id
83     end
84   end
85
86   return pushout, vertex_map
87 end
88
89 mutable struct CategoricalState
90   accounts::MonetaryCategory      # Account vertices and edges
91   flows::MonetaryCategory          # Flow vertices and edges
92   parameters::MonetaryCategory    # Parameter vertices
93
94   # Functors
95   price_functor::MonetaryFunctor   # Maps nominal to real values
96   flow_functor::MonetaryFunctor    # Maps accounts to flows
97
98   # Natural Transformations
99   price_adjustment::NaturalTransformation # Price changes over time
100  flow_aggregation::NaturalTransformation # Flow summation
101
102  # History
103  wage_hist::Vector{Float64}        # History of wage payments
104  repay_hist::Vector{Float64}       # History of repayments
105 end
106
107 # Get vertex by name from a category
108 function get_vertex(cat::MonetaryCategory, name::String)
109   idx = findfirst(v -> v.name == name, cat.vertices)
110   isnothing(idx) && error("Vertex $name not found")
111   return cat.vertices[idx]
112 end
113
114 # Add vertex to a category
115 function add_vertex!(cat::MonetaryCategory, name::String, type::String, amount::Float64)
116   id = length(cat.vertices) + 1
117   vertex = Vertex(id, name, type, amount)
118   push!(cat.vertices, vertex)
119   return id
120 end
121
122 # Add edge to a category
123 function add_edge!(cat::MonetaryCategory, source::Int, target::Int, weight::Float64)
124   edge = Edge(source, target, weight)
125   push!(cat.edges, edge)
126   return edge
127 end
128
129 # Update vertex amount
130 function update_vertex!(cat::MonetaryCategory, name::String, amount::Float64)
131   vertex = get_vertex(cat, name)
132   vertex.amount = amount
133 end
134

```

```

135 # Add mapping to functor
136 function add_mapping!(f::MonetaryFunctor, source_id::Int, target_id::Int)
137     f.vertex_map[source_id] = target_id
138 end
139
140 # Apply functor to vertex
141 function apply_functor(f::MonetaryFunctor, vertex_id::Int)
142     haskey(f.vertex_map, vertex_id) || error("No mapping for vertex $vertex_id")
143     return f.vertex_map[vertex_id]
144 end
145
146 # Add component to natural transformation
147 function add_component!(nt::NaturalTransformation, vertex_id::Int, edge::Edge)
148     nt.component_map[vertex_id] = edge
149 end
150
151 # Apply natural transformation to vertex
152 function apply_transformation(nt::NaturalTransformation, vertex_id::Int)
153     haskey(nt.component_map, vertex_id) || error("No component for vertex $vertex_id")
154     return nt.component_map[vertex_id]
155 end
156
157 function create_categorical_state(state::State)
158     # Create categories
159     accounts = MonetaryCategory("Accounts")
160     flows = MonetaryCategory("Flows")
161     parameters = MonetaryCategory("Parameters")
162
163     # Add account vertices with initial values
164     # Labor accounts
165     lab_bank = add_vertex!(accounts, "Labor_Bank", "Asset", 0.0) # A, N
166     lab_stock = add_vertex!(accounts, "Labor_Stock", "Real", 0.0) # A, R
167     lab_good = add_vertex!(accounts, "Labor_Good", "Real", 0.0) # A, R
168
169     # Resource accounts
170     res_bank = add_vertex!(accounts, "Resource_Bank", "Asset", 0.0) # A, N
171     res_stock = add_vertex!(accounts, "Resource_Stock", "Real", 0.0) # A, R
172     res_good = add_vertex!(accounts, "Resource_Good", "Real", 0.0) # A, R
173
174     # Company accounts
175     com_bank = add_vertex!(accounts, "Company_Bank", "Asset", 0.0) # A, N
176     com_loan = add_vertex!(accounts, "Company_Loan", "Liability", 0.0) # L, N
177     com_div = add_vertex!(accounts, "Company_Dividend", "Liability", 0.0) # L, N
178     com_res = add_vertex!(accounts, "Company_Resource", "Real", 20.0) # A, R - Initial value!
179     com_lab = add_vertex!(accounts, "Company_Labor", "Real", 110.0) # A, R - Initial value!
180     com_good = add_vertex!(accounts, "Company_Good", "Real", 0.0) # A, R
181
182     # Capitalist accounts
183     cap_bank = add_vertex!(accounts, "Capital_Bank", "Asset", 0.0) # A, N
184     cap_div = add_vertex!(accounts, "Capital_Dividend", "Asset", 0.0) # A, N
185     cap_good = add_vertex!(accounts, "Capital_Good", "Real", 0.0) # A, R
186
187     # Bank accounts
188     bank_loan = add_vertex!(accounts, "Bank_Loan", "Asset", 0.0) # A, N
189     bank_com = add_vertex!(accounts, "Bank_Company", "Liability", 0.0) # L, N
190     bank_lab = add_vertex!(accounts, "Bank_Labor", "Liability", 0.0) # L, N
191     bank_res = add_vertex!(accounts, "Bank_Resource", "Liability", 0.0) # L, N
192     bank_cap = add_vertex!(accounts, "Bank_Capital", "Liability", 0.0) # L, N
193
194     # Add parameter vertices
195     pars = state.Parameters
196     for field in fieldnames(typeof(pars))
197         value = getfield(pars, field)
198         add_vertex!(parameters, String(field), "Parameter", Float64(value))
199     end
200
201     # Create price functor (nominal to real)
202     price_functor = MonetaryFunctor(accounts, accounts, Dict{Int,Int}(), "PriceFunctor")
203     add_mapping!(price_functor, lab_bank, lab_stock) # Labor: nominal to real

```

```

204 add_mapping!(price_func, res_bank, res_stock) # Resource: nominal to real
205 add_mapping!(price_func, com_bank, com_good) # Company: nominal to real
206
207 # Create flow functor (accounts to flows)
208 flow_func = MonetaryFunctor(accounts, flows, Dict{Int,Int}(), "FlowFunctor")
209
210 # Create natural transformations
211 price_adj = NaturalTransformation(price_func, price_func,
212     Dict{Int,Edge}(), "PriceAdjustment")
213 flow_agg = NaturalTransformation(flow_func, flow_func,
214     Dict{Int,Edge}(), "FlowAggregation")
215
216 # Initialize memory vectors
217 wage_hist = zeros(pars.InvestmentLen)
218 repay_hist = zeros(pars.InvestmentLen)
219
220 return CategoricalState(accounts, flows, parameters,
221     price_func, flow_func,
222     price_adj, flow_agg,
223     wage_hist, repay_hist)
224 end
225
226 function categorical_state_transition(sim::DataFrame, cat_state::CategoricalState, period::Int64)
227     # Create new categorical state
228     new_cat_state = deepcopy(cat_state)
229
230     # Get parameters from categorical structure
231     pars = SystemParams() # We'll update this to use categorical parameters later
232
233     # Helper function to push history
234     p2H(hist, newelem) = [newelem; hist[1:end-1]]
235
236     # Calculate investment balances using categorical structure
237     cap_bank = get_vertex(cat_state.accounts, "Capital_Bank").amount
238     bank_cap = get_vertex(cat_state.accounts, "Bank_Capital").amount
239     InvCapBank = cap_bank - bank_cap
240
241     res_bank = get_vertex(cat_state.accounts, "Resource_Bank").amount
242     bank_res = get_vertex(cat_state.accounts, "Bank_Resource").amount
243     InvResBank = res_bank - bank_res
244
245     com_bank = get_vertex(cat_state.accounts, "Company_Bank").amount
246     bank_com = get_vertex(cat_state.accounts, "Bank_Company").amount
247     InvComBank = com_bank - bank_com
248
249     bank_loan = get_vertex(cat_state.accounts, "Bank_Loan").amount
250     com_loan = get_vertex(cat_state.accounts, "Company_Loan").amount
251     InvComLoan = bank_loan - com_loan
252
253     lab_bank = get_vertex(cat_state.accounts, "Labor_Bank").amount
254     bank_lab = get_vertex(cat_state.accounts, "Bank_Labor").amount
255     InvLabBank = lab_bank - bank_lab
256
257     InvMacro = InvCapBank + InvResBank + InvComBank + InvComLoan + InvLabBank
258
259     # Get wage and repayment history from categorical state
260     WagesPayment = sum(cat_state.wage_hist)
261     RepaysPayment = sum(cat_state.repay_hist)
262
263     # Calculate consumption using categorical structure
264     ConsumRes = res_bank * pars.ConsumRatioRes
265     ConsumLab = lab_bank * pars.ConsumRatioLab
266     ConsumCap = cap_bank * pars.ConsumRatioCap
267     Demand = ConsumRes + ConsumLab + ConsumCap
268
269     # Calculate production using categorical structure
270     com_lab = get_vertex(cat_state.accounts, "Company_Labor").amount
271     com_res = get_vertex(cat_state.accounts, "Company_Resource").amount
272     GoodProduction = 1 + pars.ScaleProd *

```

```

273         com_lab^pars.LabResSubstProd *
274         com_res^(1 - pars.LabResSubstProd)
275
276 # Calculate demand plan and surplus
277 DemandPlan = (WagesPayment + RepaysPayment) * (1 + pars.MarkUp)
278 DemandSurplus = Demand - DemandPlan
279
280 # Calculate good price
281 GoodPrice = ((period == 0 ? pars.InitialGoodPrice : 0.0) +
282             DemandPlan / GoodProduction +
283             ((DemandSurplus > 0.0) ? (DemandSurplus * pars.WindFallProfit) : 0.0))
284
285 # Calculate investment
286 Investment = pars.sigA + pars.sigB / (1.0 + exp(-DemandSurplus / pars.sigC))
287 InvestmentRes = Investment * (1 - pars.LabResourceRatio)
288 InvestmentLab = Investment * pars.LabResourceRatio
289 Repayment = Investment / pars.InvestmentLen
290
291 # Update histories in categorical state
292 new_cat_state.wage_hist = p2H(cat_state.wage_hist, InvestmentLab)
293 new_cat_state.repay_hist = p2H(cat_state.repay_hist, Repayment)
294
295 # Company bookings first (as they determine dividend decisions)
296 DividendPayment = get_vertex(cat_state.accounts, "Company_Dividend").amount
297
298 inAccComBank = Investment + ConsumLab + ConsumRes + ConsumCap
299 outAccComBank = InvestmentRes + WagesPayment + DividendPayment + RepaysPayment
300 Diff = inAccComBank - outAccComBank
301 update_vertex!(new_cat_state.accounts, "Company_Bank",
302               get_vertex(cat_state.accounts, "Company_Bank").amount + Diff)
303
304 DividendDecision = (Diff > 0.0 ? (Diff * pars.DivRateDiff) : 0.0) +
305                   ((com_bank > 0) ? (com_bank * pars.DivRateBank) : 0.0)
306
307 update_vertex!(new_cat_state.accounts, "Company_Dividend",
308               get_vertex(cat_state.accounts, "Company_Dividend").amount +
309               DividendDecision - DividendPayment)
310
311 update_vertex!(new_cat_state.accounts, "Company_Loan",
312               get_vertex(cat_state.accounts, "Company_Loan").amount +
313               Investment - RepaysPayment)
314
315 # Production factors are used up each period
316 update_vertex!(new_cat_state.accounts, "Company_Resource",
317               InvestmentRes / pars.ResourcePrice) # Only new investment remains
318
319 update_vertex!(new_cat_state.accounts, "Company_Labor",
320               WagesPayment / pars.LaborPrice) # Only new labor remains
321
322 update_vertex!(new_cat_state.accounts, "Company_Good",
323               get_vertex(cat_state.accounts, "Company_Good").amount +
324               GoodProduction - (ConsumRes + ConsumLab + ConsumCap) / GoodPrice)
325
326 # Labor bookings
327 inAccLabBank = WagesPayment
328 outAccLabBank = ConsumLab
329 update_vertex!(new_cat_state.accounts, "Labor_Bank",
330               get_vertex(cat_state.accounts, "Labor_Bank").amount +
331               inAccLabBank - outAccLabBank)
332
333 inAccLabLab = pars.ReNewLab
334 outAccLabLab = WagesPayment / pars.LaborPrice
335 update_vertex!(new_cat_state.accounts, "Labor_Stock",
336               get_vertex(cat_state.accounts, "Labor_Stock").amount +
337               inAccLabLab - outAccLabLab)
338
339 inAccLabGood = ConsumLab / GoodPrice
340 outAccLabGood = get_vertex(cat_state.accounts, "Labor_Good").amount * pars.DecayGoodLab
341 update_vertex!(new_cat_state.accounts, "Labor_Good",

```

```

342     get_vertex(cat_state.accounts, "Labor_Good").amount +
343     inAccLabGood - outAccLabGood)
344
345 # Resource bookings
346 inAccResBank = InvestmentRes
347 outAccResBank = ConsumRes
348 update_vertex!(new_cat_state.accounts, "Resource_Bank",
349     get_vertex(cat_state.accounts, "Resource_Bank").amount +
350     inAccResBank - outAccResBank)
351
352 inAccResRes = pars.ReNewRes
353 outAccResRes = InvestmentRes / pars.ResourcePrice
354 update_vertex!(new_cat_state.accounts, "Resource_Stock",
355     get_vertex(cat_state.accounts, "Resource_Stock").amount +
356     inAccResRes - outAccResRes)
357
358 inAccResGood = ConsumRes / GoodPrice
359 outAccResGood = get_vertex(cat_state.accounts, "Resource_Good").amount * pars.DecayGoodRes
360 update_vertex!(new_cat_state.accounts, "Resource_Good",
361     get_vertex(cat_state.accounts, "Resource_Good").amount +
362     inAccResGood - outAccResGood)
363
364 # Capital bookings
365 DividendIncome = get_vertex(cat_state.accounts, "Capital_Dividend").amount
366
367 update_vertex!(new_cat_state.accounts, "Capital_Dividend",
368     get_vertex(cat_state.accounts, "Capital_Dividend").amount +
369     DividendDecision - DividendIncome)
370
371 update_vertex!(new_cat_state.accounts, "Capital_Bank",
372     get_vertex(cat_state.accounts, "Capital_Bank").amount +
373     DividendIncome - ConsumCap)
374
375 update_vertex!(new_cat_state.accounts, "Capital_Good",
376     get_vertex(cat_state.accounts, "Capital_Good").amount +
377     ConsumCap / GoodPrice -
378     get_vertex(cat_state.accounts, "Capital_Good").amount * pars.DecayGoodCap)
379
380 # Bank bookings
381 update_vertex!(new_cat_state.accounts, "Bank_Loan",
382     get_vertex(cat_state.accounts, "Bank_Loan").amount +
383     Investment - RepaysPayment)
384
385 update_vertex!(new_cat_state.accounts, "Bank_Company",
386     get_vertex(cat_state.accounts, "Bank_Company").amount +
387     Investment + ConsumRes + ConsumLab + ConsumCap -
388     (InvestmentRes + WagesPayment + DividendPayment + RepaysPayment))
389
390 update_vertex!(new_cat_state.accounts, "Bank_Labor",
391     get_vertex(cat_state.accounts, "Bank_Labor").amount +
392     WagesPayment - ConsumLab)
393
394 update_vertex!(new_cat_state.accounts, "Bank_Resource",
395     get_vertex(cat_state.accounts, "Bank_Resource").amount +
396     InvestmentRes - ConsumRes)
397
398 update_vertex!(new_cat_state.accounts, "Bank_Capital",
399     get_vertex(cat_state.accounts, "Bank_Capital").amount +
400     DividendIncome - ConsumCap)
401
402 # Record state in the format of the recursive implementation
403 push!(sim, (Period=period,
404     InvCapBank=InvCapBank,
405     InvResBank=InvResBank,
406     InvComBank=InvComBank,
407     InvComLoan=InvComLoan,
408     InvLabBank=InvLabBank,
409     InvMacro=InvMacro,
410     ConsumRes=ConsumRes,

```



```

411 ConsumLab=ConsumLab,
412 ConsumCap=ConsumCap,
413 Demand=Demand,
414 DemandPlan=DemandPlan,
415 DemandSurplus=DemandSurplus,
416 GoodProduction=GoodProduction,
417 GoodPrice=GoodPrice,
418 Investment=Investment,
419 InvestmentRes=InvestmentRes,
420 InvestmentLab=InvestmentLab,
421 Repayment=Repayment,
422 WagesPayment=WagesPayment,
423 RepaysPayment=RepaysPayment,
424 Diff=Diff,
425 DividendDecision=DividendDecision,
426 DividendPayment=DividendPayment,
427 DividendIncome=DividendIncome,
428 AccLabBank=get_vertex(cat_state.accounts, "Labor_Bank").amount,
429 AccLabLab=get_vertex(cat_state.accounts, "Labor_Stock").amount,
430 AccLabGood=get_vertex(cat_state.accounts, "Labor_Good").amount,
431 AccResBank=get_vertex(cat_state.accounts, "Resource_Bank").amount,
432 AccResRes=get_vertex(cat_state.accounts, "Resource_Stock").amount,
433 AccResGood=get_vertex(cat_state.accounts, "Resource_Good").amount,
434 AccCapBank=get_vertex(cat_state.accounts, "Capital_Bank").amount,
435 AccCapDiv=get_vertex(cat_state.accounts, "Capital_Dividend").amount,
436 AccCapGood=get_vertex(cat_state.accounts, "Capital_Good").amount,
437 AccComBank=get_vertex(cat_state.accounts, "Company_Bank").amount,
438 AccComLoan=get_vertex(cat_state.accounts, "Company_Loan").amount,
439 AccComDiv=get_vertex(cat_state.accounts, "Company_Dividend").amount,
440 AccComRes=get_vertex(cat_state.accounts, "Company_Resource").amount,
441 AccComLab=get_vertex(cat_state.accounts, "Company_Labor").amount,
442 AccComGood=get_vertex(cat_state.accounts, "Company_Good").amount,
443 AccBankComLoan=get_vertex(cat_state.accounts, "Bank_Loan").amount,
444 AccBankComBank=get_vertex(cat_state.accounts, "Bank_Company").amount,
445 AccBankLabBank=get_vertex(cat_state.accounts, "Bank_Labor").amount,
446 AccBankResBank=get_vertex(cat_state.accounts, "Bank_Resource").amount,
447 AccBankCapBank=get_vertex(cat_state.accounts, "Bank_Capital").amount)
448 )
449 return sim, new_cat_state
450 end
451
452
453
454 function run_categorical_simulation(nperiods::Int64, output_file::String)
455     sim = DataFrame()
456     state = State(Parameters=SystemParams())
457     cat_state = create_categorical_state(state)
458
459     for period in 0:nperiods
460         sim, cat_state = categorical_state_transition(sim, cat_state, period)
461     end
462
463     # Replace near zero values with 0.0
464     eps = 1e-12
465     for col in names(sim)
466         if eltype(sim[:, col]) <: Number
467             sim[:, col] = map(x -> abs(x) < eps ? 0.0 : x, sim[:, col])
468         end
469     end
470
471     # Save results
472     CSV.write(output_file, sim)
473     println("Categorical simulation results saved to: $output_file")
474
475     # Create visualizations
476     create_categorical_time_series_plots(sim, replace(output_file, ".csv" => ".plots.png"))
477
478     return sim
479 end

```

```

480 # Create comprehensive time series plots for categorical simulation
481 function create_categorical_time_series_plots(sim::DataFrame, output_file::String)
482 # Get all variable names except Period
483 var_names = filter(x -> x != :Period, names(sim))
484 n_vars = length(var_names)
485
486 # Calculate grid dimensions (aim for roughly square layout)
487 n_cols = ceil(Int, sqrt(n_vars))
488 n_rows = ceil(Int, n_vars / n_cols)
489
490 # Create subplots
491 plots_array = []
492
493 for (i, var) in enumerate(var_names)
494 # Determine if this is one of the invariance variables (compare strings)
495 is_inv = string(var) in ["InvCapBank", "InvResBank", "InvComBank", "InvComLoan", "InvLabBank", "
496     InvMacro"]
497
498 # Calculate y-axis limits
499 y_lims = if is_inv
500     # For invariance variables, center around zero
501     max_abs = maximum(abs.(sim[:, var]))
502     # Handle case where all values are zero
503     if max_abs == 0.0
504         (-1.0, 1.0) # Set a default range when all values are zero
505     else
506         (-max_abs * 1.1, max_abs * 1.1) # Add 10% padding, centered at zero
507     end
508 else
509     # For other variables, use actual min/max with padding
510     min_val = minimum(sim[:, var])
511     max_val = maximum(sim[:, var])
512     range_val = max_val - min_val
513     if range_val == 0.0
514         # Handle constant values
515         (min_val - 1.0, max_val + 1.0)
516     else
517         # Add 10% padding to both sides
518         padding = range_val * 0.1
519         (min_val - padding, max_val + padding)
520     end
521 end
522
523 # Create plot with centered zero
524 plot_args = (sim.Period, sim[:, var])
525 plot_kwargs = Dict(
526     :title => string(var),
527     :xlabel => "Period",
528     :ylabel => string(var),
529     :legend => false,
530     :linewidth => 2,
531     :titlefontsize => 8,
532     :labelfontsize => 6,
533     :tickfontsize => 5,
534     :color => is_inv ? :red : :blue,
535     :ylims => y_lims
536 )
537
538 p = plot(plot_args...; plot_kwargs...)
539
540 # Add zero line if zero is within the visible range
541 if y_lims[1] <= 0 <= y_lims[2]
542     hline!(p, [0], color=:black, linestyle=:dash, alpha=0.5)
543 end
544
545 push!(plots_array, p)
546 end
547

```

```

548 # Combine all plots
549 combined_plot = plot(plots_array...,
550     layout=(n_rows, n_cols),
551     size=(1600, 1200),
552     margin=2Plots.mm)
553
554 # Save the plot
555 savefig(combined_plot, output_file)
556 println("Categorical time series plots saved to: $output_file")
557 end
558
559 run_categorical_simulation(100, joinpath(@__DIR__, "money.theory.cat.csv"))
560
561 # Compare the recursive and categorical simulation results
562 function compare_simulations(recursive_file::String, categorical_file::String)
563     # Read both CSV files
564     recursive_df = CSV.read(recursive_file, DataFrame)
565     categorical_df = CSV.read(categorical_file, DataFrame)
566
567     # Check if dataframes have same dimensions
568     if size(recursive_df) != size(categorical_df)
569         println("Warning: Dataframes have different dimensions!")
570         return false
571     end
572
573     # Check if column names match
574     if names(recursive_df) != names(categorical_df)
575         println("Warning: Column names don't match!")
576         return false
577     end
578
579     # Compare numerical values with tolerance
580     eps = 1e-10
581     all_equal = true
582
583     for col in names(recursive_df)
584         max_diff = maximum(abs.(recursive_df[:, col] - categorical_df[:, col]))
585         if max_diff > eps
586             println("Warning: Column $col has maximum difference of $max_diff")
587             all_equal = false
588         end
589     end
590
591     if all_equal
592         println("Recursive and categorical simulations match within tolerance of $eps")
593     end
594
595     return all_equal
596 end
597
598 # Compare the simulation results
599 recursive_file = joinpath(@__DIR__, "money.theory.rec.cat.csv")
600 categorical_file = joinpath(@__DIR__, "money.theory.cat.csv")
601 compare_simulations(recursive_file, categorical_file)

```

A.2.3 Categorical Program

```
1 using DataFrames
2 using CSV
3 using Parameters
4 using Plots
5
6 include("money.theory.rec.cat.jl")
7
8 mutable struct Category
9     name::String                # Category identifier for debugging/visualization
10    objects::Dict{String,Float64} # Economic entities with their current values
11    morphisms::Vector{Dict{String,Any}} # Economic transactions with full metadata
12    next_id::Int                # Unique identifier counter for morphisms
13 end
14
15 function Category(name::String)
16     Category(name, Dict{String,Float64}(), Vector{Dict{String,Any}}(), 1)
17 end
18
19 mutable struct CategoricalFunctor
20     name::String
21     source_category::Category
22     target_category::Category
23     object_mappings::Dict{String,String}
24     morphism_mappings::Dict{String,Function}
25 end
26
27 function CategoricalFunctor(name::String, source::Category, target::Category)
28     CategoricalFunctor(name, source, target, Dict{String,String}(), Dict{String,Function}())
29 end
30
31 function add_mapping!(functor::CategoricalFunctor, source_obj::String, target_obj::String, transform_func::
32     Function)
33     functor.object_mappings[source_obj] = target_obj
34     functor.morphism_mappings[source_obj] = transform_func
35 end
36
37 function apply_functor!(functor::CategoricalFunctor, source_obj::String, context::Dict{String,Any})
38     if haskey(functor.object_mappings, source_obj)
39         target_obj = functor.object_mappings[source_obj]
40         transform_func = functor.morphism_mappings[source_obj]
41
42         source_value = functor.source_category.objects[source_obj]
43         transformed_value = transform_func(source_value, context)
44         functor.target_category.objects[target_obj] = transformed_value
45
46         return transformed_value
47     end
48     return nothing
49 end
50
51 mutable struct CategoricalNaturalTransformation
52     name::String
53     source_functor::CategoricalFunctor
54     target_functor::CategoricalFunctor
55     components::Dict{String,Function}
56 end
57
58 function CategoricalNaturalTransformation(name::String, source_functor::CategoricalFunctor, target_functor::
59     CategoricalFunctor)
60     CategoricalNaturalTransformation(name, source_functor, target_functor, Dict{String,Function}())
61 end
62
63 function add_component!(nat_trans::CategoricalNaturalTransformation, object::String, component_func::Function)
64     nat_trans.components[object] = component_func
65 end
```

```

64
65 function apply_transformation! (nat_trans::CategoricalNaturalTransformation, object::String, context::Dict{
    String,Any})
66     if haskey(nat_trans.components, object)
67         component_func = nat_trans.components[object]
68
69         source_value = nat_trans.source_functor.source_category.objects[object]
70         transformed_value = component_func(source_value, context)
71
72         if haskey(nat_trans.source_functor.object_mappings, object)
73             target_obj = nat_trans.source_functor.object_mappings[object]
74             nat_trans.target_functor.target_category.objects[target_obj] = transformed_value
75         end
76
77         return transformed_value
78     end
79     return nothing
80 end
81
82 mutable struct CategoricalPullback
83     name::String
84     object1::String
85     object2::String
86     category1::Category
87     category2::Category
88     pullback_object::String
89     pullback_category::Category
90     validation_func::Function
91     execution_func::Function
92 end
93
94 function create_pullback(name::String, obj1::String, obj2::String, cat1::Category, cat2::Category,
95     validation_func::Function, execution_func::Function)
96     pullback_cat = Category("$ (name)_pullback")
97     pullback_obj = "$ (obj1)_$ (obj2)_pullback"
98     pullback_cat.objects[pullback_obj] = 0.0
99
100     CategoricalPullback(name, obj1, obj2, cat1, cat2, pullback_obj, pullback_cat, validation_func,
        execution_func)
101 end
102
103 function execute_pullback! (pullback::CategoricalPullback, context::Dict{String,Any})
104     val1 = pullback.category1.objects[pullback.object1]
105     val2 = pullback.category2.objects[pullback.object2]
106
107     if pullback.validation_func(val1, val2, context)
108         result = pullback.execution_func(val1, val2, context)
109         pullback.pullback_category.objects[pullback.pullback_object] = result
110
111         return result
112     end
113
114     return nothing
115 end
116
117 mutable struct CategoricalPushout
118     name::String
119     input_objects::Vector{String}
120     input_categories::Vector{Category}
121     pushout_object::String
122     pushout_category::Category
123     aggregation_func::Function
124     computation_func::Function
125 end
126
127 function create_pushout(name::String, input_objs::Vector{String}, input_cats::Vector{Category},
128     aggregation_func::Function, computation_func::Function)
129     pushout_cat = Category("$ (name)_pushout")
130     pushout_obj = "$ (name)_result"

```

```

131     pushout_cat.objects[pushout_obj] = 0.0
132
133     CategoricalPushout(name, input_objs, input_cats, pushout_obj, pushout_cat, aggregation_func,
134                        computation_func)
135 end
136 function execute_pushout!(pushout::CategoricalPushout, context::Dict{String,Any})
137     input_values = Float64[]
138     for (i, obj) in enumerate(pushout.input_objects)
139         push!(input_values, pushout.input_categories[i].objects[obj])
140     end
141
142     aggregated_value = pushout.aggregation_func(input_values, context)
143
144     result = pushout.computation_func(aggregated_value, context)
145     pushout.pushout_category.objects[pushout.pushout_object] = result
146
147     return result
148 end
149 mutable struct TrueCategoricalState
150     accounts::Category
151     flows::Category
152     parameters::Category
153
154     price_func::CategoricalFunctor
155     flow_func::CategoricalFunctor
156
157     price_adjustment::CategoricalNaturalTransformation
158     flow_aggregation::CategoricalNaturalTransformation
159
160     investment_pullback::CategoricalPullback
161     dividend_pullback::CategoricalPullback
162
163     production_pushout::CategoricalPushout
164
165     wage_hist::Vector{Float64}
166     repay_hist::Vector{Float64}
167 end
168
169 function create_categorical_state(state::State)
170     accounts = Category("Accounts")
171     flows = Category("Flows")
172     parameters = Category("Parameters")
173
174     accounts.objects["Labor_Bank"] = 0.0
175     accounts.objects["Labor_Stock"] = 0.0
176     accounts.objects["Labor_Good"] = 0.0
177
178     accounts.objects["Resource_Bank"] = 0.0
179     accounts.objects["Resource_Stock"] = 0.0
180     accounts.objects["Resource_Good"] = 0.0
181
182     accounts.objects["Company_Bank"] = 0.0
183     accounts.objects["Company_Loan"] = 0.0
184     accounts.objects["Company_Dividend"] = 0.0
185     accounts.objects["Company_Resource"] = 20.0
186     accounts.objects["Company_Labor"] = 110.0
187     accounts.objects["Company_Good"] = 0.0
188
189     accounts.objects["Capital_Bank"] = 0.0
190     accounts.objects["Capital_Dividend"] = 0.0
191     accounts.objects["Capital_Good"] = 0.0
192
193     accounts.objects["Bank_Loan"] = 0.0
194     accounts.objects["Bank_Company"] = 0.0
195     accounts.objects["Bank_Labor"] = 0.0
196     accounts.objects["Bank_Resource"] = 0.0
197     accounts.objects["Bank_Capital"] = 0.0
198

```

```

199 pars = state.Parameters
200 for field in fieldnames(typeof(pars))
201     parameters.objects[String(field)] = Float64(getfield(pars, field))
202 end
203
204 price_functor = CategoricalFunctor("PriceFunctor", accounts, flows)
205
206 flow_functor = CategoricalFunctor("FlowFunctor", accounts, flows)
207
208 add_mapping!(price_functor, "Labor_Bank", "Labor_Real",
209     (nominal, ctx) -> nominal / get(ctx, "labor_price", 1.0))
210 add_mapping!(price_functor, "Resource_Bank", "Resource_Real",
211     (nominal, ctx) -> nominal / get(ctx, "resource_price", 1.0))
212 add_mapping!(price_functor, "Company_Bank", "Company_Real",
213     (nominal, ctx) -> nominal / get(ctx, "good_price", 1.0))
214 add_mapping!(price_functor, "Capital_Bank", "Capital_Real",
215     (nominal, ctx) -> nominal / get(ctx, "good_price", 1.0))
216
217 add_mapping!(flow_functor, "Labor_Bank", "Labor_Flow",
218     (amount, ctx) -> amount * get(ctx, "labor_velocity", 1.0))
219 add_mapping!(flow_functor, "Resource_Bank", "Resource_Flow",
220     (amount, ctx) -> amount * get(ctx, "resource_velocity", 1.0))
221 add_mapping!(flow_functor, "Company_Bank", "Company_Flow",
222     (amount, ctx) -> amount * get(ctx, "company_velocity", 1.0))
223
224 price_adjustment = CategoricalNaturalTransformation("PriceAdjustment", price_functor, price_functor)
225
226 flow_aggregation = CategoricalNaturalTransformation("FlowAggregation", flow_functor, flow_functor)
227
228 add_component!(price_adjustment, "Labor_Bank",
229     (value, ctx) -> value * get(ctx, "price_evolution", 1.0))
230 add_component!(price_adjustment, "Resource_Bank",
231     (value, ctx) -> value * get(ctx, "price_evolution", 1.0))
232 add_component!(price_adjustment, "Company_Bank",
233     (value, ctx) -> value * get(ctx, "price_evolution", 1.0))
234
235 # FLOW AGGREGATION COMPONENTS: How each flow category evolves
236 add_component!(flow_aggregation, "Labor_Bank",
237     (value, ctx) -> value + get(ctx, "labor_injection", 0.0))
238 add_component!(flow_aggregation, "Resource_Bank",
239     (value, ctx) -> value + get(ctx, "resource_injection", 0.0))
240 add_component!(flow_aggregation, "Company_Bank",
241     (value, ctx) -> value + get(ctx, "company_injection", 0.0))
242
243 investment_pullback = create_pullback("Investment", "Company_Bank", "Bank_Loan", accounts, accounts,
244     (com_bank, bank_loan, ctx) -> true, # Investment can create money through banking
245     (com_bank, bank_loan, ctx) -> get(ctx, "investment_amount", 0.0)
246 )
247
248 dividend_pullback = create_pullback("Dividend", "Company_Bank", "Capital_Dividend", accounts, accounts,
249     (com_bank, cap_div, ctx) -> com_bank >= get(ctx, "dividend_amount", 0.0),
250     (com_bank, cap_div, ctx) -> get(ctx, "dividend_amount", 0.0)
251 )
252
253 production_pushout = create_pushout("Production",
254     ["Company_Labor", "Company_Resource"],
255     [accounts, accounts],
256     (values, ctx) -> (values[1], values[2]),
257     (values, ctx) -> 1 + get(ctx, "scale_prod", 0.0) *
258         values[1]^get(ctx, "lab_exp", 0.5) *
259         values[2]^(1 - get(ctx, "lab_exp", 0.5))
260 )
261
262 flows.objects["Labor_Real"] = 0.0
263 flows.objects["Resource_Real"] = 0.0
264 flows.objects["Company_Real"] = 0.0
265 flows.objects["Capital_Real"] = 0.0
266 flows.objects["Labor_Flow"] = 0.0
267 flows.objects["Resource_Flow"] = 0.0

```

```

268 flows.objects["Company_Flow"] = 0.0
269
270 return TrueCategoricalState(
271     accounts, flows, parameters,
272     price_functor, flow_functor,
273     price_adjustment, flow_aggregation,
274     investment_pullback, dividend_pullback,
275     production_pushout,
276     zeros(Int(parameters.objects["InvestmentLen"])),
277     zeros(Int(parameters.objects["InvestmentLen"]))
278 )
279 end
280
281 function categorical_state_transition(sim::DataFrame, cat_state::TrueCategoricalState, period::Int64)
282     new_cat_state = deepcopy(cat_state)
283
284     pars = new_cat_state.parameters.objects
285
286     p2H(hist, newelem) = [newelem; hist[1:end-1]]
287
288
289     cap_bank = new_cat_state.accounts.objects["Capital_Bank"]
290     bank_cap = new_cat_state.accounts.objects["Bank_Capital"]
291     InvCapBank = cap_bank - bank_cap
292
293     res_bank = new_cat_state.accounts.objects["Resource_Bank"]
294     bank_res = new_cat_state.accounts.objects["Bank_Resource"]
295     InvResBank = res_bank - bank_res
296
297     com_bank = new_cat_state.accounts.objects["Company_Bank"]
298     bank_com = new_cat_state.accounts.objects["Bank_Company"]
299     InvComBank = com_bank - bank_com
300
301     bank_loan = new_cat_state.accounts.objects["Bank_Loan"]
302     com_loan = new_cat_state.accounts.objects["Company_Loan"]
303     InvComLoan = bank_loan - com_loan
304
305     lab_bank = new_cat_state.accounts.objects["Labor_Bank"]
306     bank_lab = new_cat_state.accounts.objects["Bank_Labor"]
307     InvLabBank = lab_bank - bank_lab
308
309     InvMacro = InvCapBank + InvResBank + InvComBank + InvComLoan + InvLabBank
310
311     WagesPayment = sum(new_cat_state.wage_hist)
312     RepaysPayment = sum(new_cat_state.repay_hist)
313
314     ConsumRes = res_bank * pars["ConsumRatioRes"]
315     ConsumLab = lab_bank * pars["ConsumRatioLab"]
316     ConsumCap = cap_bank * pars["ConsumRatioCap"]
317
318     Demand = ConsumRes + ConsumLab + ConsumCap
319
320     prod_context = Dict{String,Any} (
321         "scale_prod" => pars["ScaleProd"],
322         "lab_exp" => pars["LabResSubstProd"]
323     )
324     GoodProduction = execute_pushout!(new_cat_state.production_pushout, prod_context)
325
326     DemandPlan = (WagesPayment + RepaysPayment) * (1 + pars["MarkUp"])
327
328     DemandSurplus = Demand - DemandPlan
329
330     GoodPrice = ((period == 0 ? pars["InitialGoodPrice"] : 0.0) +
331         DemandPlan / GoodProduction +
332         (DemandSurplus > 0.0 ? (DemandSurplus * pars["WindFallProfit"]) : 0.0))
333
334     Investment = pars["sigA"] + pars["sigB"] / (1.0 + exp(-DemandSurplus / pars["sigC"]))
335     InvestmentRes = Investment * (1 - pars["LabResourceRatio"])
336     InvestmentLab = Investment * pars["LabResourceRatio"]

```



```

337 Repayment = Investment / pars["InvestmentLen"]
338
339 new_cat_state.wage_hist = p2H(cat_state.wage_hist, InvestmentLab)
340 new_cat_state.repay_hist = p2H(cat_state.repay_hist, Repayment)
341
342 price_context = Dict{String,Any} (
343     "labor_price" => pars["LaborPrice"],
344     "resource_price" => pars["ResourcePrice"],
345     "good_price" => GoodPrice
346 )
347 apply_functor!(new_cat_state.price_functor, "Labor_Bank", price_context)
348 apply_functor!(new_cat_state.price_functor, "Resource_Bank", price_context)
349 apply_functor!(new_cat_state.price_functor, "Company_Bank", price_context)
350
351 evolution_context = Dict{String,Any} (
352     "price_evolution" => GoodPrice / pars["InitialGoodPrice"],
353     "labor_injection" => InvestmentLab,
354     "resource_injection" => InvestmentRes,
355     "company_injection" => Investment
356 )
357
358 apply_transformation!(new_cat_state.price_adjustment, "Labor_Bank", evolution_context)
359 apply_transformation!(new_cat_state.price_adjustment, "Resource_Bank", evolution_context)
360 apply_transformation!(new_cat_state.price_adjustment, "Company_Bank", evolution_context)
361
362 apply_transformation!(new_cat_state.flow_aggregation, "Labor_Bank", evolution_context)
363 apply_transformation!(new_cat_state.flow_aggregation, "Resource_Bank", evolution_context)
364 apply_transformation!(new_cat_state.flow_aggregation, "Company_Bank", evolution_context)
365
366 DividendPayment = new_cat_state.accounts.objects["Company_Dividend"]
367 inAccComBank = Investment + ConsumLab + ConsumRes + ConsumCap
368 outAccComBank = InvestmentRes + WagesPayment + DividendPayment + RepaysPayment
369 Diff = inAccComBank - outAccComBank
370
371 DividendDecision = (Diff > 0.0 ? (Diff * pars["DivRateDiff"]) : 0.0) +
372     ((com_bank > 0) ? (com_bank * pars["DivRateBank"]) : 0.0)
373
374 DividendIncome = new_cat_state.accounts.objects["Capital_Dividend"]
375
376 investment_context = Dict{String,Any} ("investment_amount" => Investment)
377 investment_valid = execute_pullback!(new_cat_state.investment_pullback, investment_context)
378
379 dividend_context = Dict{String,Any} ("dividend_amount" => DividendDecision)
380 dividend_valid = execute_pullback!(new_cat_state.dividend_pullback, dividend_context)
381
382 new_cat_state.accounts.objects["Company_Bank"] = com_bank + Diff
383 new_cat_state.accounts.objects["Company_Dividend"] = new_cat_state.accounts.objects["Company_Dividend"] +
384     DividendDecision - DividendPayment
385 new_cat_state.accounts.objects["Company_Loan"] = com_loan + Investment - RepaysPayment
386 new_cat_state.accounts.objects["Company_Resource"] = InvestmentRes / pars["ResourcePrice"]
387 new_cat_state.accounts.objects["Company_Labor"] = WagesPayment / pars["LaborPrice"]
388 new_cat_state.accounts.objects["Company_Good"] = new_cat_state.accounts.objects["Company_Good"] +
389     GoodProduction - (ConsumRes + ConsumLab + ConsumCap) / GoodPrice
390
391 new_cat_state.accounts.objects["Labor_Bank"] = lab_bank + WagesPayment - ConsumLab
392 new_cat_state.accounts.objects["Labor_Stock"] = new_cat_state.accounts.objects["Labor_Stock"] + pars["
393     ReNewLab"] - WagesPayment / pars["LaborPrice"]
394 new_cat_state.accounts.objects["Labor_Good"] = new_cat_state.accounts.objects["Labor_Good"] + ConsumLab /
395     GoodPrice - new_cat_state.accounts.objects["Labor_Good"] * pars["DecayGoodLab"]
396
397 new_cat_state.accounts.objects["Resource_Bank"] = res_bank + InvestmentRes - ConsumRes
398 new_cat_state.accounts.objects["Resource_Stock"] = new_cat_state.accounts.objects["Resource_Stock"] + pars["
399     ReNewRes"] - InvestmentRes / pars["ResourcePrice"]
400 new_cat_state.accounts.objects["Resource_Good"] = new_cat_state.accounts.objects["Resource_Good"] +
401     ConsumRes / GoodPrice - new_cat_state.accounts.objects["Resource_Good"] * pars["DecayGoodRes"]
402
403 new_cat_state.accounts.objects["Capital_Dividend"] = new_cat_state.accounts.objects["Capital_Dividend"] +
404     DividendDecision - DividendIncome
405 new_cat_state.accounts.objects["Capital_Bank"] = cap_bank + DividendIncome - ConsumCap

```

```

399 new_cat_state.accounts.objects["Capital_Good"] = new_cat_state.accounts.objects["Capital_Good"] + ConsumCap
400 / GoodPrice - new_cat_state.accounts.objects["Capital_Good"] * pars["DecayGoodCap"]
401
402 new_cat_state.accounts.objects["Bank_Loan"] = bank_loan + Investment - RepaysPayment
403 new_cat_state.accounts.objects["Bank_Company"] = bank_com + Investment + ConsumRes + ConsumLab + ConsumCap
404 - (InvestmentRes + WagesPayment + DividendPayment + RepaysPayment)
405 new_cat_state.accounts.objects["Bank_Labor"] = bank_lab + WagesPayment - ConsumLab
406 new_cat_state.accounts.objects["Bank_Resource"] = bank_res + InvestmentRes - ConsumRes
407 new_cat_state.accounts.objects["Bank_Capital"] = bank_cap + DividendIncome - ConsumCap
408
409 push!(sim, (Period=period,
410 InvCapBank=InvCapBank,
411 InvResBank=InvResBank,
412 InvComBank=InvComBank,
413 InvComLoan=InvComLoan,
414 InvLabBank=InvLabBank,
415 InvMacro=InvMacro,
416 ConsumRes=ConsumRes,
417 ConsumLab=ConsumLab,
418 ConsumCap=ConsumCap,
419 Demand=Demand,
420 DemandPlan=DemandPlan,
421 DemandSurplus=DemandSurplus,
422 GoodProduction=GoodProduction,
423 GoodPrice=GoodPrice,
424 Investment=Investment,
425 InvestmentRes=InvestmentRes,
426 InvestmentLab=InvestmentLab,
427 Repayment=Repayment,
428 WagesPayment=WagesPayment,
429 RepaysPayment=RepaysPayment,
430 Diff=Diff,
431 DividendDecision=DividendDecision,
432 DividendPayment=DividendPayment,
433 DividendIncome=DividendIncome,
434 AccLabBank=cat_state.accounts.objects["Labor_Bank"],
435 AccLabLab=cat_state.accounts.objects["Labor_Stock"],
436 AccLabGood=cat_state.accounts.objects["Labor_Good"],
437 AccResBank=cat_state.accounts.objects["Resource_Bank"],
438 AccResRes=cat_state.accounts.objects["Resource_Stock"],
439 AccResGood=cat_state.accounts.objects["Resource_Good"],
440 AccCapBank=cat_state.accounts.objects["Capital_Bank"],
441 AccCapDiv=cat_state.accounts.objects["Capital_Dividend"],
442 AccCapGood=cat_state.accounts.objects["Capital_Good"],
443 AccComBank=cat_state.accounts.objects["Company_Bank"],
444 AccComLoan=cat_state.accounts.objects["Company_Loan"],
445 AccComDiv=cat_state.accounts.objects["Company_Dividend"],
446 AccComRes=cat_state.accounts.objects["Company_Resource"],
447 AccComLab=cat_state.accounts.objects["Company_Labor"],
448 AccComGood=cat_state.accounts.objects["Company_Good"],
449 AccBankComLoan=cat_state.accounts.objects["Bank_Loan"],
450 AccBankComBank=cat_state.accounts.objects["Bank_Company"],
451 AccBankLabBank=cat_state.accounts.objects["Bank_Labor"],
452 AccBankResBank=cat_state.accounts.objects["Bank_Resource"],
453 AccBankCapBank=cat_state.accounts.objects["Bank_Capital"])
454
455 return sim, new_cat_state
456 end
457
458
459
460 function run_categorical_simulation(nperiods::Int64, output_file::String)
461     println("Starting truly categorical simulation...")
462
463     sim = DataFrame()
464     state = State(Parameters=SystemParams())
465     cat_state = create_categorical_state(state)

```

```

466 for period in 0:nperiods
467     sim, cat_state = categorical_state_transition(sim, cat_state, period)
468
469     if period % 10 == 0
470         println("Period $period completed")
471     end
472 end
473
474
475 eps = 1e-12
476 for col in names(sim)
477     if eltype(sim[:, col]) <: Number
478         sim[:, col] = map(x -> abs(x) < eps ? 0.0 : x, sim[:, col])
479     end
480 end
481
482 CSV.write(output_file, sim)
483 println("Categorical simulation results saved to: $output_file")
484
485 create_truly_categorical_time_series_plots(sim, replace(output_file, ".csv" => ".plots.png"))
486
487 return sim
488 end
489
490 function create_truly_categorical_time_series_plots(sim::DataFrame, output_file::String)
491     println("Creating truly categorical time series plots...")
492
493     var_names = filter(x -> x != :Period, names(sim))
494     n_vars = length(var_names)
495
496     n_cols = ceil{Int, sqrt}(n_vars)
497     n_rows = ceil{Int, n_vars / n_cols}
498
499     plots_array = []
500
501     for (i, var) in enumerate(var_names)
502         is_inv = string(var) in ["InvCapBank", "InvResBank", "InvComBank", "InvComLoan", "InvLabBank", "
503             InvMacro"]
504
505         y_lims = if is_inv
506             max_abs = maximum(abs.(sim[:, var]))
507             if max_abs == 0.0
508                 (-1.0, 1.0)
509             else
510                 (-max_abs * 1.1, max_abs * 1.1)
511             end
512         else
513             min_val = minimum(sim[:, var])
514             max_val = maximum(sim[:, var])
515             range_val = max_val - min_val
516             if range_val == 0.0
517                 (min_val - 1.0, max_val + 1.0)
518             else
519                 padding = range_val * 0.1
520                 (min_val - padding, max_val + padding)
521             end
522         end
523
524         plot_args = (sim.Period, sim[:, var])
525         plot_kwargs = Dict(
526             :title => string(var),
527             :xlabel => "Period",
528             :ylabel => string(var),
529             :legend => false,
530             :linewidth => 2,
531             :titlefontsize => 8,
532             :labelfontsize => 6,
533             :tickfontsize => 5,
534             :color => is_inv ? :red : :blue,

```

```

534         :ylims => y_ylims
535     )
536
537     p = plot(plot_args...; plot_kwargs...)
538
539     if y_ylims[1] <= 0 <= y_ylims[2]
540         hline!(p, [0], color=:black, linestyle=:dash, alpha=0.5)
541     end
542
543     push!(plots_array, p)
544 end
545
546 combined_plot = plot(plots_array...,
547     layout=(n_rows, n_cols),
548     size=(1600, 1200),
549     margin=2Plots.mm
550 )
551
552 savefig(combined_plot, output_file)
553 println("Truly categorical time series plots saved to: $output_file")
554 end
555
556 function verify_categorical_implementation()
557     println("="^80)
558     println("TRULY CATEGORICAL IMPLEMENTATION VERIFICATION")
559     println("="^80)
560
561     recursive_file = joinpath(@__DIR__, "money.theory.rec.cat.csv")
562     pseudo_categorical_file = joinpath(@__DIR__, "money.theory.cat.csv")
563     true_categorical_file = joinpath(@__DIR__, "money.theory.cat.sim.csv")
564
565     run_categorical_simulation(100, true_categorical_file)
566
567     recursive_df = CSV.read(recursive_file, DataFrame)
568     pseudo_categorical_df = CSV.read(pseudo_categorical_file, DataFrame)
569     true_categorical_df = CSV.read(true_categorical_file, DataFrame)
570
571     eps = 1e-10
572     all_match = true
573
574     println("\nNumerical comparison (tolerance: $eps):")
575
576     for col in names(recursive_df)
577         max_diff_pseudo = maximum(abs.(recursive_df[:, col] - pseudo_categorical_df[:, col]))
578         max_diff_true = maximum(abs.(recursive_df[:, col] - true_categorical_df[:, col]))
579         max_diff_pseudo_true = maximum(abs.(pseudo_categorical_df[:, col] - true_categorical_df[:, col]))
580
581         if max_diff_pseudo > eps || max_diff_true > eps || max_diff_pseudo_true > eps
582             println(" $col: max_diff(rec->pseudo)=$max_diff_pseudo, max_diff(rec->>true)=$max_diff_true,
583                 max_diff(pseudo->>true)=$max_diff_pseudo_true")
584             all_match = false
585         else
586             println(" $col: All implementations match")
587         end
588     end
589
590 # VERIFICATION RESULT REPORTING: Summarize findings and implications
591 if all_match
592     println("\n SUCCESS: categorical implementation produces identical results!")
593     println(" - Production pushout performs actual Cobb-Douglas calculation")
594     println(" - Investment pullback validates monetary creation")
595     println(" - Dividend pullback validates wealth distribution")
596     println(" - Price functor transforms nominal to real values")
597     println(" - Natural transformations evolve system temporally")
598     println(" - Categorical parameter category replaces SystemParams()")
599 else
600     println("\n FAILURE: Implementations produce different results")
601     println(" - Check categorical implementation for computational errors")
602     println(" - Verify pullback/pushout execution logic")

```

```

602     println("    - Examine functor application consistency")
603     println("    - Review natural transformation components")
604 end
605
606 return all_match
607 end
608
609 if abspath(PROGRAM_FILE) == @__FILE__
610     println("Executing categorical economic simulation...")
611     println("\n COMPARE IMPLEMENTATION:")
612     println("    - Prove categorical structures can perform actual economic computation")
613     println("    - Maintain identical numerical results across all implementations")
614
615     println("")
616
617     verify_categorical_implementation()
618 end

```

B References

- Samson Abramsky and Bob Coecke. A categorical semantics of quantum protocols. *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science (LICS)*, 2004. doi: 10.1109/LICS.2004.1319636.
- Samson Abramsky and Viktor Winschel. Coalgebraic analysis of subgame-perfect equilibria in infinite games without discounting. *Mathematical Structures in Computer Science*, 27(5):751–761, 2015. ISSN 0960-1295, 1469-8072. doi: 10.1017/s0960129515000365.
- Samson Abramsky, John C. Baez, Fabio Gadducci, and Viktor Winschel. Categorical methods at the crossroads (dagstuhl perspectives workshop 14182), 2014. URL <https://drops.dagstuhl.de/entities/document/10.4230/DagRep.4.4.49>.
- Samson Abramsky, Alexander Kurz, Pierre Lescanne, and Viktor Winschel. Coalgebraic semantics of reflexive economics (dagstuhl seminar 15042), 2015. URL <https://drops.dagstuhl.de/entities/document/10.4230/DagRep.5.1.197>.
- Bob Coecke and Aleks Kissinger. *Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning*. Cambridge University Press, Cambridge, UK, 2017. ISBN 978-1-107-10422-8.
- Andrée C. Ehresmann and Jean-Paul Vanbremeersch. *Memory Evolutive Systems; Hierarchy, Emergence, Cognition*. Elsevier Science, 2007. ISBN 9780444560919.
- Bernd Fischer and Johann Schumann. AutoBayes: a system for generating data analysis programs from statistical models. 13(3):508, 483, 2003.
- Brendan Fong, David I. Spivak, and Rémy Tuyéras. Backprop as functor: A compositional perspective on supervised learning, 2019. URL <http://arxiv.org/abs/1711.10455>.
- Nicholas Georgescu-Roegen. *The entropy law and the economic process*. Harvard University Press, 1971. ISBN 978-0-674-25780-1.
- Neil Ghani, Jules Hedges, Viktor Winschel, and Philipp Zahn. Compositional game theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*, pages 472–481. ACM, 2018. doi: 10.1145/3209108.3209165. URL <https://dl.acm.org/doi/10.1145/3209108.3209165>.
- Michèle Giry. A categorical approach to probability theory. In *Categorical Aspects of Topology and Analysis*, pages 68–85. Springer, 1982. doi: 10.1007/BFb0092872.
- Daniel Harenberg, Stefano Marelli, Bruno Sudret, and Viktor Winschel. Uncertainty quantification and global sensitivity analysis for economic models. *Quantitative Economics*, 10(1):1–41, 2019. ISSN 1759-7323. doi: 10.3982/qe866. URL <http://qeconomics.org/ojs/index.php/qe/article/view/640>. Publisher: The Econometric Society.
- Florian Heiss and Viktor Winschel. Likelihood approximation by numerical integration on sparse grids. *Journal of Econometrics*, 144(1):62–80, 2008. ISSN 0304-4076. doi: 10.1016/j.jeconom.2007.12.004. URL <https://linkinghub.elsevier.com/retrieve/pii/S0304407607002552>. Publisher: Elsevier BV.
- International Monetary Fund, Organization for Economic Cooperation and Development, United Nations, European Commission, and World Bank. *System of National Accounts 2008*. International Monetary Fund, 2009. ISBN 978-92-1-161522-7.
- Piergiulio Katis, N. Sabadini, and R. F. C. Walters. On partita doppia, 2008. URL <http://arxiv.org/abs/0803.2429>.
- Erwin Klein. *Economic Theories and Their Relational Structures: A Model-theoretic Characterization*. Palgrave Macmillan, 1998. ISBN 0-333-68792-2. URL https://www.google.de/books/edition/Economic_Theories_and_their_Relational_S/XH-GDAAAQBAJ?hl=en&gbpv=0.

- Renée Menéndez. *Die Organisationsstrukturen des Kreditgeldkapitalismus oder Theorie der bootstrap-Ökonomie: ein Beitrag zur Rekonstruktion der deutschen Nationalökonomie*. phdthesis, Free University of Berlin, 1989. URL <https://stabikat.de/Record/275143929?sid=8100118>.
- Renée Menéndez. *Theorie des Kapitalismus als Sozialwissenschaft: Prolegomena zu einer sozialen Zinstheorie*. Number Bd. 179 in Politikwissenschaft. Lit, 2010. ISBN 978-3-643-10901-9.
- Renée Menéndez and Viktor Winschel. Monetary macro accounting theory, 2025. URL <https://arxiv.org/abs/2506.21651>.
- Elinor Ostrom. *Understanding Institutional Diversity*. Princeton University Press, Princeton, New Jersey, 2005. ISBN 9780691122380. doi: 10.1515/9781400831739.
- Evan Patterson, Owen Lynch, and James Fairbanks. Categorical data structures for technical computing. *Compositionality*, 4(5), 2022. doi: 10.32408/compositionality-4-5.
- Salvador Cruz Rambaud, José García Pérez, Robert A Nehmer, and Derek J S Robinson. *Algebraic Models for Accounting Systems*. World Scientific, 2010. ISBN 978-981-4287-11-1. URL <https://books.google.de/books?id=6tZpDQAAQBAJ>.
- J. J. M. M. Rutten. Universal coalgebra: a theory of systems. *Theoretical Computer Science*, 249(1):3–80, 2000. ISSN 0304-3975. doi: [https://doi.org/10.1016/S0304-3975\(00\)00056-6](https://doi.org/10.1016/S0304-3975(00)00056-6). URL <https://www.sciencedirect.com/science/article/pii/S0304397500000566>.
- August-Wilhelm Scheer. *The Composable Enterprise: Agile, Flexible, Innovative: A Gamechanger for Organisations, Digitisation and Business Software*. Springer Fachmedien Wiesbaden, 2024. ISBN 978-3-658-43088-7 978-3-658-43089-4. doi: 10.1007/978-3-658-43089-4. URL <https://link.springer.com/10.1007/978-3-658-43089-4>.
- Peter Selinger. A survey of graphical languages for monoidal categories. 2009. URL <http://arxiv.org/abs/0908.3347>.
- Peter Selinger. A survey of graphical languages for monoidal categories. *New Structures for Physics*, 813: 289–355, 2011. doi: 10.1007/978-3-642-12821-9_4.
- Jan C. Willems. The behavioral approach to open and interconnected systems. 27(6):46–99, 2007. ISSN 1941-000X. doi: 10.1109/MCS.2007.906923. URL <https://ieeexplore.ieee.org/document/4384643>. Conference Name: IEEE Control Systems Magazine.
- Viktor Winschel. *The Empirical Analysis of Exchange Rate Regimes and Nonlinear Structural Econometrics*. phdthesis, University of Mannheim, 2005.
- Viktor Winschel and Markus Krätzig. Solving, estimating, and selecting nonlinear dynamic models without the curse of dimensionality. *Econometrica*, 78(2):803–821, 2010. ISSN 0012-9682. doi: 10.3982/ecta6297. URL <http://doi.wiley.com/10.3982/ECTA6297>. Publisher: The Econometric Society.